

高级搜索树

伸展树：逐层伸展

When we stand on the tops of Things
And like the Trees, look down
The smoke all cleared away from it
And Mirrors on the scene
Just laying light

我要一步一步往上爬，在最高点乘着叶片往前飞

邓俊辉

deng@tsinghua.edu.cn

局部性/Locality: 刚被访问过的数据, 极有可能很快地再次被访问

❖ 这一现象在信息处理过程中屡见不鲜...

[Pfaff-2004] Performance Analysis Of BSTs In System Software

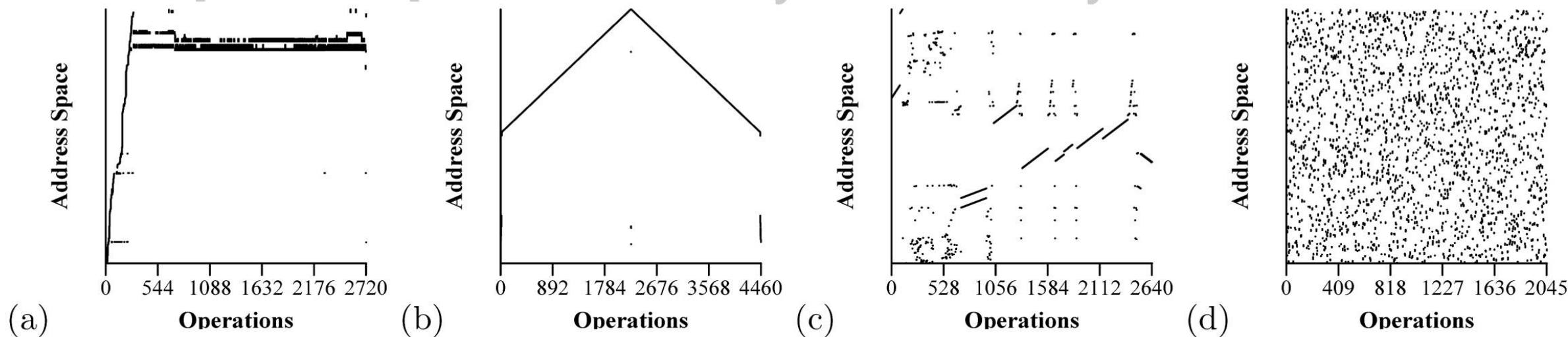


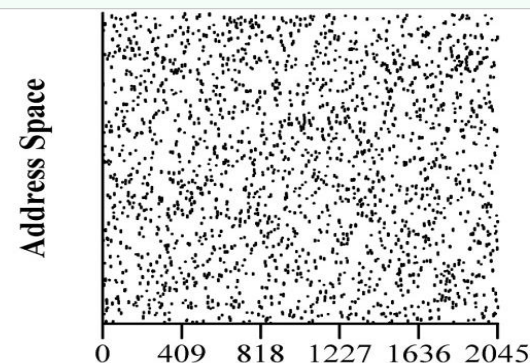
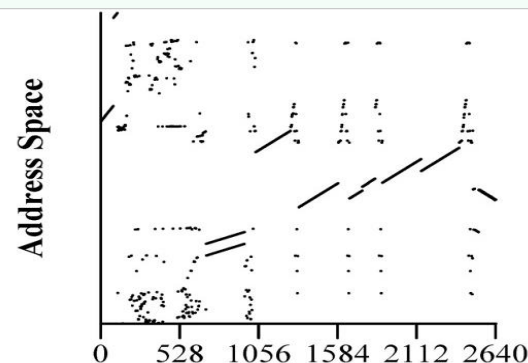
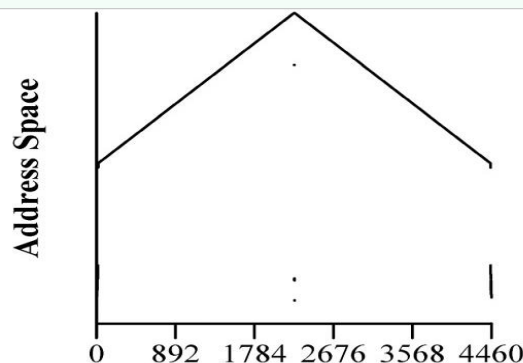
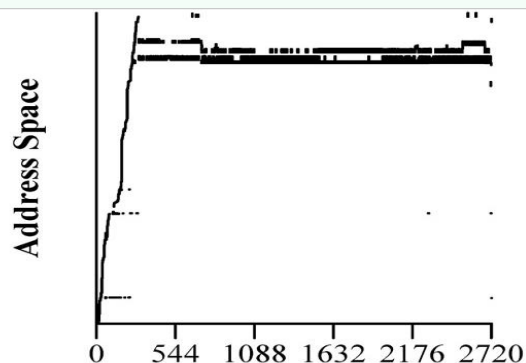
Figure 2: Call sequences in (a) Mozilla 1.0, (b) VMware GSX Server 2.0.1, (c) squid running under User-Mode Linux 2.4.18.48, and (d) random test sets. Part (b) omits one `mmap-munmap` pair for memory region `0x20000000` to `0x30000000` and (c) omits address space gaps; the others are complete.

BST的局部性

❖ 时间：刚被访问过的**节点**，极有可能**很快地再次**被访问

空间：下一将要访问的节点，极有可能就在刚被访问过节点的**附近**

❖ 对AVL**连续的** m 次查找 ($m \gg n$)，共需 $O(m \cdot \log n)$ 时间——能否利用局部性加速？



❖ 自适应链表：节点一旦被访问，随即移动到**最前端**

模仿：BST的节点一旦被访问，随即调整到**树根**

❖ 难点：如何实现这种**调整**？调整过程自身的**复杂度**如何控制？

逐层伸展

❖ 节点v一旦被访问

随即被**推送**至根

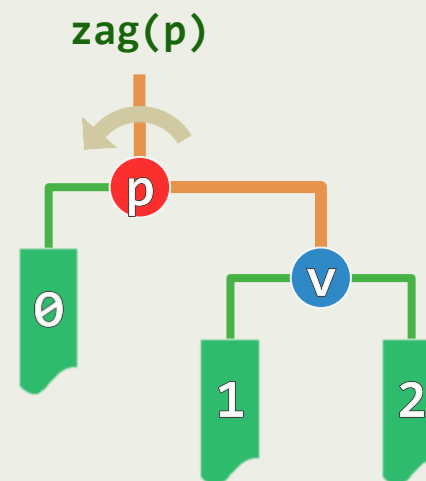
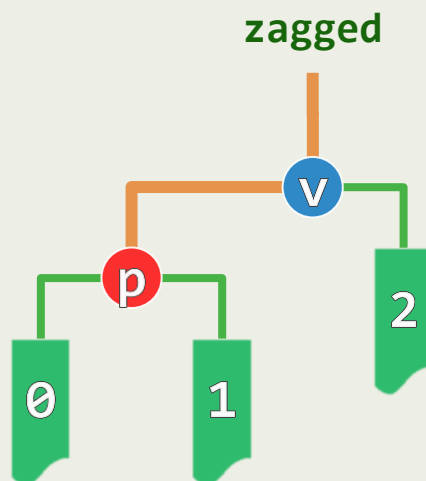
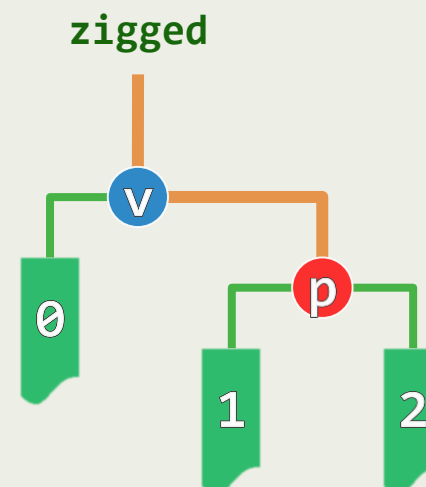
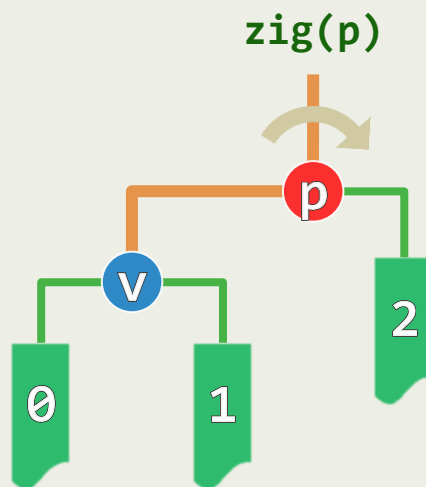
❖ 与其说“推”，不如说“爬”

一步一步地往上爬

❖ 自下而上，逐层**旋转**

- zig(v→parent)

- zag(v→parent)



实例

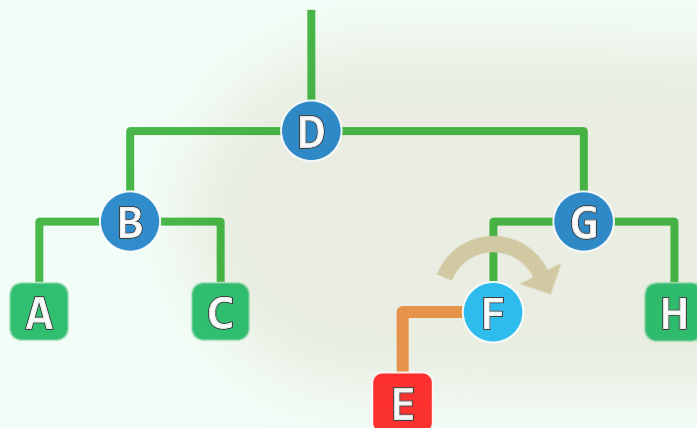
❖ 伸展过程的效率

是否足够地高?

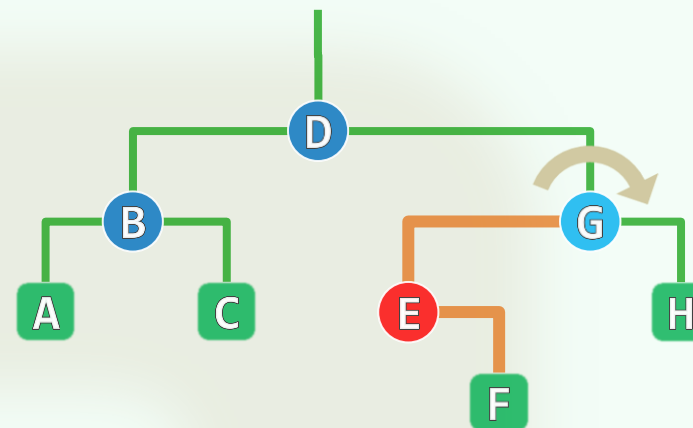
❖ 这取决于

- 树的初始形态和
- 节点的访问次序

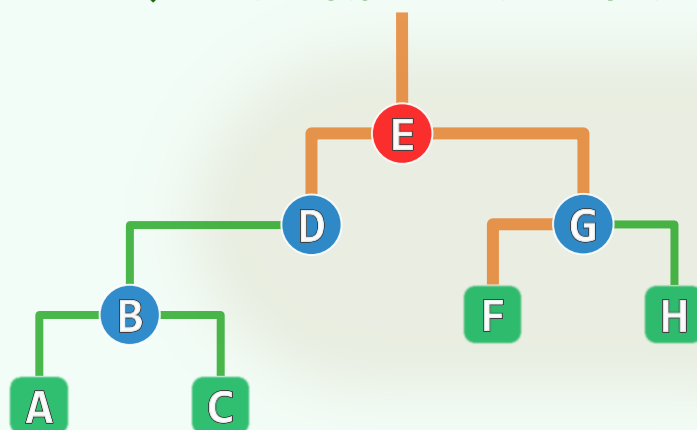
a) 访问E之后, 做zig(F)



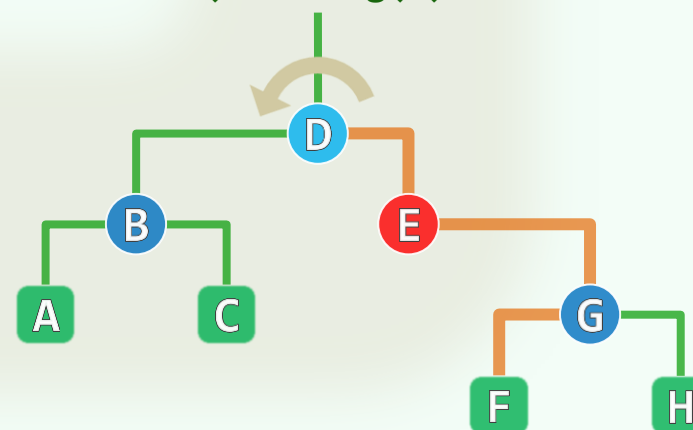
b) 继而zig(G)



d) 经3次旋转, E最终调整至树根



c) 继而zag(D)



最坏情况：死蛇挂树

❖ 树高

始终不低于 $n/2$

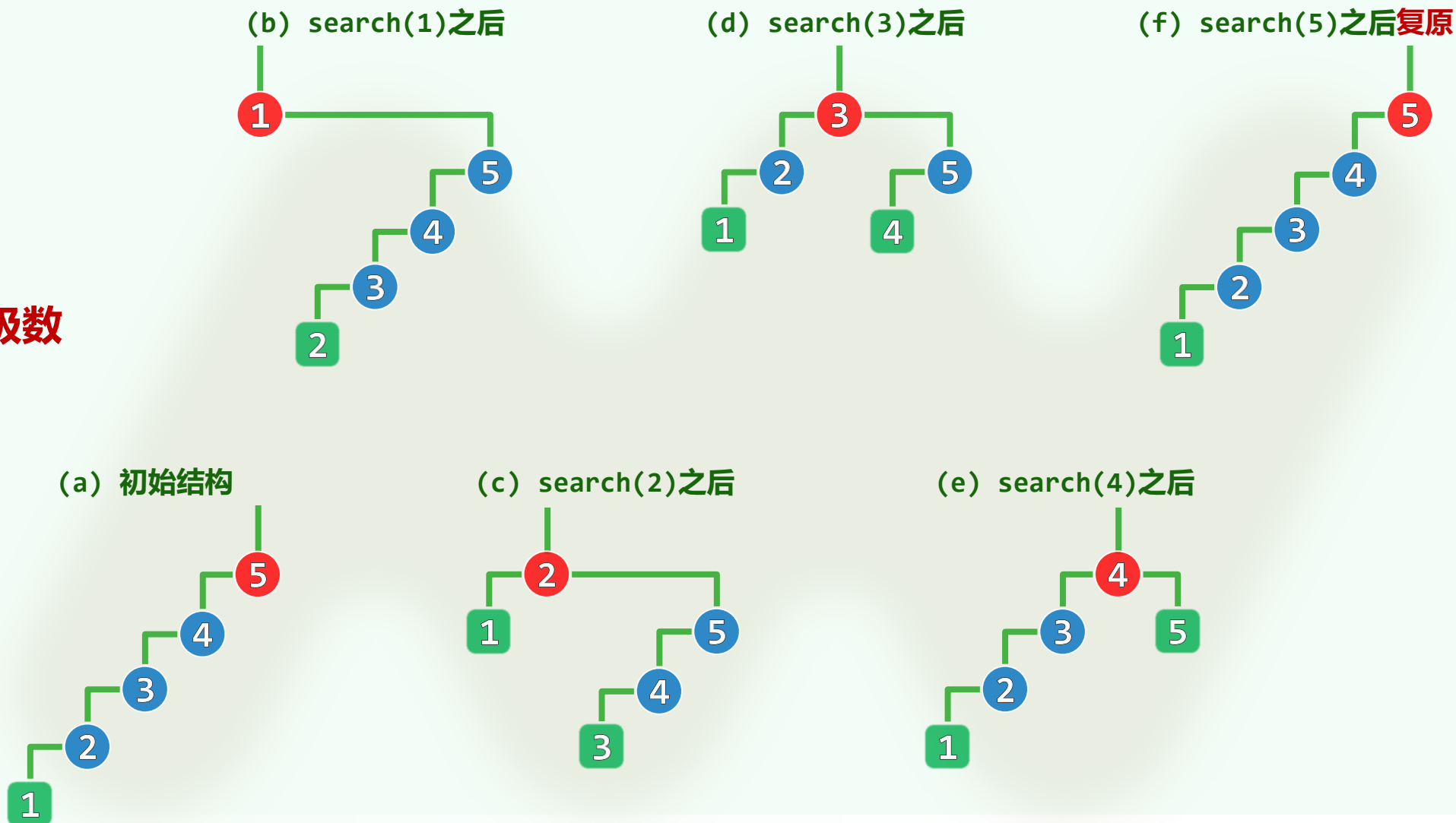
❖ 旋转次数

呈周期性的算术级数

❖ 每一周期

- 累计 $\Omega(n^2)$
- 均摊 $\Omega(n)$

❖ 怎么破?



08-A2

高级搜索树

伸展树：双层伸展

與時遷移，應物變化；立俗施事，無所不宜；指約而易操，事少而功多

賈政道：“不用全打開，怕疊起來倒費事。”

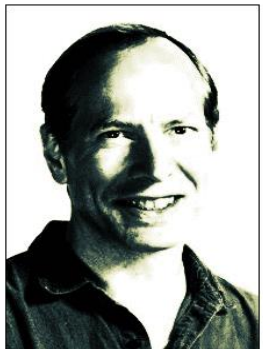
詹光便與馮紫英一層一層折好收拾

邓俊辉

deng@tsinghua.edu.cn

双层伸展

❖ Self-Adjusting Binary Trees

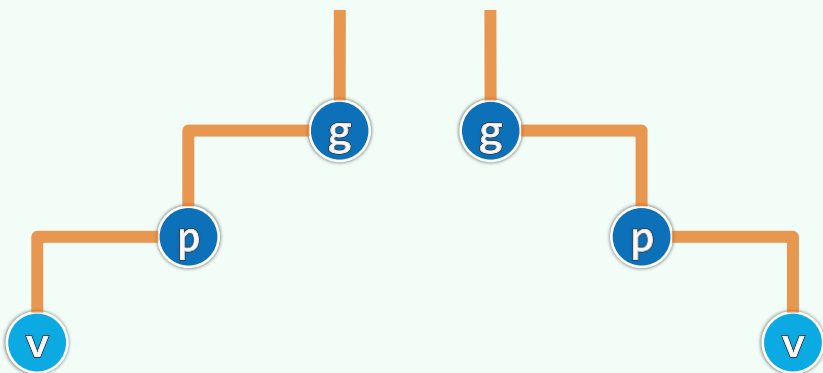


D. D. Sleator

R. E. Tarjan

J. ACM, 32:652-686, 1985

❖ 构思的精髓：向上追溯两层，而非一层



❖ 反复考察祖孙三代：

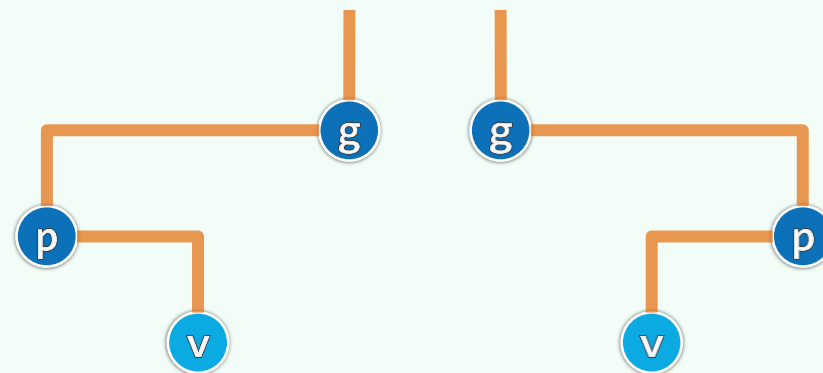
$g = \text{parent}(p)$, $p = \text{parent}(v)$, v

❖ 根据它们的相对位置，经两次旋转

使 v 上升两层，成为（子）树根

❖ 如此，性能的确会有改善？

❖ 具体地，应该如何旋转？



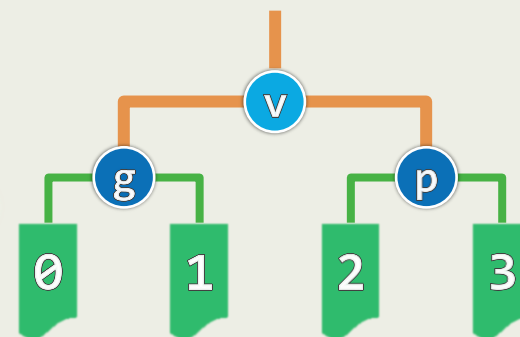
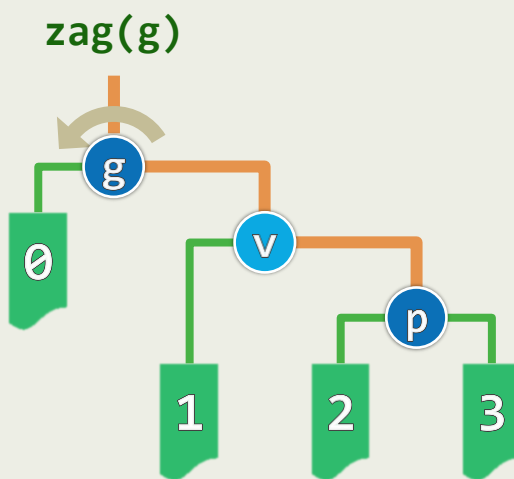
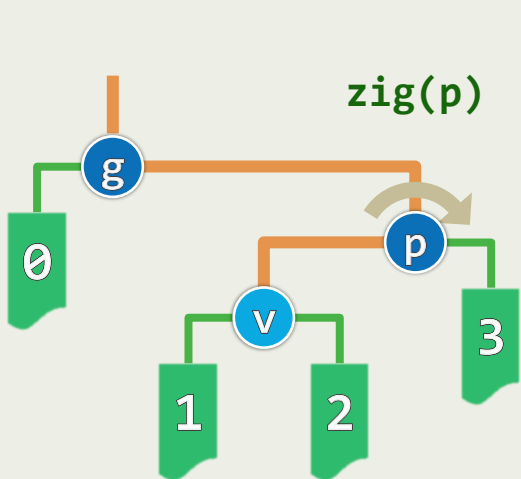
zig-zag | zag-zig

❖ 此时的v按中序遍历次序居中

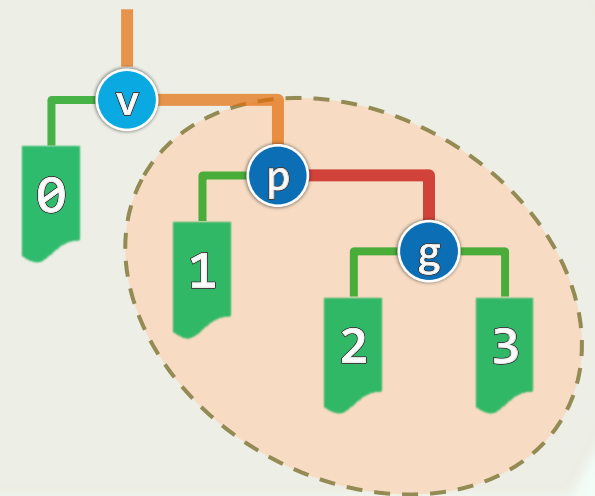
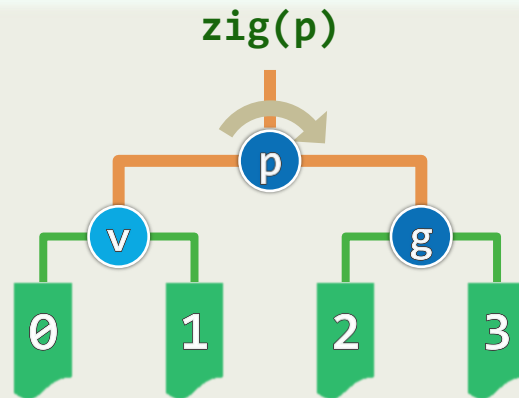
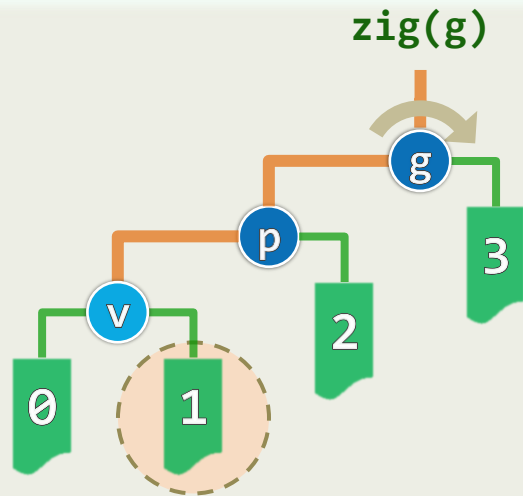
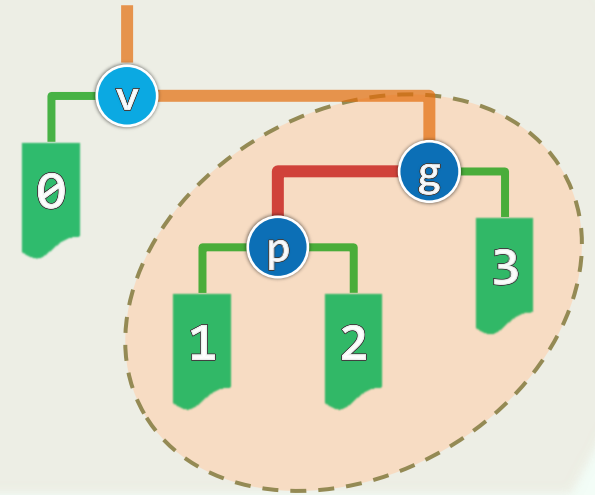
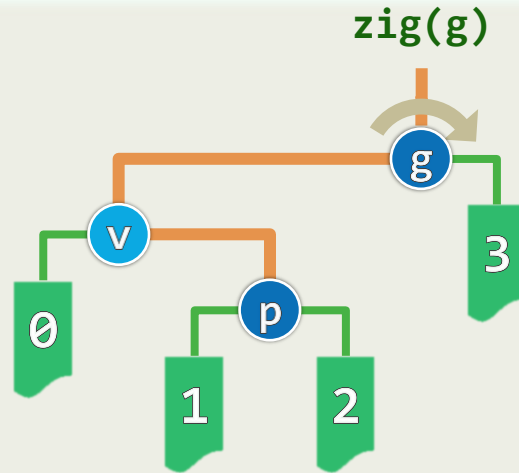
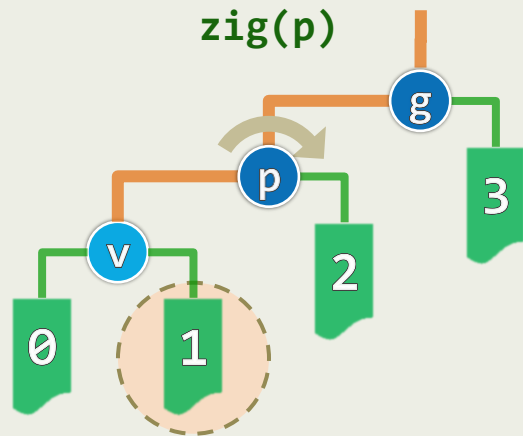
❖ 故若欲使之成为根，最终无非一种姿势

❖ 如此调整的效果，与逐层调整别无二致！

❖ 难道，就这样...平淡无奇？



zig-zig | zag-zag



zig-zig / zag-zag

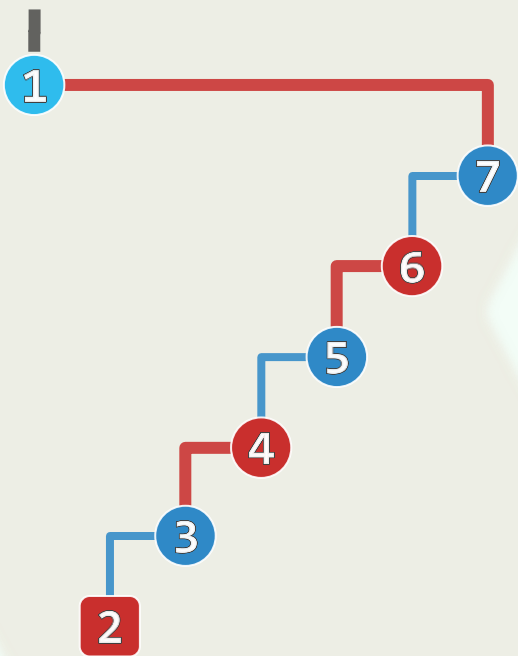
❖ 节点访问之后，对应路径的长度随即**折半**

❖ 最坏情况不致**持续**发生！

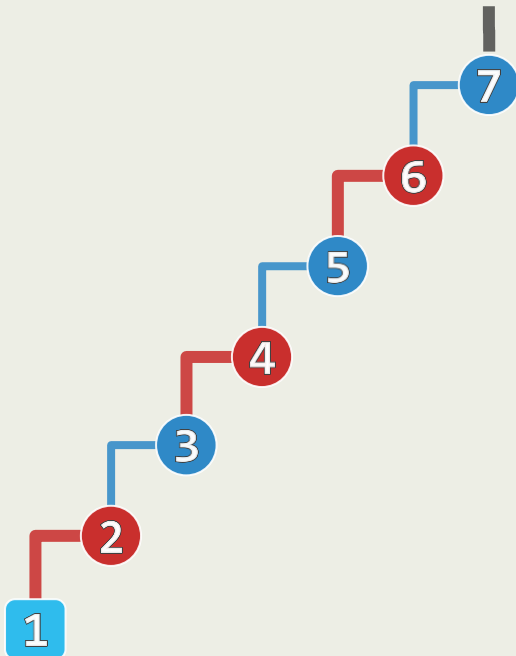
//含羞草般的**折叠**效果

习题[8-2]：伸展操作**均摊**仅需 $\mathcal{O}(\log n)$ 时间

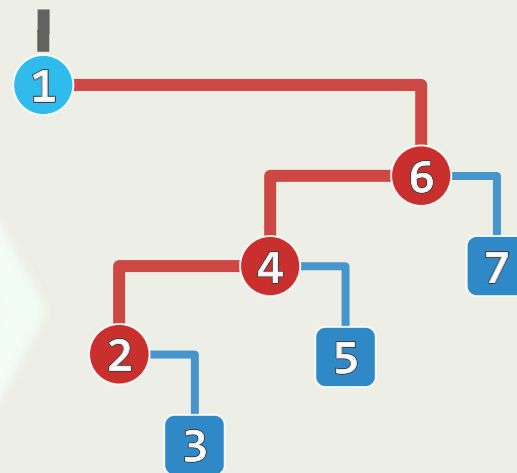
逐层调整



search(1)



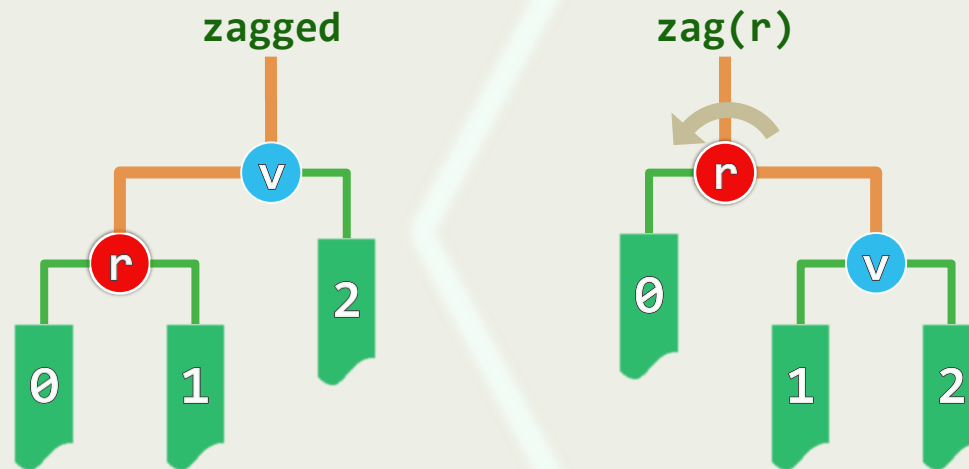
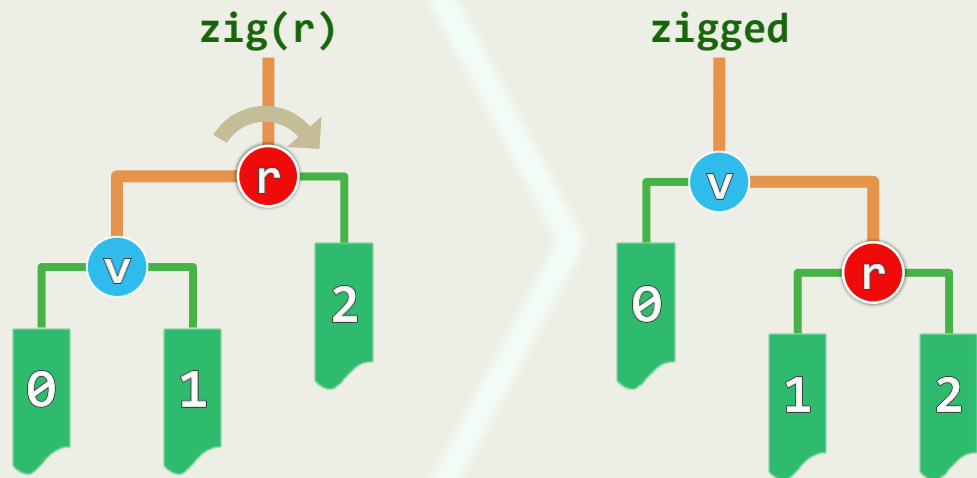
双层调整



zig | zag

❖ 要是v只有父亲，没有祖父呢？

❖ 此时必有： $v.parent() == T.root()$



❖ 只需做**单次**旋转：zig(r)或zag(r)

❖ 好在，这种情况至多（在最后）出现**一次**

08-A3

高级搜索树

伸展树：算法实现

邓俊辉

deng@tsinghua.edu.cn

到了所在，住了腳，便把這驢似紙一般折疊
起來，其厚也只比張紙，放在巾箱裡面

接口

```
template <typename T> class Splay : public BST<T> { //由BST派生
```

```
protected:
```

```
    BNP<T> splay( BNP<T> ); //将v伸展至根
```

```
public: //伸展树的查找也会引起整树的结构调整, 故search()也需重写
```

```
    BNP<T> search( const T& ); //查找 (重写)
```

```
    BNP<T> insert( const T& ); //插入 (重写)
```

```
    bool remove( const T& ); //删除 (重写)
```

```
}; //Splay
```

伸展算法：总体思路

```
template <typename T> BNP<T> Splay<T>::splay( BNP<T> v ) {  
    while ( BNP<T> p = v->parent ) //自下而上  
        if ( BNP<T> g = p->parent ) { //双层伸展  
            if ( BNP<T> gg = v->parent = g->parent ) //调整之后  
                g == gg->lc ? gg->attachLc(v) : gg->attachRc(v); //v应以原曾祖父为父  
  
            switch ( ((p == g->rc) << 1) | (v == p->rc) )  
            { /* 视p、v之拐向，分四种情况 */ }  
  
            g->updateHeight(); p->updateHeight();  
        } else { /*单层伸展：只可能出现在最后一步*/ }  
    (_root = v)->updateHeight(); return v; //v终成树根  
}  
//splay
```

伸展算法：双层伸展

switch (((p == g->rc) << 1) | (v == p->rc)) { //视p、v之拐向，分四种情况

case 0b00 : g->attachLc(p->rc); /*2*/

p->attachRc(g);

p->attachLc(v->rc); /*1*/

v->attachRc(p);

break; //zig-zig

case 0b01 : p->attachRc(v->lc); g->attachLc(v->rc); //zig-zag

v->attachRc(g); v->attachLc(p); break;

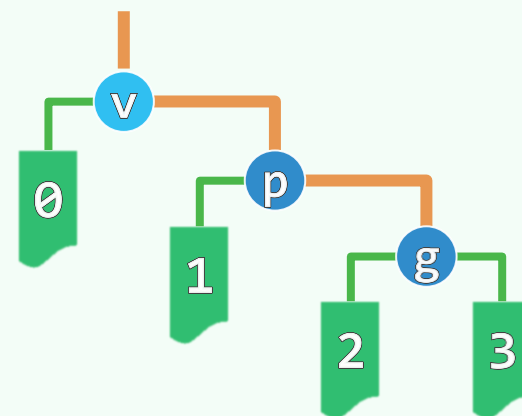
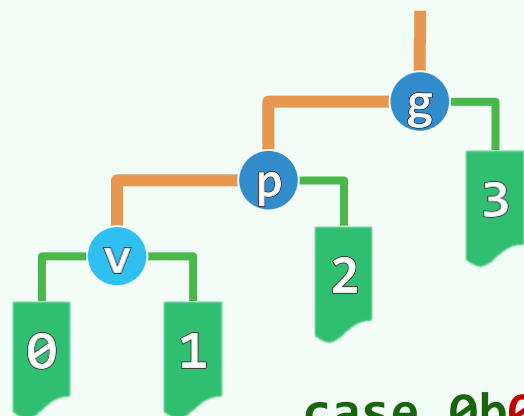
case 0b10 : p->attachLc(v->rc); g->attachRc(v->lc); //zag-zig

v->attachLc(g); v->attachRc(p); break;

default/*0b11*/: g->attachRc(p->lc); p->attachRc(v->lc); //zag-zag

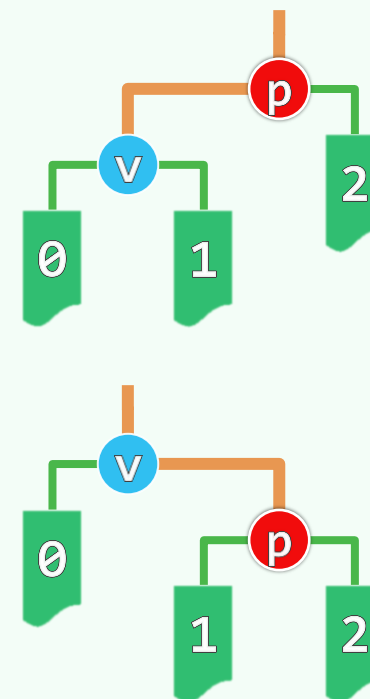
p->attachLc(g); v->attachLc(p); break;

}



伸展算法：单层伸展

```
if ( BNP<T> g = p->parent ) {  
    /* 双层伸展 */  
}  
else { //单层伸展：只可能出现在最后一步  
    if ( v == p->lc )  
        { p->attachLc( v->rc ); v->attachRc( p ); }  
    else  
        { p->attachRc( v->lc ); v->attachLc( p ); }  
    p->updateHeight(); v->parent = NULL;  
} //else
```



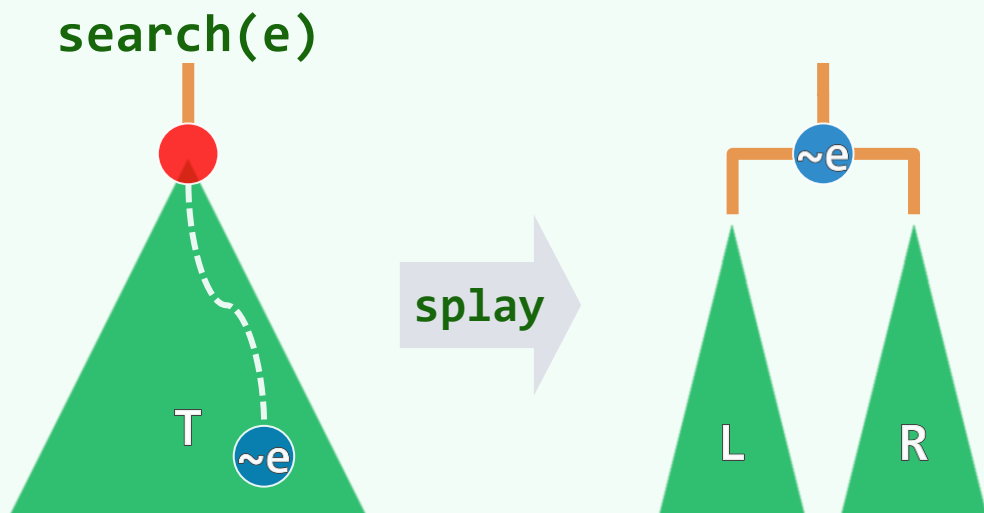
查找算法

```
❖ template <typename T> BNP<T> Splay<T>::search( const T & e ) {  
    BST::search( e ); //按BST标准算法查找: 无论结果如何, 最后抵达的节点  
    return _x ? splay(_x) : _p ? splay(_p) : NULL; //都将随即伸展为_root  
}
```

❖ 伸展树的查找, 与常规BST::search()不同:
很可能会改变树的结构, 不再属于静态操作

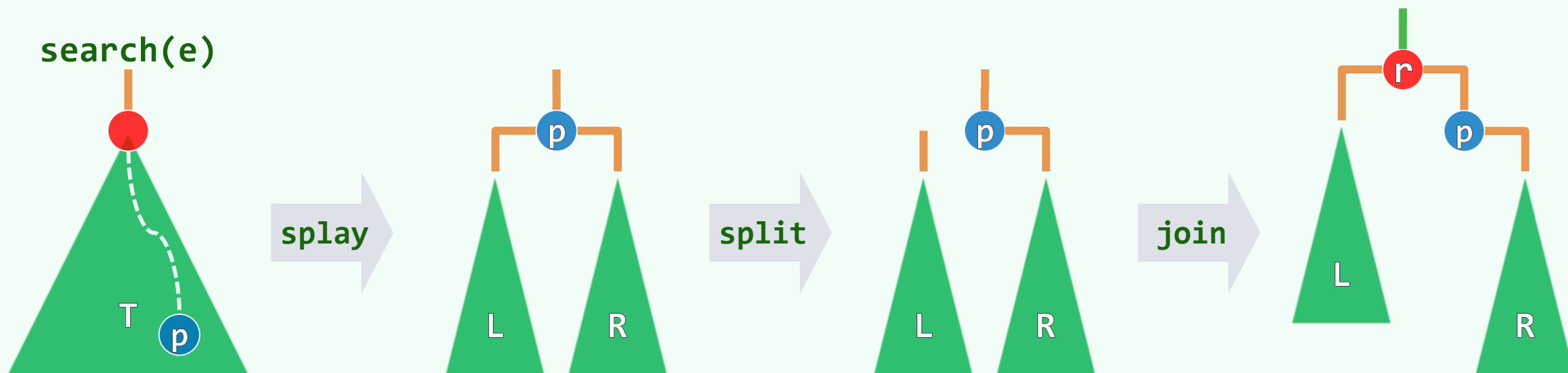
❖ 无论如何, 返回节点皆合情理:

- 成功 (新根 _x->data == e)
- 失败 (临近者 _p->data ~ e)
- 空树 (NULL)



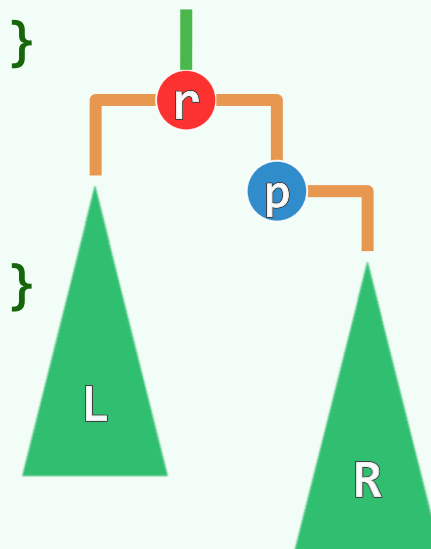
插入算法

- ❖ 无论如何，首先需做 `Splay::search(e)` —— WLOG，设查找失败
e 的临近者 `_p` 将被伸展至根 —— WLOG，设 $e < _p \rightarrow \text{data}$
- ❖ 创建新的根节点 `_root = BinNode<T>( e, NULL, _p→lc, _p )`
- ❖ 如此，接下来不仅 `_root` 会便于访问，`_p` 也会 —— 可更充分地利用空间局部性



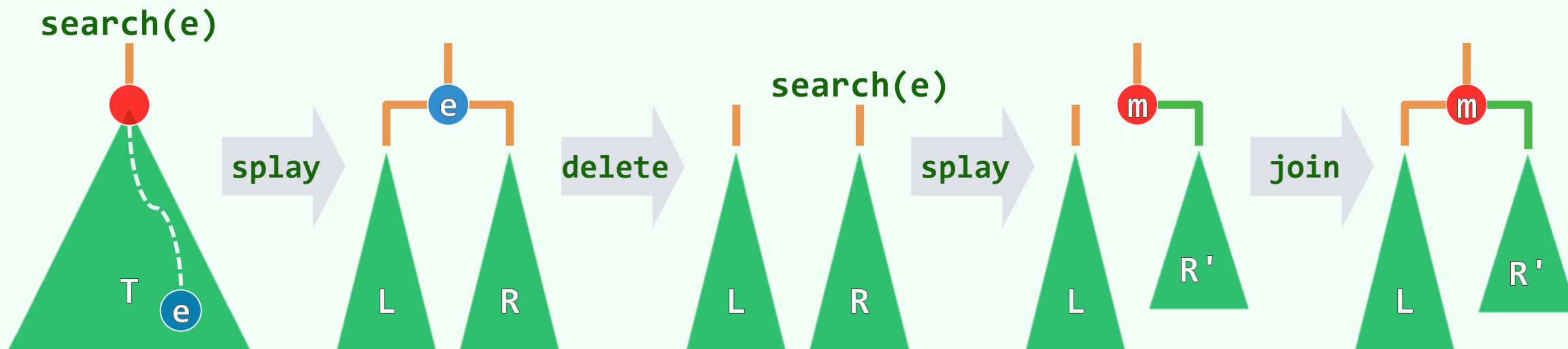
插入算法实现

```
template <typename T> BNP<T> Splay<T>::insert( const T & e ) {  
    if ( !_root ) { _size = 1; return _root = new BinNode<T>( e ); }  
    search(e); if ( e == _root->data ) return _root; //查找成功 (否则, _p已伸展至根)  
    if ( _p->data < e ) //在右侧嫁接 (rc或为空, lc == _p必非空)  
    { _root = new BinNode<T>(e, NULL, _p, _p->rc); _p->rc = NULL; }  
    else //e < _p->data: 在左侧嫁接 (lc或为空, rc == _p必非空)  
    { _root = new BinNode<T>(e, NULL, _p->lc, _p); _p->lc = NULL; }  
    _p->updateHeightAbove(); _size++; return _root;  
} //insert
```



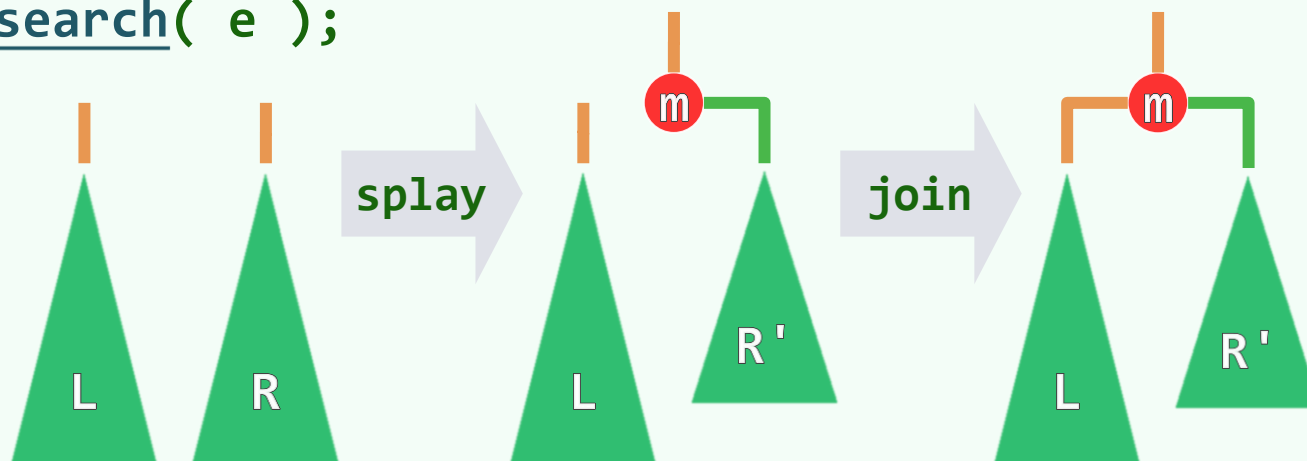
删除算法

- ❖ 无论如何，首先需做 `Splay::search(e)` —— WLOG，设查找成功
树根即是目标节点 $v \rightarrow \text{data} == e$ —— WLOG，设右子树 R 非空
- ❖ 在 R 中再次查找 e ，其中最小的节点 m 将会伸展为 `_root`，且无左孩子：令 L 作为其左子树
- ❖ 如此，接下来不仅 m 会便于访问， m 此前的祖先们也会 —— 可更充分地利用空间局部性



删除算法实现

```
template <typename T> bool Splay<T>::remove( const T & e ) {  
    search(e); if ( !_root || e != _root->data ) return false; //反之, e必已伸展至根  
    BNP<T> L = _root->lc, R = _root->rc; delete _root; _size--; //删  
    if ( !R ) { if ( _root = L ) _root->parent = NULL; } //若R空, 则L即是余树  
    else { //否则, 再次查找必败, 但R的最小节点会伸展为_root, 且无左孩子; 故可令L为其左子树  
        (_root = R)->parent = NULL; search( e );  
        _root->attachLc(L);  
        _root->updateHeight();  
    }  
    return true;  
}
```



$\Theta(8-A4)$

高级搜索树

伸展树：均摊分析

轉圓石於千仞之山者，勢也

所谓物价，其实就是我称之为生命的那部分，必须在交换时支付：要么立即支付，要么以后支付

邓俊辉

deng@tsinghua.edu.cn

S的势能

❖ 任何一棵伸展树在任何时刻，都可以**假想地**视作具有势能：

$$\Phi(S) = \log \left(\prod_{v \in S} \text{size}(v) \right) = \sum_{v \in S} \log(\text{size}(v)) = \sum_{v \in S} \text{rank}(v) = \sum_{v \in S} \log V$$

❖ 直觉：越**平衡/倾侧**的树，势能越**小/大**

- 单链： $\Phi(S) = \log n! = \mathcal{O}(n \log n)$

- 满树： $\Phi(S) = \log \prod_{d=0}^h (2^{h-d+1} - 1)^{2^d} \leq \log \prod_{d=0}^h (2^{h-d+1})^{2^d}$

$$= \log \prod_{d=0}^h 2^{(h-d+1) \cdot 2^d} = \sum_{d=0}^h (h-d+1) \cdot 2^d = (h+1) \cdot \sum_{d=0}^h 2^d - \sum_{d=0}^h d \cdot 2^d$$

$$= (h+1) \cdot (2^{h+1} - 1) - [(h-1) \cdot 2^{h+1} + 2] = 2^{h+2} - h - 3 = \mathcal{O}(n)$$

T的上界

❖ 考查对伸展树 S 的 $m \gg n$ 次连续访问（不妨仅考查 `search()`）

❖ 若记： $A^{(k)} = T^{(k)} + \Delta\Phi^{(k)}$, $k = 0, 1, 2, \dots, m$

则有： $A - \mathcal{O}(n \log n) \leq \underline{T} = \underline{A - \Delta\Phi} \leq A + \mathcal{O}(n \log n)$

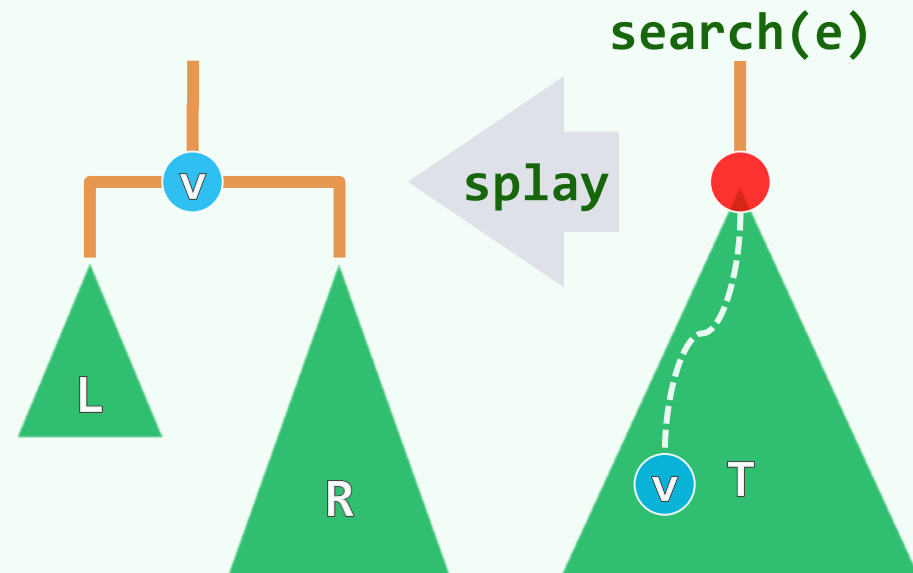
❖ 故若能证明： $A = \mathcal{O}(m \log n)$

则必有： $T = \mathcal{O}(m \log n)$

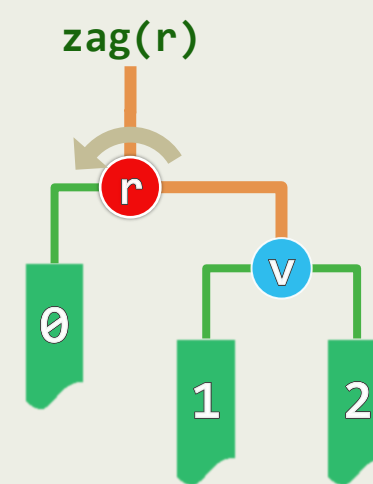
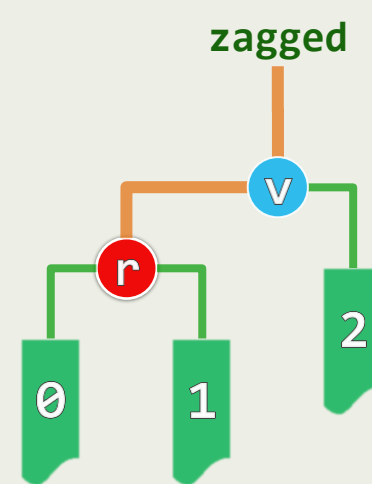
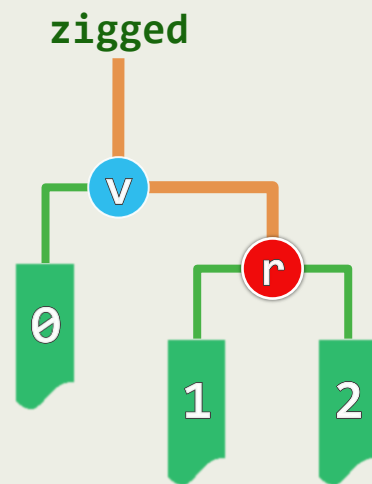
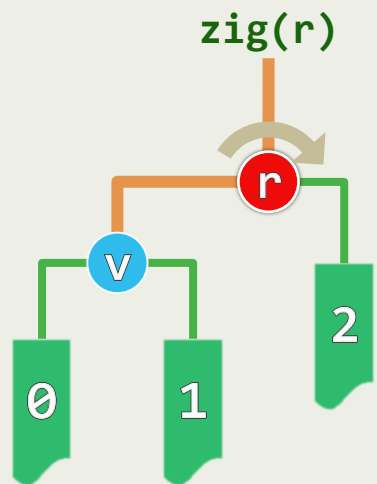
❖ 好消息是，尽管 $T^{(k)}$ 的变化幅度可能很大，我们却能证明：

$A^{(k)}$ 都不致超过被伸展节点 v 的势能变化量，即： $\mathcal{O}(rank^{(k)}(v) - rank^{(k-1)}(v)) = \mathcal{O}(\log n)$

❖ 事实上， $A^{(k)}$ 不过是 v 的若干次连续伸展操作（时间成本）的累积，这些操作无非三种情况...

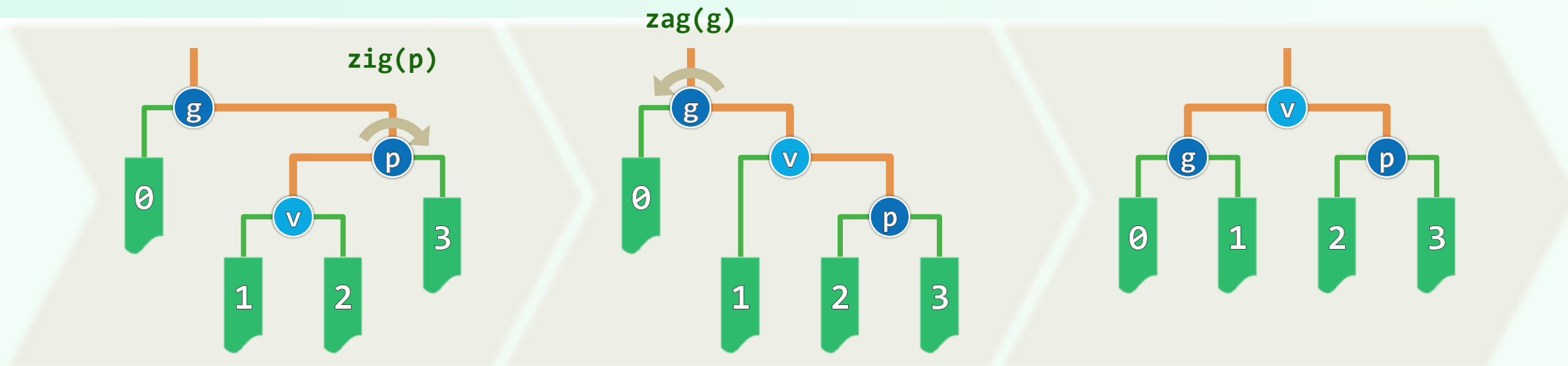


Zig / Zag



$$\begin{aligned}
 A_i^{(k)} &= T_i^{(k)} + \Delta\Phi(S_i^{(k)}) = 1 + \Delta\text{rank}_i(v) + \Delta\text{rank}_i(r) \\
 &= 1 + [\text{rank}_i(v) - \text{rank}_{i-1}(v)] + \underline{[\text{rank}_i(r) - \text{rank}_{i-1}(r)]} \\
 &< 1 + [\text{rank}_i(v) - \text{rank}_{i-1}(v)]
 \end{aligned}$$

zig-zag / zag-zig



$$A_i^{(k)} = T_i^{(k)} + \Delta\Phi(S_i^{(k)}) = \mathbf{2} + \Delta\text{rank}_i(g) + \Delta\text{rank}_i(p) + \Delta\text{rank}_i(v)$$

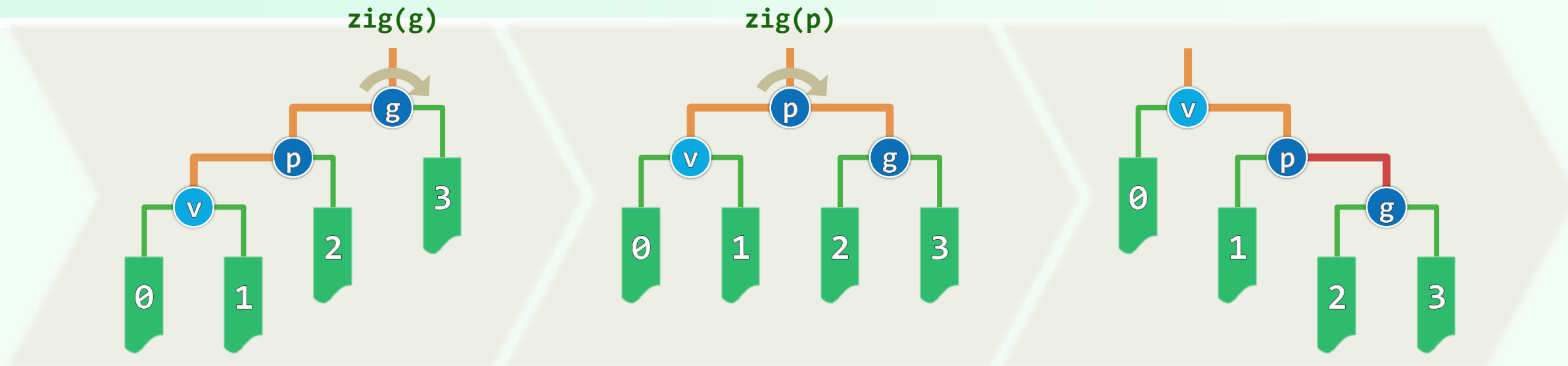
$$= 2 + [\text{rank}_i(g) - \cancel{\text{rank}_{i-1}(g)}] + [\text{rank}_i(p) - \underline{\text{rank}_{i-1}(p)}] + [\cancel{\text{rank}_i(v)} - \underline{\text{rank}_{i-1}(v)}]$$

$$< 2 + \underline{\text{rank}_i(g) + \text{rank}_i(p)} - \mathbf{2} \cdot \text{rank}_{i-1}(v) \quad \boxed{\because 0 \leq \text{rank}_{i-1}(v) < \text{rank}_{i-1}(p)}$$

$$< \cancel{2} + \underline{\mathbf{2} \cdot \text{rank}_i(v)} - \cancel{2} - 2 \cdot \text{rank}_{i-1}(v) \quad \boxed{\because (\log G_i + \log P_i)/2 \leq \log((G_i + P_i)/\mathbf{2}) < \log(V_i/2)}$$

$$= \mathbf{2} \cdot (\text{rank}_i(v) - \text{rank}_{i-1}(v))$$

zig-zig / zag-zag



$$A_i^{(k)} = T_i^{(k)} + \Delta\Phi(S_i^{(k)}) = \mathbf{2} + \Delta\text{rank}_i(g) + \Delta\text{rank}_i(p) + \Delta\text{rank}_i(v)$$

$$= 2 + [\text{rank}_i(g) - \cancel{\text{rank}_{i-1}(g)}] + [\text{rank}_i(p) - \underline{\text{rank}_{i-1}(p)}] + [\cancel{\text{rank}_i(v)} - \underline{\text{rank}_{i-1}(v)}]$$

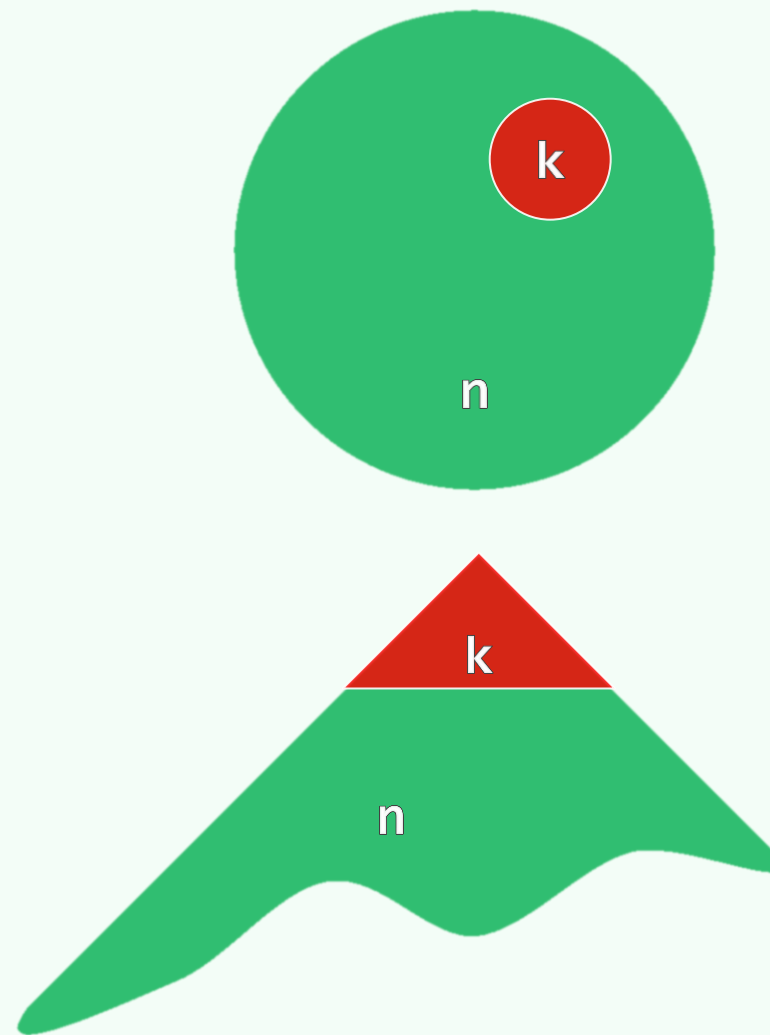
$$< 2 + \underline{\text{rank}_i(g) + \text{rank}_i(p)} - \mathbf{2} \cdot \text{rank}_{i-1}(v) \quad \boxed{\because 0 \leq \text{rank}_{i-1}(v) < \text{rank}_{i-1}(p)}$$

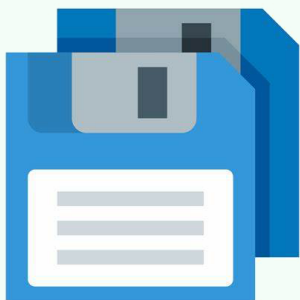
$$< 2 + \text{rank}_i(g) + \underline{\text{rank}_i(\mathbf{v})} - 2 \cdot \text{rank}_{i-1}(v) \quad \boxed{\because 0 \leq \text{rank}_i(p) < \text{rank}_i(v)}$$

$$< \mathbf{3} \cdot (\text{rank}_i(v) - \text{rank}_{i-1}(v)) \quad \boxed{\because (\log G_i + \log V_{i-1})/2 \leq \log((G_i + V_{i-1})/\mathbf{2}) < \log(V_i/2)}$$

综合评价

- ❖ 无需记录高度或平衡因子；编程实现简单——优于AVL树
均摊复杂度 $\mathcal{O}(\log n)$ —— 与AVL树相当
- ❖ 局部性强、缓存命中率极高时（即 $k \ll n \ll m$ ）
 - 效率甚至可以更高 —— 自适应的 $\mathcal{O}(\log k)$
 - 任何**连续的m次**查找，仅需 $\mathcal{O}(m \log k + n \log n)$ 时间
- ❖ 若**反复地顺序**访问任一子集，均摊成本仅为**常数**
——**单层**伸展的最**坏**情况，成了**双层**伸展的最好情况
- ❖ 不能杜绝**单次**最坏情况，不适用于对**瞬时**效率敏感的场所
- ❖ 复杂度的**分析**有点复杂





θ8-B1

高级搜索树

B-树：大数据

640K ought to be enough for anybody.

- B. Gates, 1981

百日又不是百年。你权当百日后才娶我。你就忍一忍，
一百天很快就过去了...

邓俊辉

deng@tsinghua.edu.cn

现实A: 存储器容量的增长速度 << 应用问题规模的增长速度

1 Kilobyte = $2^{10} = 10^3$

1 Megabyte = $2^{20} = 10^6$

1 Gigabyte = $2^{30} = 10^9$

1 Terabyte = $2^{40} = 10^{12}$

1 Petabyte = $2^{50} = 10^{15}$

1 Exabyte = $2^{60} = 10^{18}$

1 Zettabyte = $2^{70} = 10^{21}$

1 Yottabyte = $2^{80} = 10^{24}$

1 Nonabyte = $2^{90} = 10^{27}$

1 Doggabyte = $2^{100} = 10^{30}$

❖ 存储器? - RAM: 不就是...**无限**个...寄存器吗? !

- Turing: 不就是...**无限**长的...纸带吗? !

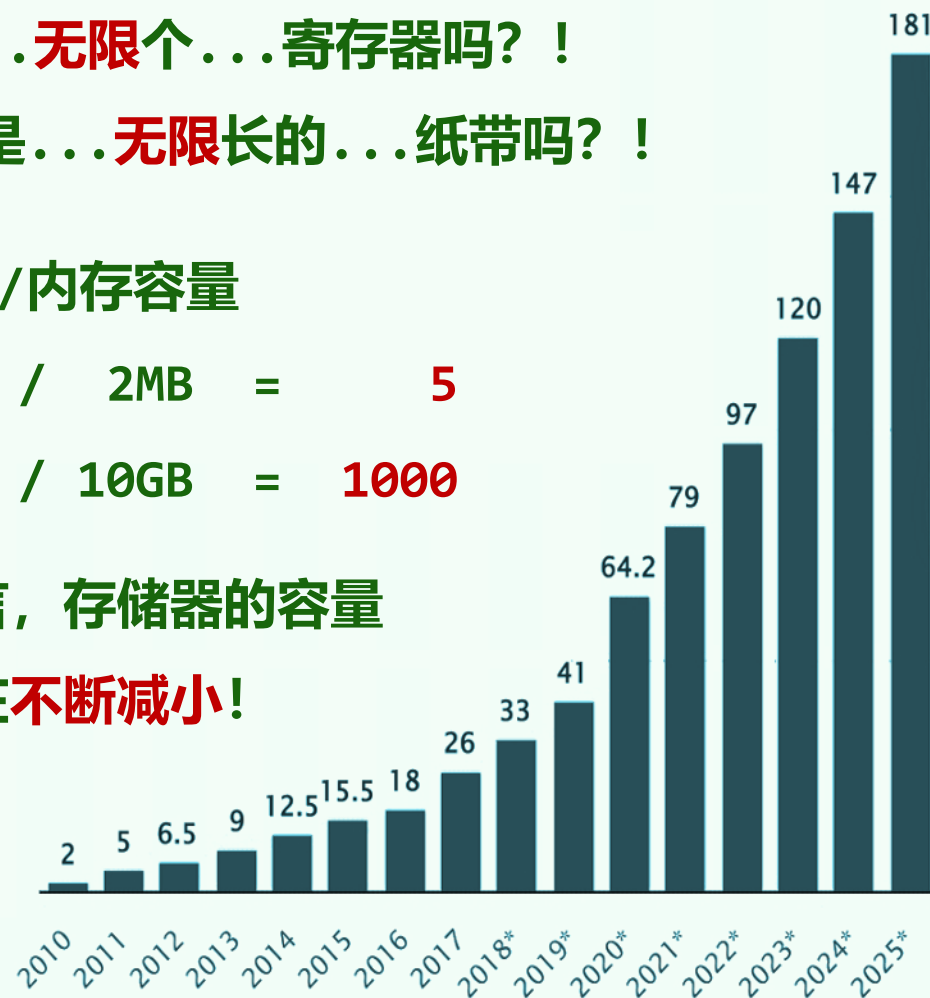
❖ 典型的数据库规模/内存容量

1990: 10MB / 2MB = 5

2020: 10TB / 10GB = 1000

❖ 相对而言, 存储器的容量

实际上在**不断减小**!



现实B：在特定工艺及成本下，存储器都是容量与速度的折中产物

1 Kilobyte = $2^{10} = 10^3$

1 Megabyte = $2^{20} = 10^6$

1 Gigabyte = $2^{30} = 10^9$

1 Terabyte = $2^{40} = 10^{12}$

1 Petabyte = $2^{50} = 10^{15}$

1 Exabyte = $2^{60} = 10^{18}$

1 Zettabyte = $2^{70} = 10^{21}$

1 Yottabyte = $2^{80} = 10^{24}$

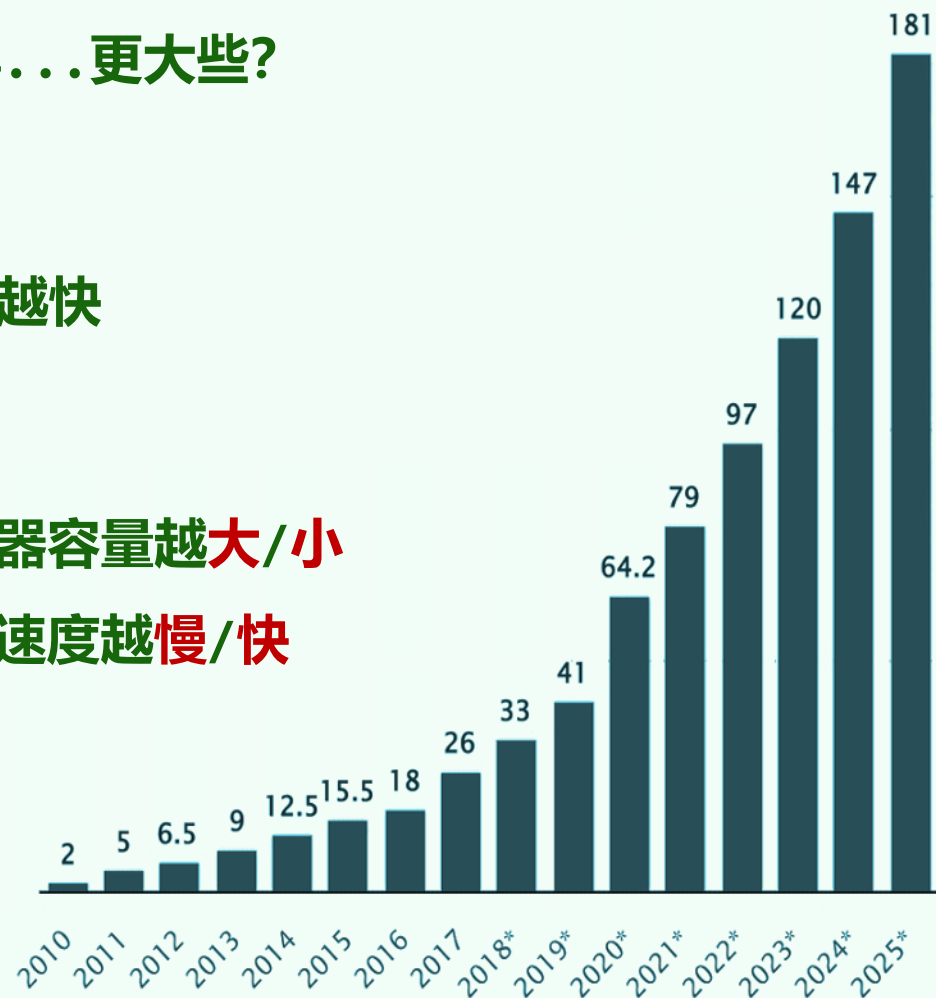
1 Nonabyte = $2^{90} = 10^{27}$

1 Doggabyte = $2^{100} = 10^{30}$

❖ 为何...不把存储器做得...更大些?
非不为，实不能!

❖ 存储器越大、越快
成本也越高

❖ 存储器容量越大/小
访问速度越慢/快



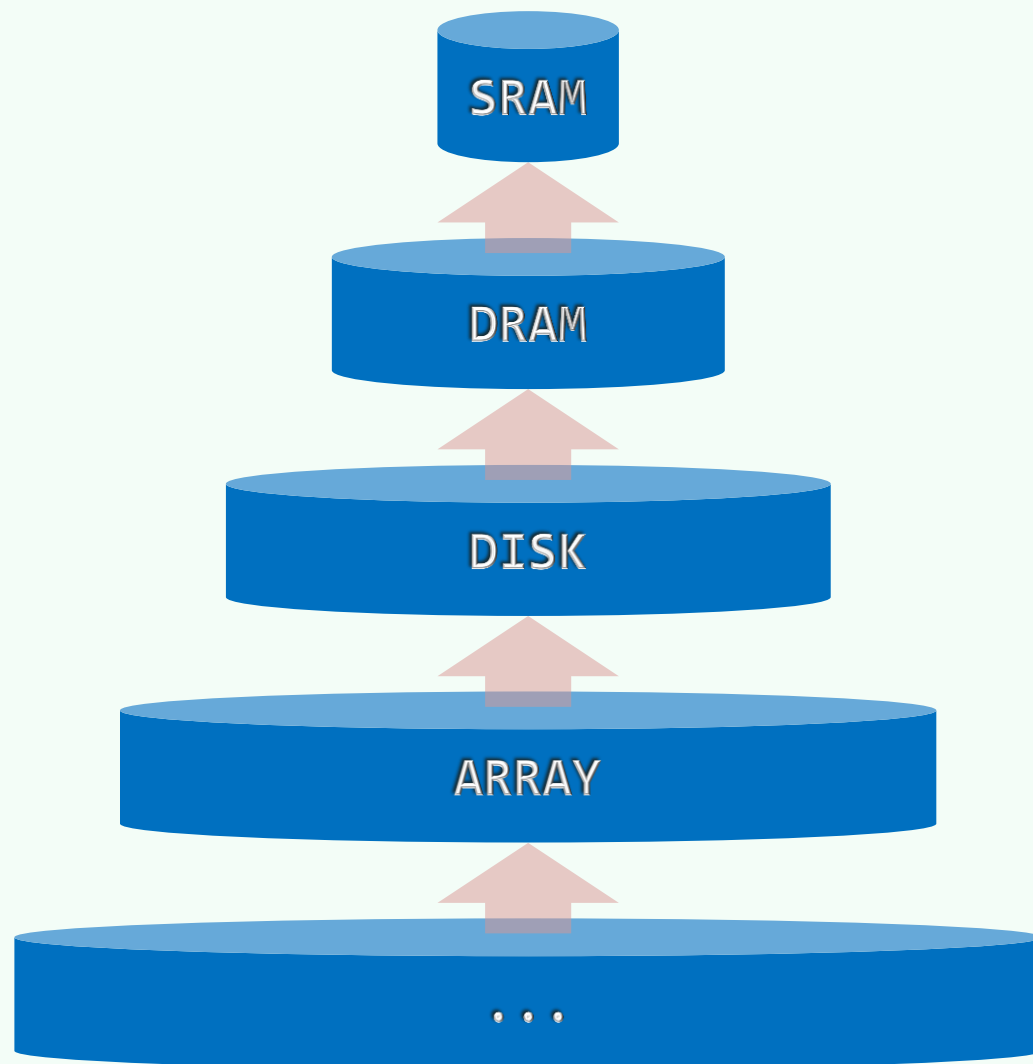
现实C：实用的存储系统，由不同类型的存储器级联而成，以综合其各自的优势

❖ 不同类型存储器的

容量和访问速度，差异悬殊

	#cycles	sec
CPU Register :	0	ns
SRAM/cache :	4~75	ns
DRAM/main memory :	10^2	ns
DISK :	10^7	ms

- ❖ 若一次内存访问需要一秒，则一次磁盘访问就需一天
为避免一次磁盘访问，我们宁愿访问内存1000次



分级存储：利用数据访问的局部性

❖ 机制与策略

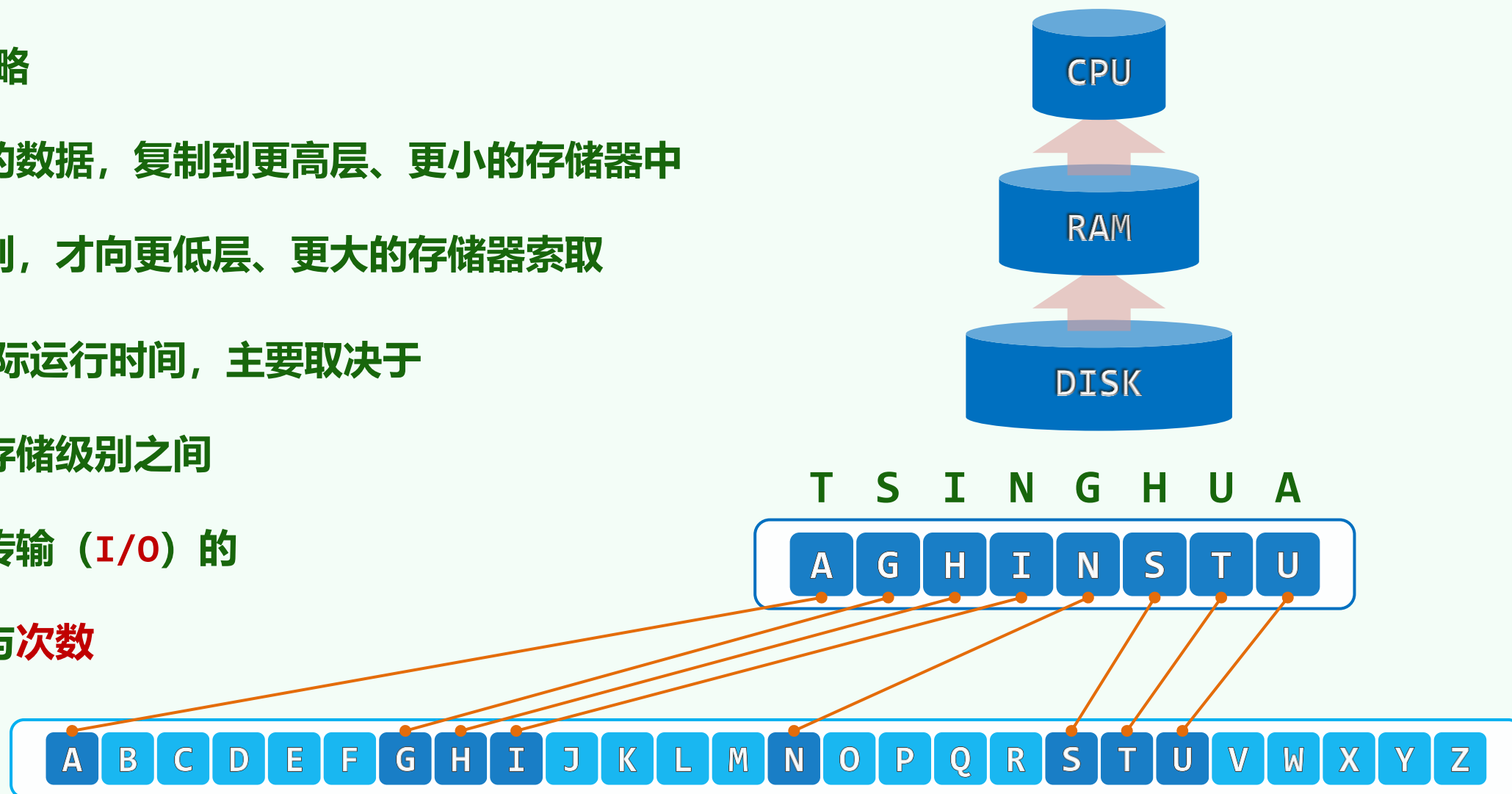
- 常用的数据，复制到更高层、更小的存储器中
- 找不到，才向更低层、更大的存储器索取

❖ 算法的实际运行时间，主要取决于

相邻存储级别之间

数据传输 (I/O) 的

速度与次数



现实D：在外存读写1B，与读写1KB几乎一样快

❖ 以页（page）为单位，借助缓冲区**批量**访问，可大大缩短单位字节的**平均**访问时间

❖ #include <stdio.h>

```
#define BUFSIZ 512 //缓冲区默认容量

int setvbuf( //定制缓冲区

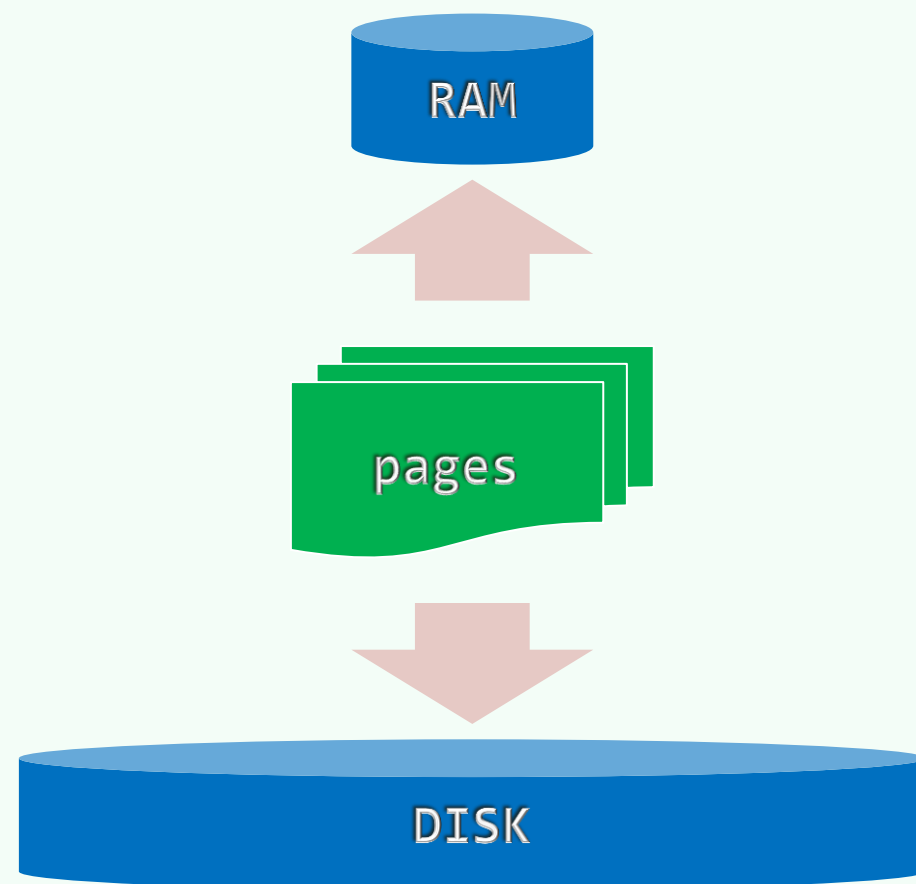
    FILE* fp, //流

    char* buf, //缓冲区

    int _Mode, //_IOFBF | _IOLBF | _IONBF

    size_t size); //缓冲区容量

int fflush( FILE* fp ); //强制清空缓冲区
```



高级搜索树

B-树：缓存

To my mind the most interesting thing in art is the personality of artists;
and if that is singular, I am willing to excuse a thousand faults.

He has given signs of himself which are visible to those
who seek him, and not to those who do not seek him.

邓俊辉

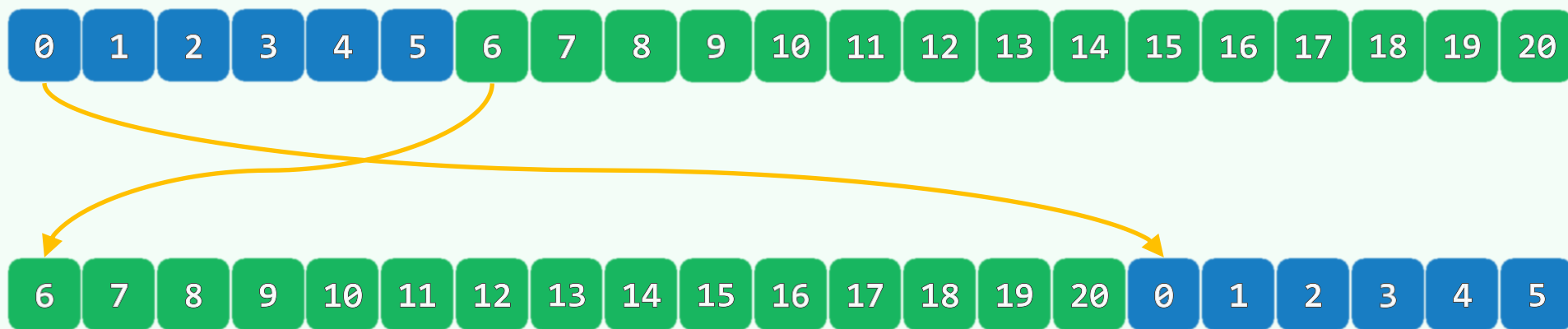
deng@tsinghua.edu.cn

就地循环位移

❖ 仅用 $O(1)$ 辅助空间，将数组 $A[0, n)$ 中的元素向左循环移动 k 个单元

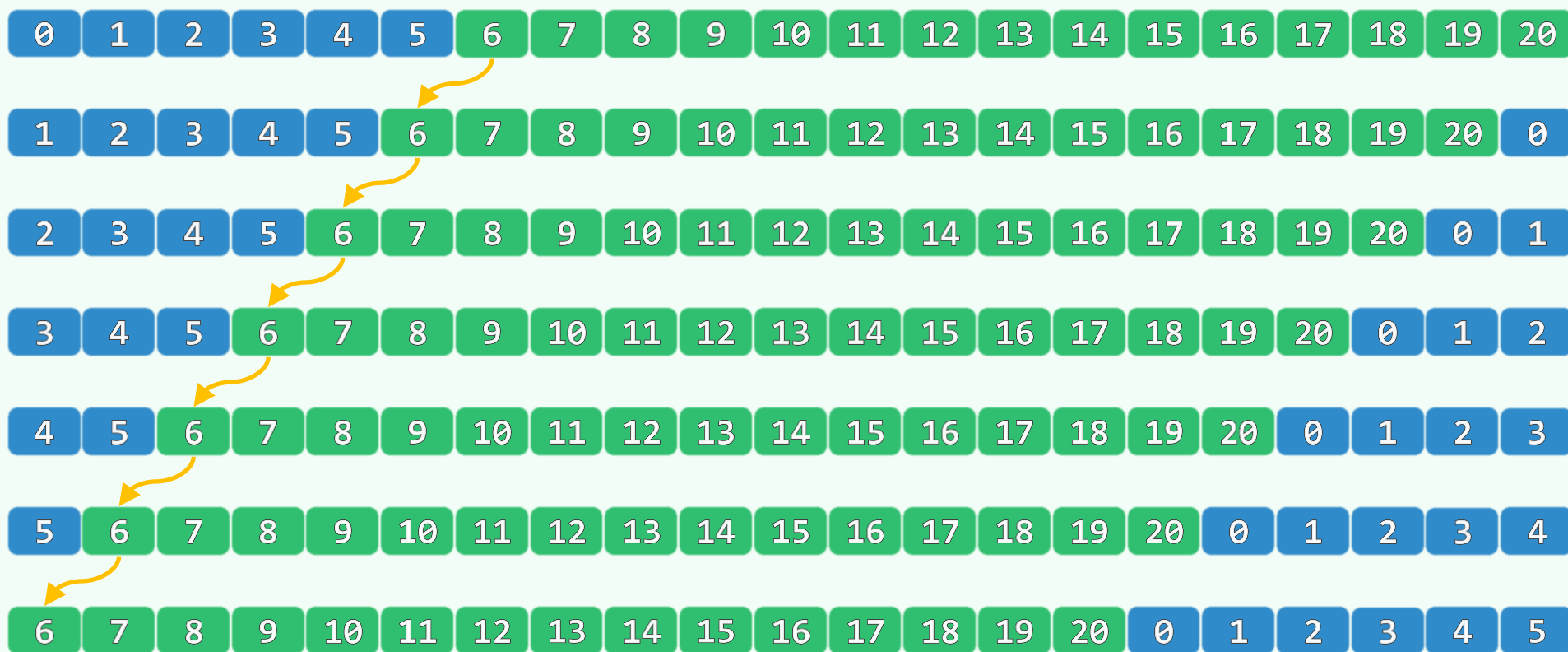
```
void shift( int * A, int n, int k );
```

❖ 比如: `shift( A, 21, 6 );`



蛮力版

```
void shift0( int * A, int n, int k ) //反复以1为间距循环左移  
{ while ( k-- ) shift( A, n, 0, 1 ); } //共迭代k次,  $O(n*k)$ 
```



迭代版: Stride- k Reference Pattern (1/2)

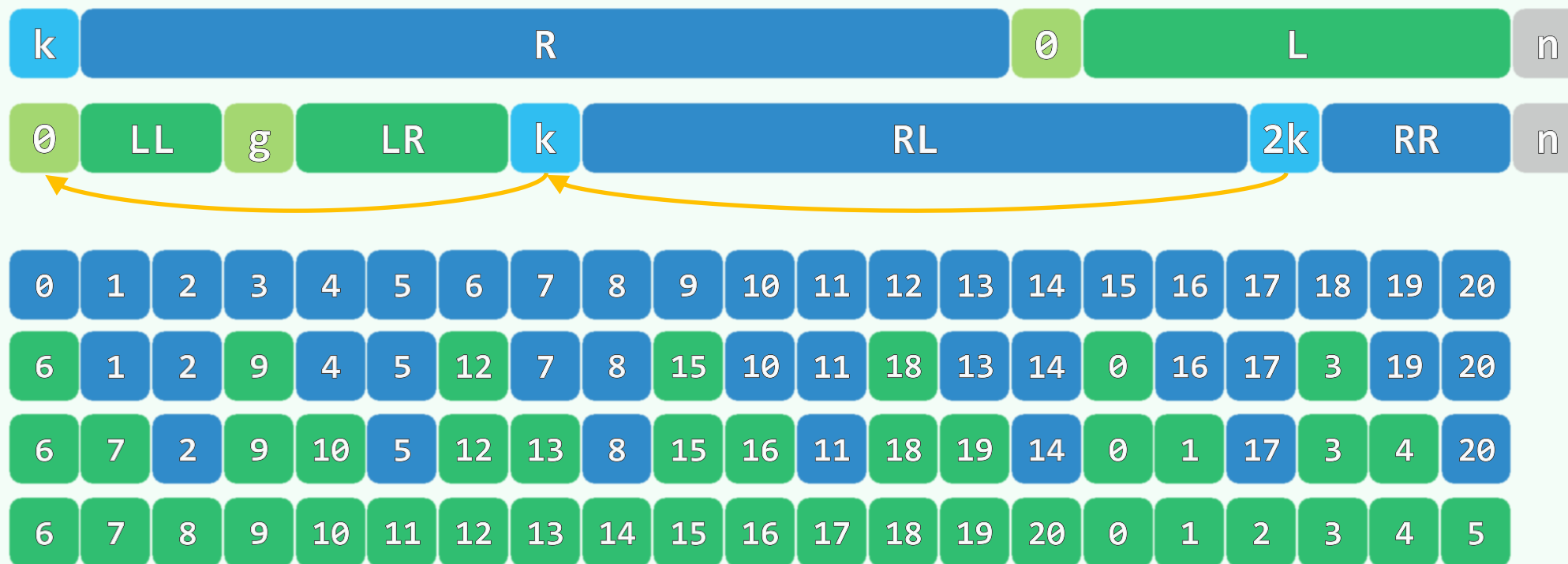
```
int shift( int * A, int n, int s, int  $k$  ) { //  $O( n / \text{GCD}(n, k) )$   
  
    int b = A[s]; int i = s, j = (s +  $k$ ) % n; int mov = 0; //mov记录移动次数  
  
    while ( s != j ) //从A[s]出发, 以k为间隔, 依次左移k位  
    { A[i] = A[j]; i = j; j = (j +  $k$ ) % n; mov++; }  
  
    A[i] = b; return mov + 1; //最后, 起始元素转入对应位置  
}  
//[0, n)由关于k的 $g = \text{GCD}(n, k)$ 个同余类组成, shift(s, k)能够且只能够使其中之一就位
```

其它的同余类呢...



迭代版: Stride- k Reference Pattern (2/2)

```
void shift1(int* A, int n, int  $k$ ) { //经多轮迭代, 实现数组循环左移 $k$ 位, 累计 $O(n+g)$   
    for (int s = 0, mov = 0; mov < n; s++) // $O(g) = O(\text{GCD}(n, k))$   
        mov += shift(A, n, s,  $k$ );  
}
```



倒置版: Stride-1 Reference Pattern

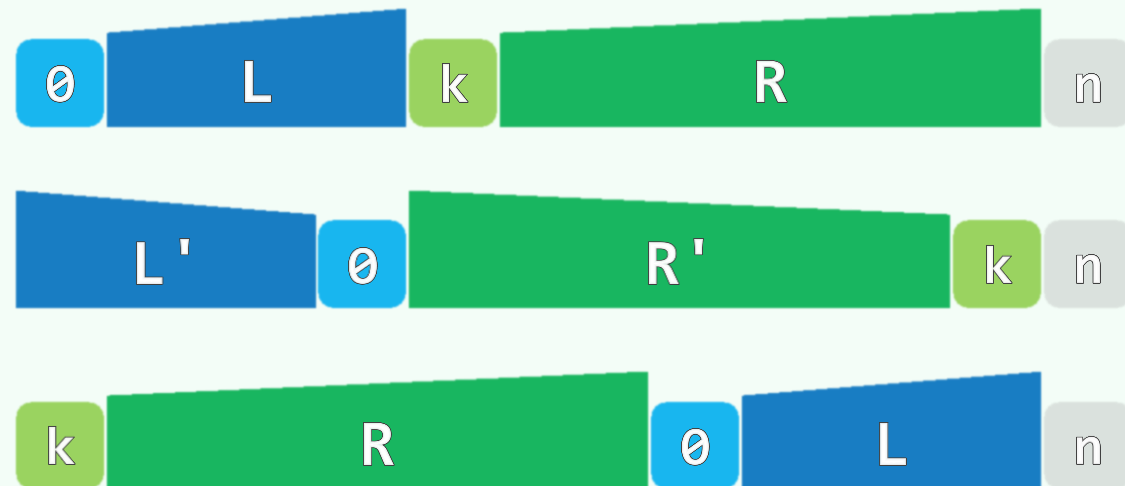
```
void shift2( int * A, int n, int k ) {
```

```
    reverse( A, k ); //  $\mathcal{O}(3k/2)$ 
```

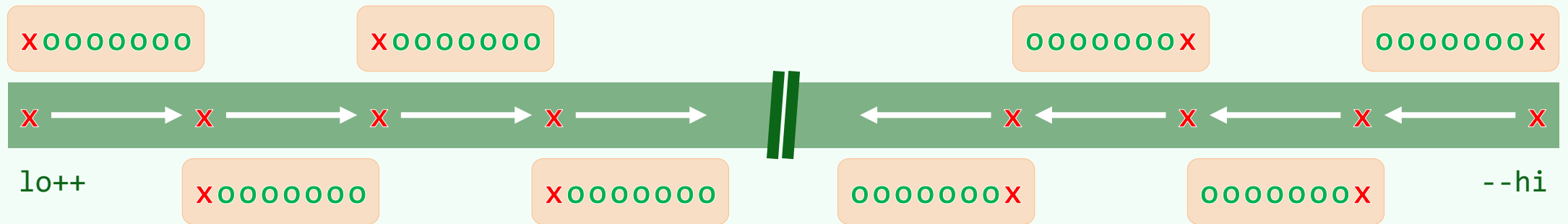
```
    reverse( A + k, n - k ); //  $\mathcal{O}(3(n-k)/2)$ 
```

```
    reverse( A, n ); //  $\mathcal{O}(3n/2)$ 
```

```
} //  $\mathcal{O}(3n)$ 
```



Cache-Friendly Code



08-B3

高级搜索树

B-树：结构

昨夜西風凋碧樹，獨上高樓，望盡天涯路

妻子好合，如鼓瑟琴；兄弟既翕，和樂且湛

邓俊辉

deng@tsinghua.edu.cn

等价变换

❖ 平衡的多路搜索树

R. Bayer & E. McCreight

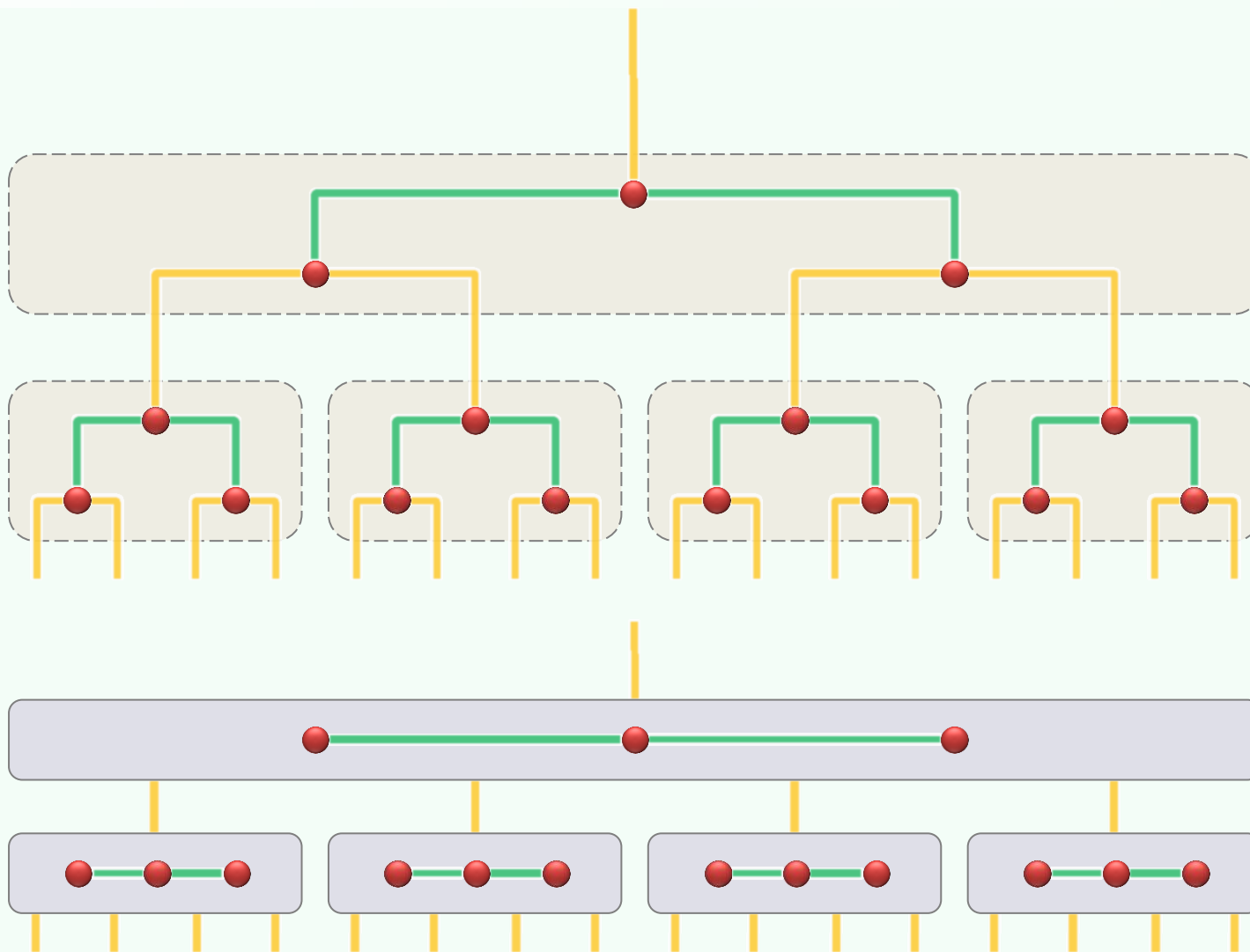
1970

❖ 每 d 代合并为一个超级节点

- $m = 2^d$ 路
- $m-1$ 个关键码

❖ 逻辑上与BBST完全等价

既如此，B-树之意义何在？



I/O优化：多级存储系统中使用B-树，可针对外部查找，大大减少I/O次数

❖ 难道，AVL还不够？比如，若有 $n = 1\text{G}$ 个记录...

- 每次查找需做 $\log_2 10^9 \approx 30$ 次I/O
- 每次只读出单个关键码，效率太低

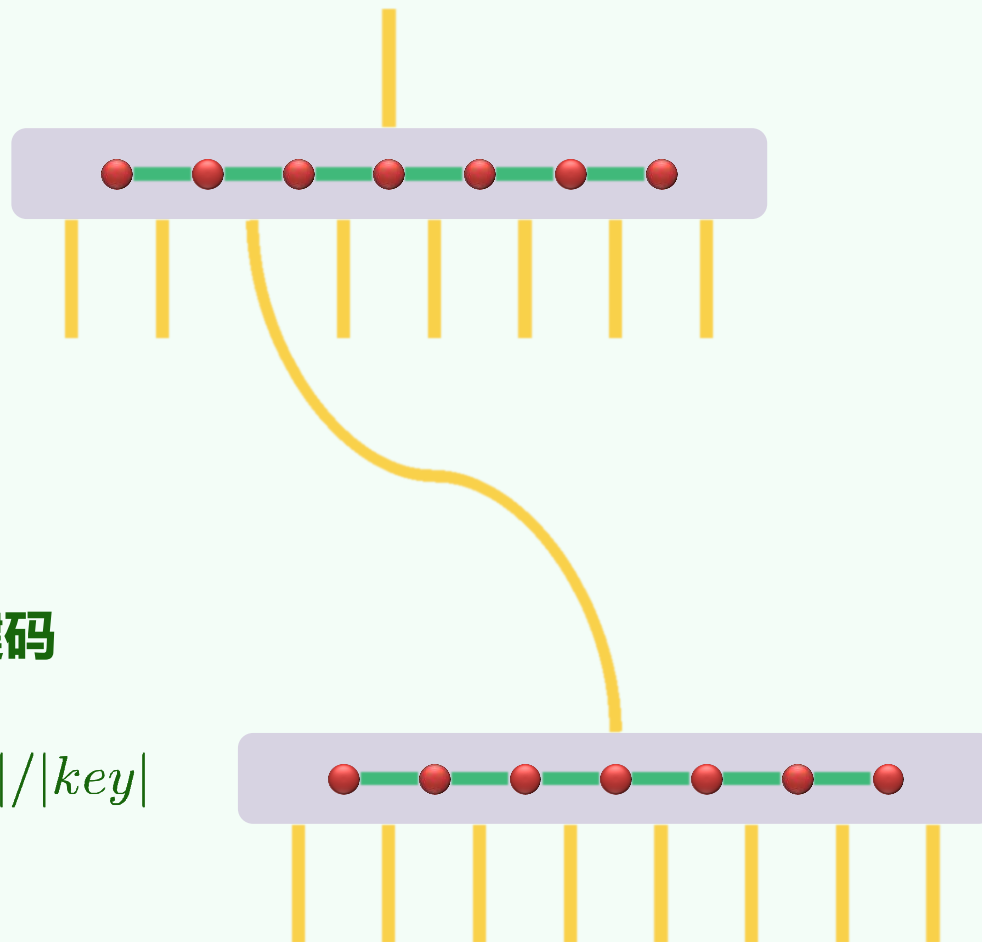
❖ B-树又能如何？

- 充分利用外存的批量访问，将此特点转化为优点
- 每下降一层，都以超级节点为单位，读入一组关键码

❖ 具体多大一组？视磁盘的数据块大小而定， $m = |page|/|key|$

- 比如，目前多数数据库系统采用 $m = 200 \sim 300$

❖ 回到上例，若取 $m = 256$ ，则每次查找只需 $\log_{256} 10^9 \leq 4$ 次I/O



外部节点 + 叶子

❖ 所谓**m阶B-树**，即

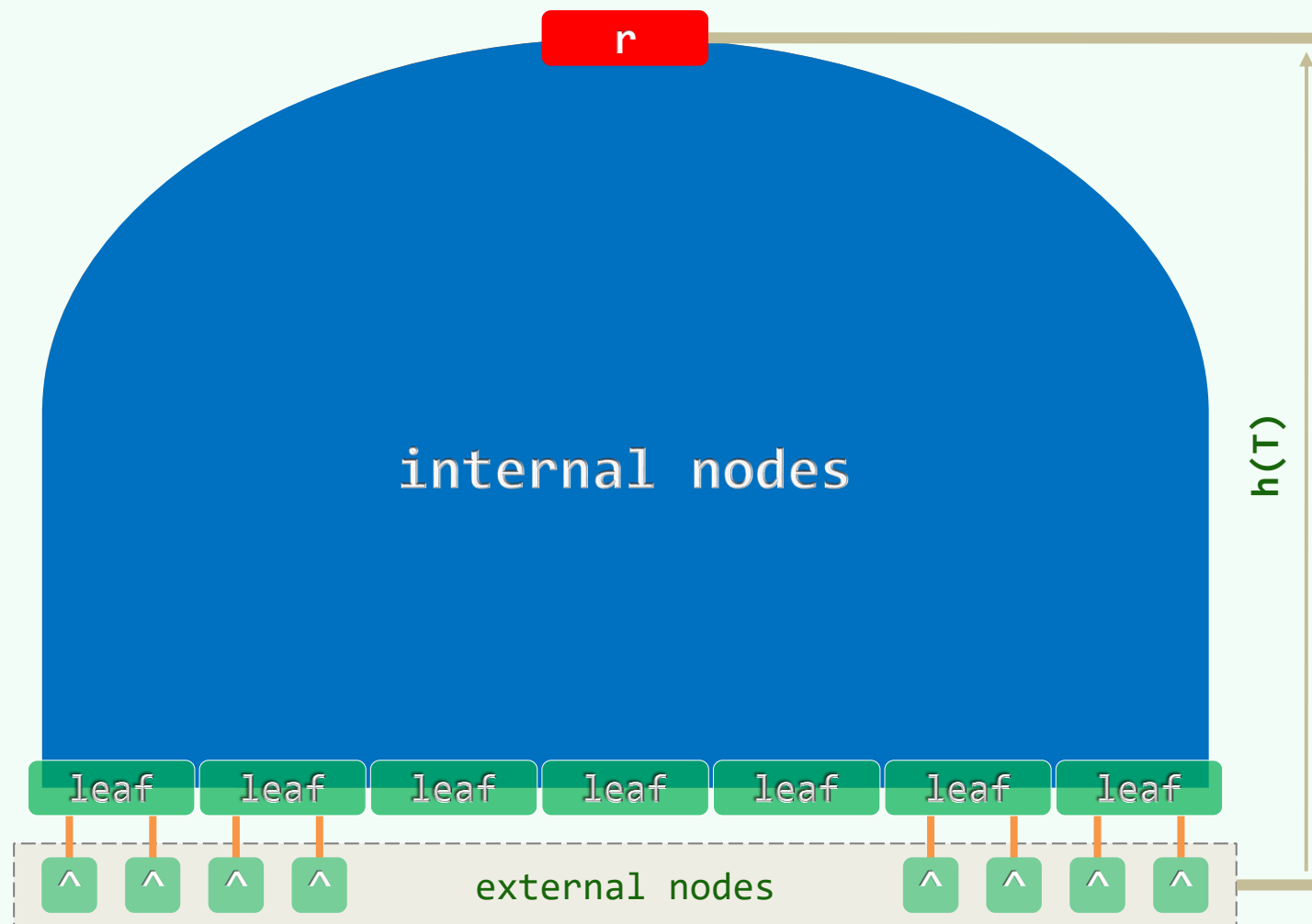
m路完全平衡搜索树 ($m \geq 3$)

❖ **外部节点**的深度统一相等

约定以此深度作为树高 h

❖ **叶节点**的深度统一相等 ($h-1$)

❖ 注意，这里的**叶节点**与**外部节点**不同



内部节点

❖ 各含 $n \leq m - 1$ 个关键码:

$$K_1 < K_2 < K_3 < \cdots < K_n$$

❖ 各有 $n + 1 \leq m$ 个分支:

$$A_0, A_1, A_2, A_3, \dots, A_n$$

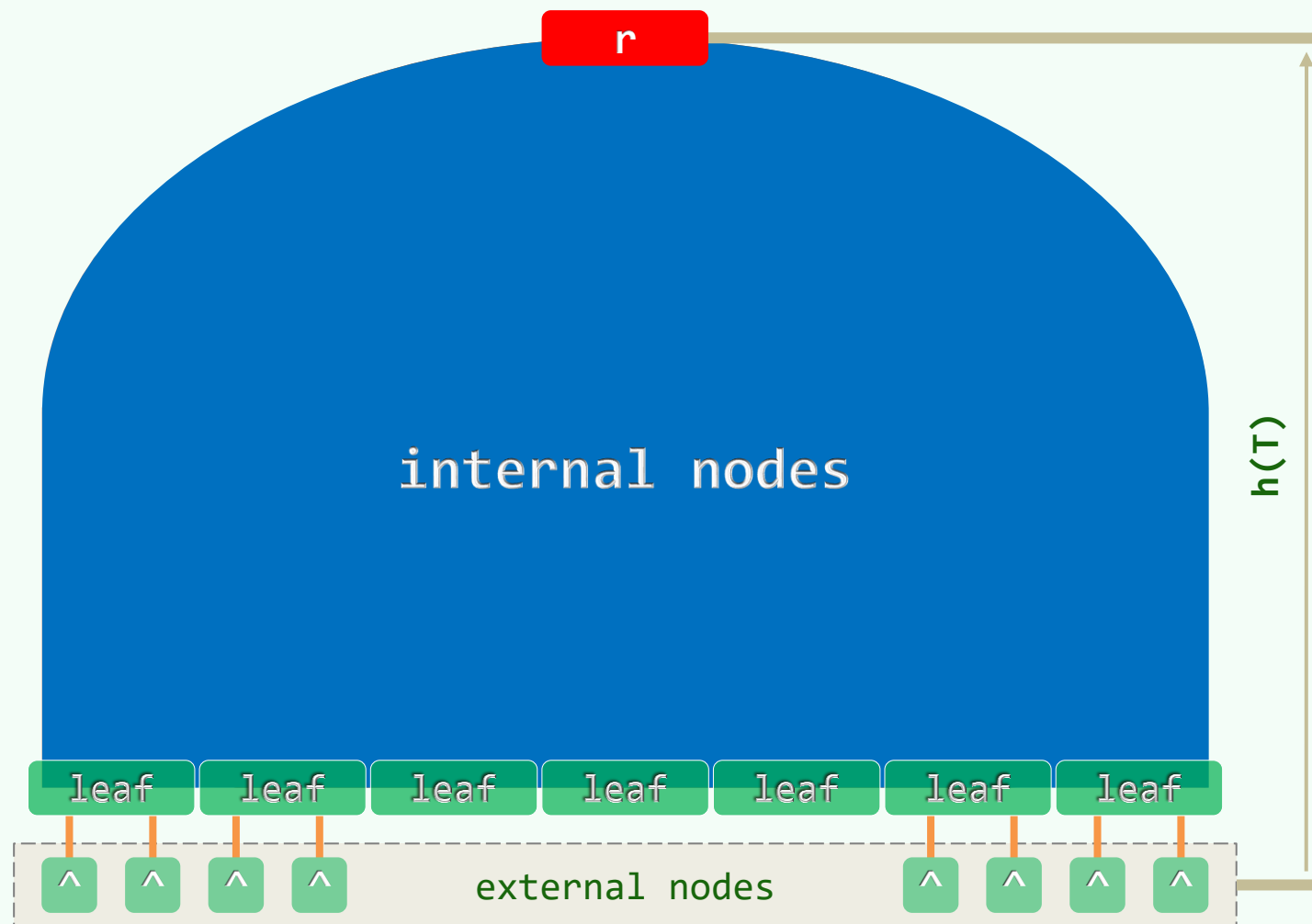
❖ 反过来, 分支数也不能太少

- 树根: $2 \leq n + 1$

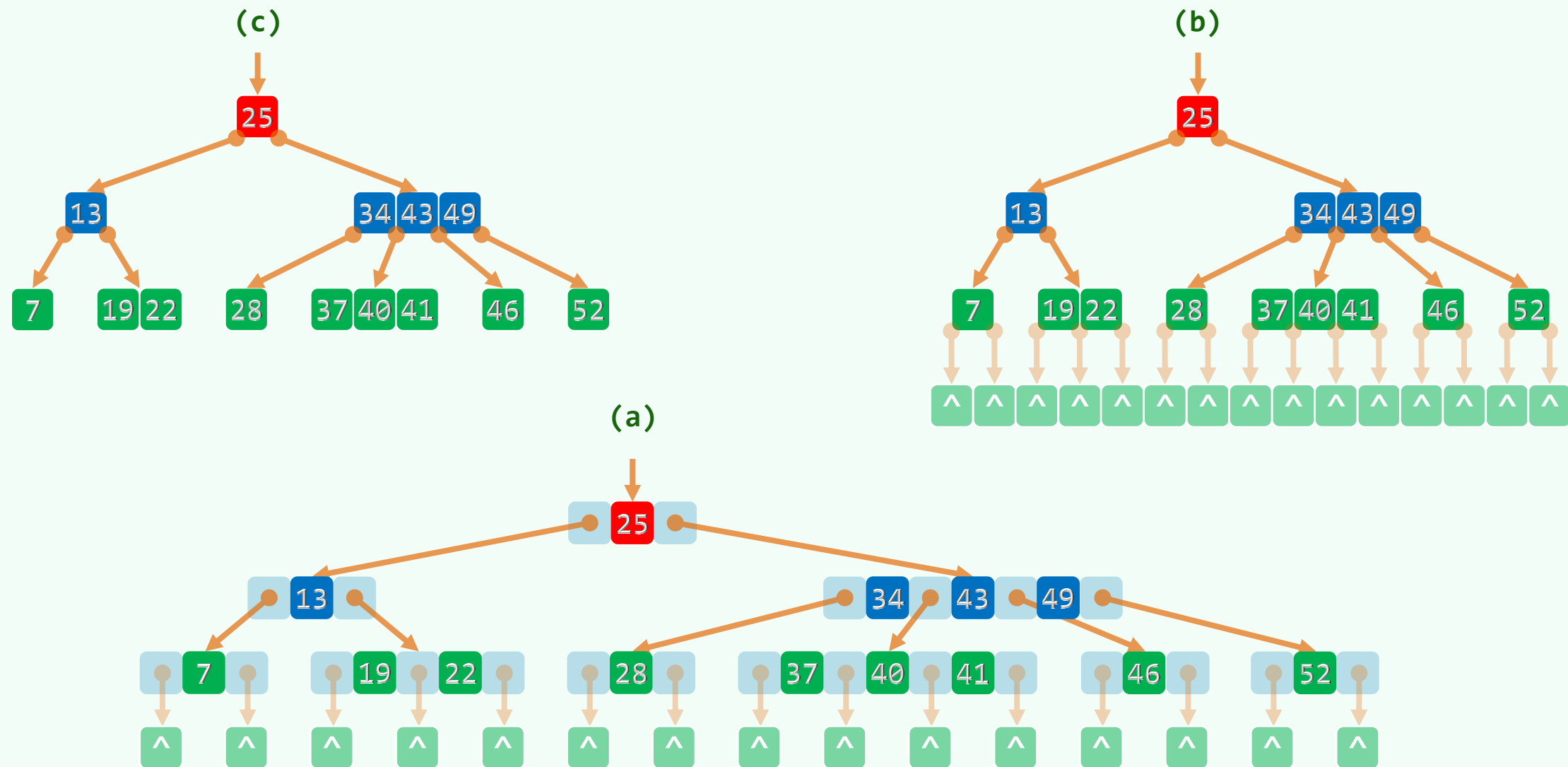
- 其余: $\lceil m/2 \rceil \leq n + 1$

❖ 故亦称作 $(\lceil m/2 \rceil, m)$ -树:

(3,5)-树、(9,18)-树、...



紧凑表示



实例

❖ $m = 3$: 2-3-树, $(2, 3)$ -树

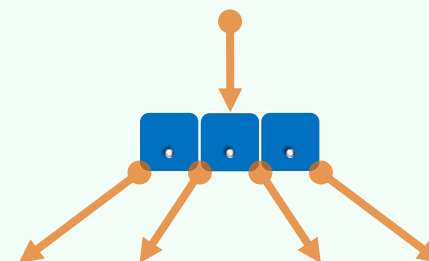
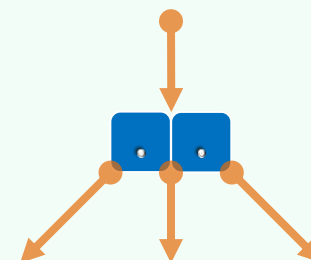
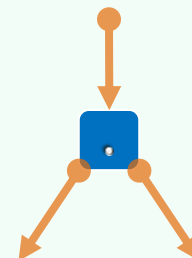
最简单的B-树 // J. Hopcroft, 1970

- 各（内部）节点的分支数，可能是2或3
- 各节点所含key的数目，可能是1或2

❖ $m = 4$: 2-3-4-树, $(2, 4)$ -树

- 各节点的分支数，可能是2、3或4
- 各节点所含key的数目，可能是1、2或3

❖ 留意把玩4阶B-树，稍后对于理解红黑树大有裨益



BTNode

```
template <typename T> using BTNP = BTNode <T>*; //B-Tree Node Position
```

```
template <typename T> struct BTNode { //B-树节点
```

```
    BTNP<T> parent; //父亲
```

```
    Rank height; //Height(NULL) == 0
```

```
    Vector<T> key; //关键码 (总比孩子少一个)
```

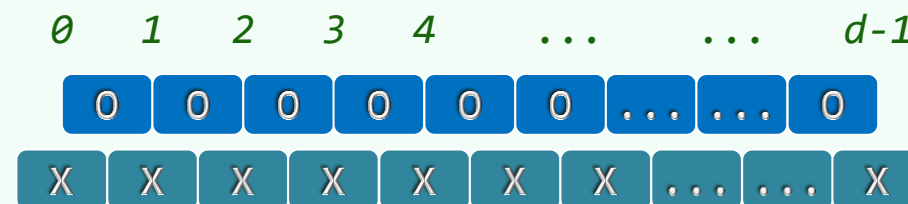
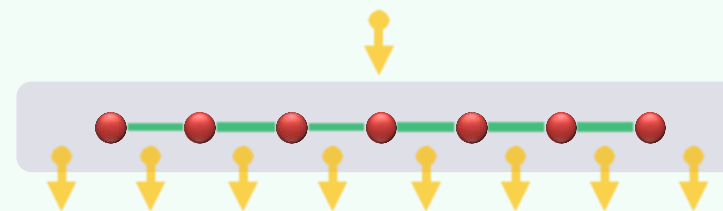
```
    Vector<BTNP<T>> child; //孩子
```

```
    BTNode( Rank h = 1, BTNP<T> c = NULL )
```

```
        : parent(NULL), height(h) //即便是空树, 也有一个退化的节点
```

```
        { child.insert( c ); if ( c ) c->parent = this; } //无关键码、单分支
```

```
}; //BTNode
```



BTree

```
template <typename T> struct BTree { //B-树
    Rank _size, _m, BTNP<T> _root; //关键码总数、阶次、根节点
    void solveOverflow( BTNP<T> ); //上溢修复
    void solveUnderflow( BTNP<T> ); //下溢修复
    BTree( Rank m = 3 ) : _m(m), _size(0) { _root = new BTNode<T>; }
    ~BTree() { clean( _root ); } //释放所有节点 (对照示例代码自学)
    BTNP<T> search( const T& ); //查找
    bool insert( const T& ); //插入
    bool remove( const T& ); //删除
}; //BTree
```

08-B4

高级搜索树

B-树：查找

高至天低至深海
每寸搜索着这天下
寻觅着那个"它"

...按照模型的运算量，用现有的最高计算能力模拟百分之一秒的聚变过程，就需大约二十年时间。而研究过程中的模拟需要反复进行，这使得模型的实际应用成为不可能

邓俊辉

deng@tsinghua.edu.cn

算法

从（常驻RAM的）根节点开始

只要当前节点不是外部节点

在当前节点中**顺序查找** //RAM内部

若找到目标关键码，则

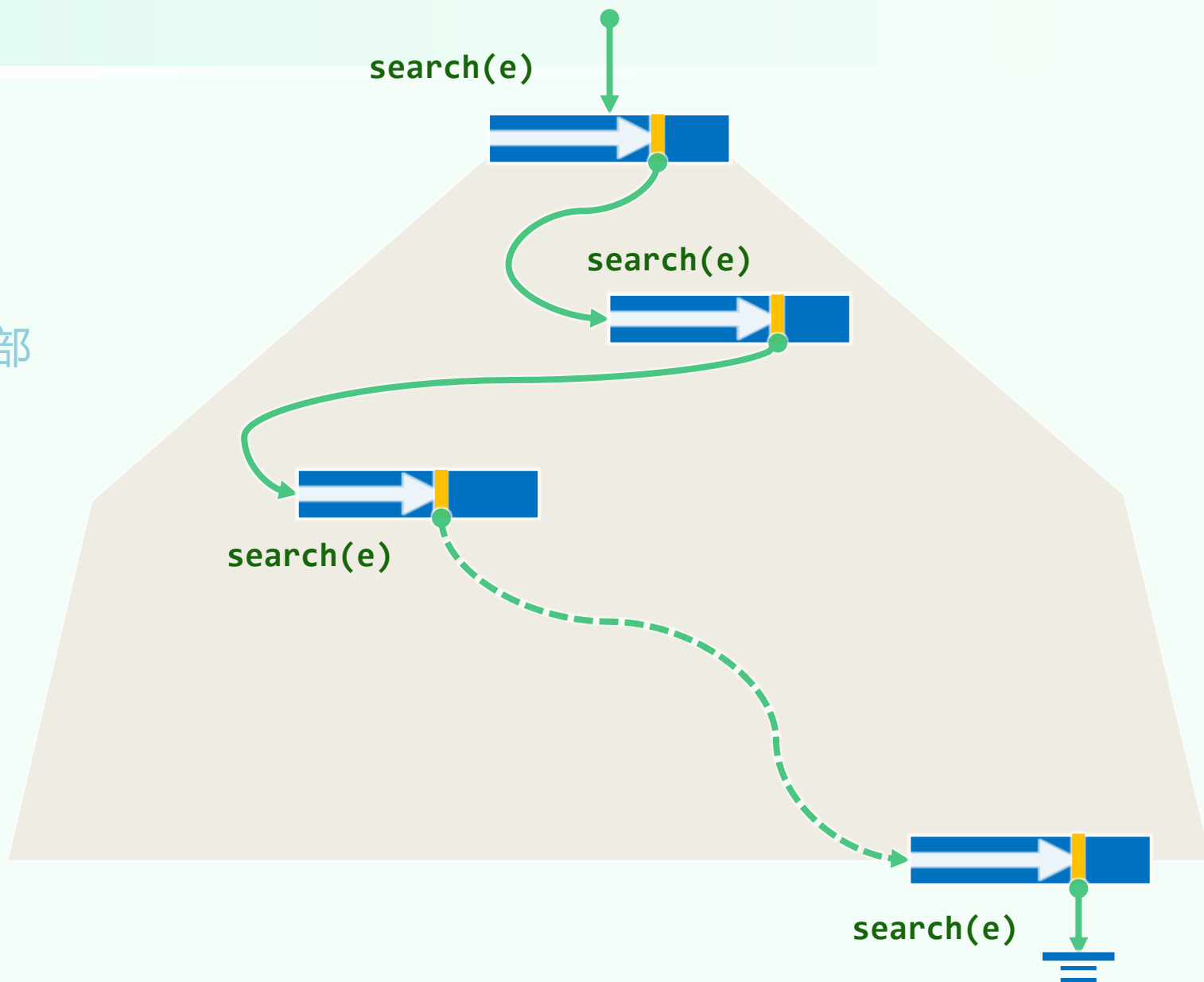
返回查找成功

否则 //止于某一向下的引用

沿引用找到孩子节点

将其**读入内存** //I/O耗时

返回查找失败

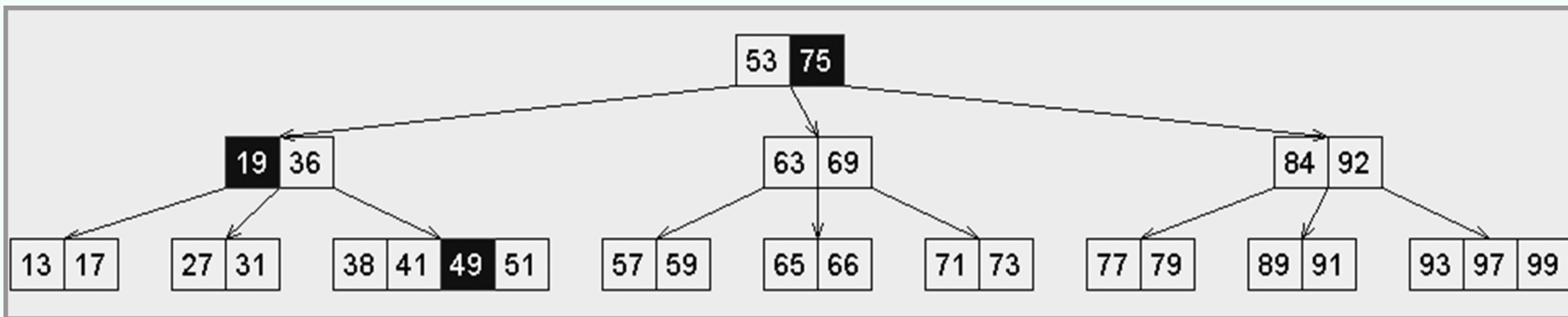


实例

❖ (3,5)-树: 53 97 36 89 41 75 19 84 77 79 51 57 99 91
 92 93 17 73 13 66 59 49 63 65 71 69 27 31 38

成功查找: 75, 19, 49

失败查找: 5, 45



实现

```
template <typename T> BTNP<T> BTree<T>::search( const T& e ) {
```

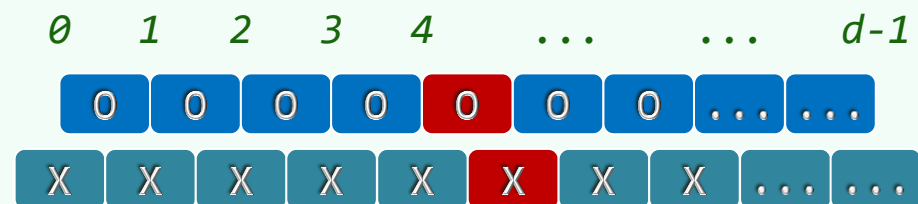
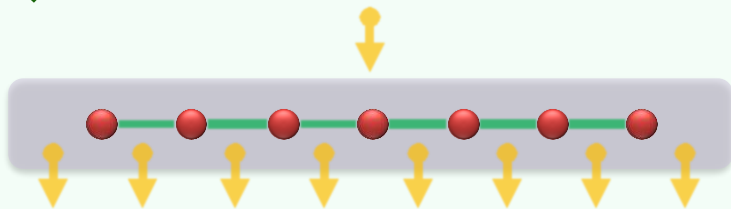
```
    BTNP<T> x = _root; //自根而下
```

```
    for ( Rank k = -1; x->child[0]; x = x->child[k+1] ) { //逐层深入 (I/O)
```

```
        k = x->key.search( e ); //简便起见用了二分查找，但此时改用顺序查找效率反而更高
```

```
        if ( -1 != k && e == v->key[k] ) break; //一旦发现e，随即返回
```

```
    }
```



```
    return x; //成功时，x为e所属节点；失败时，x为可插入e的叶节点
```

```
} //search
```

性能

❖ 约定：根节点常驻RAM

❖ 忽略内存中的查找

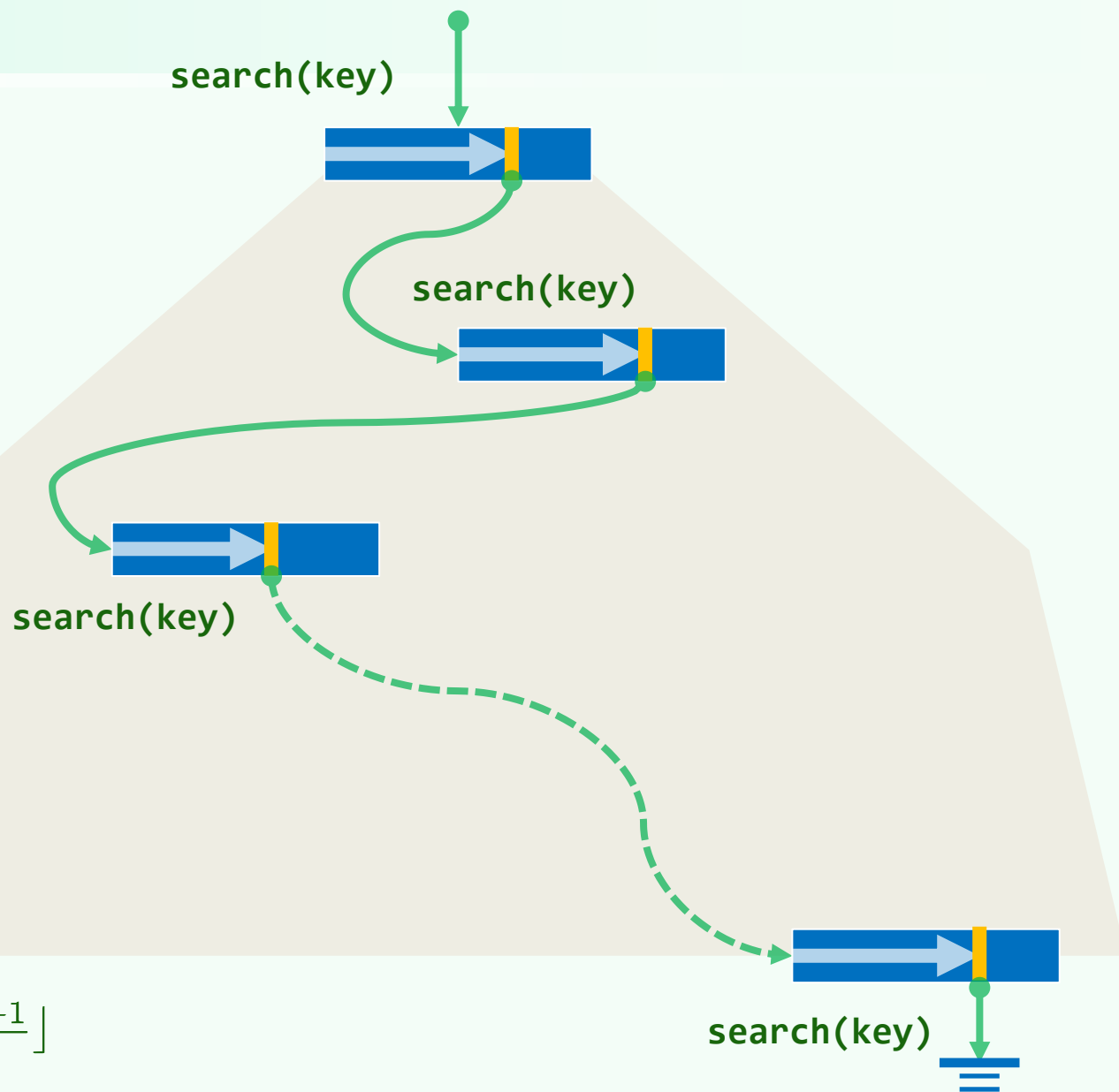
运行时间主要取决于I/O次数

❖ 在每一深度至多一次I/O

❖ 故运行时间为 $\mathcal{O}(\log n)$

❖ 可以证明：

$$\log_m (N + 1) \leq h \leq 1 + \lceil \log_{\lceil m/2 \rceil} \frac{N+1}{2} \rceil$$



含N个关键码的m阶B-树，可能有多高？

❖ 为此，内部节点应尽可能地“瘦”

$$n_k \geq 2 \times \lceil m/2 \rceil^{k-1}, \quad \forall k > 0$$

❖ 考查外部节点所在的那层：

$$N + 1 = n_h \geq 2 \times \lceil m/2 \rceil^{h-1}$$

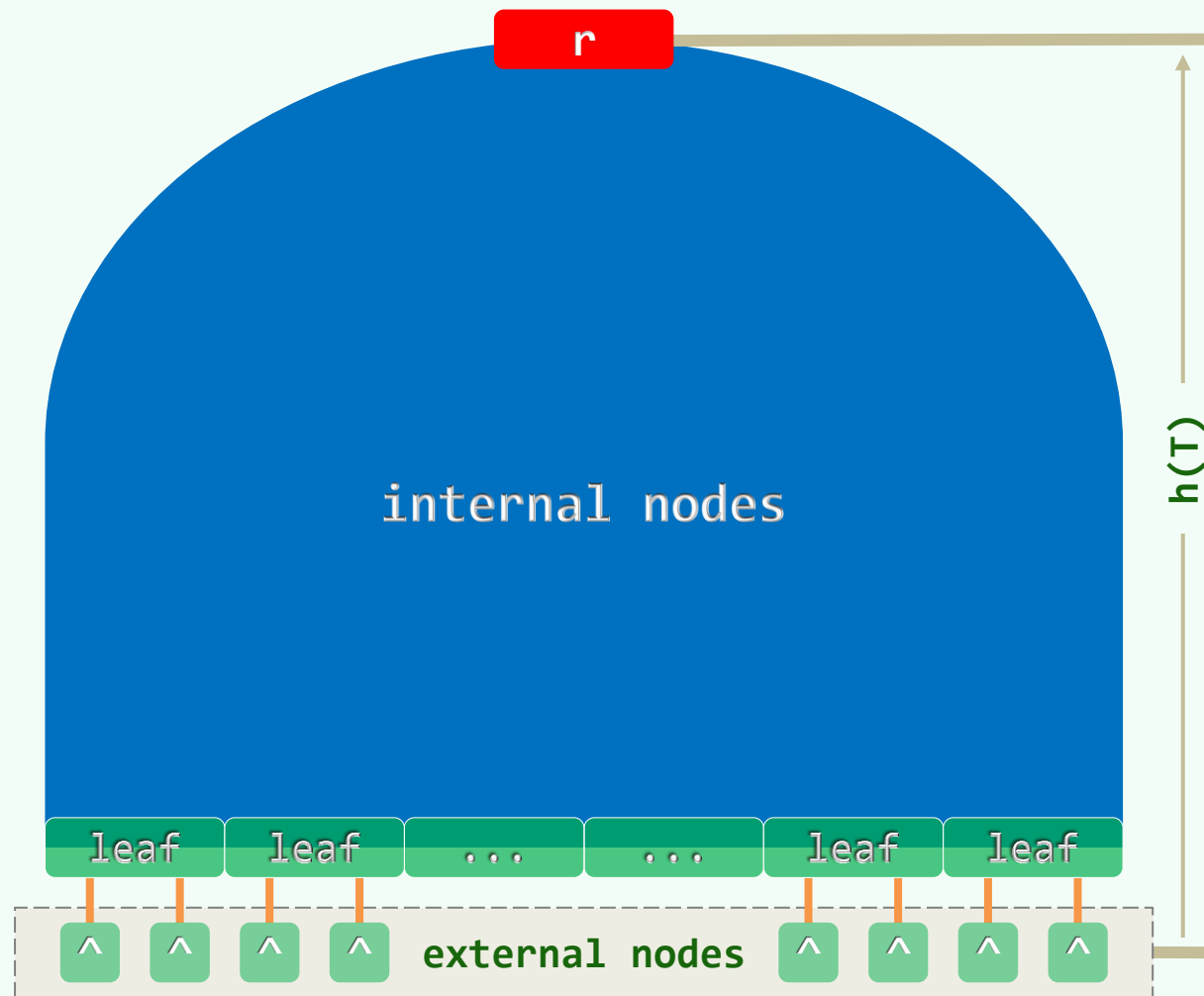
$$h \leq 1 + \lfloor \log_{\lceil \frac{m}{2} \rceil} \frac{N+1}{2} \rfloor = \mathcal{O}(\log_m N)$$

❖ 相对于BBST：

$$\log_{\lceil \frac{m}{2} \rceil} (N/2) / \log_2 N = 1/(\log_2 m - 1)$$

若取 $m = 256$ ，树高约降低至1/7

...用4年上完大学，还是28年？



含N个关键码的m阶B-树，可能有多矮？

❖ 为此，内部节点应尽可能地“胖”

$$n_k \leq m^k, \quad \forall k \geq 0$$

❖ 依然，考查外部节点所在的那层

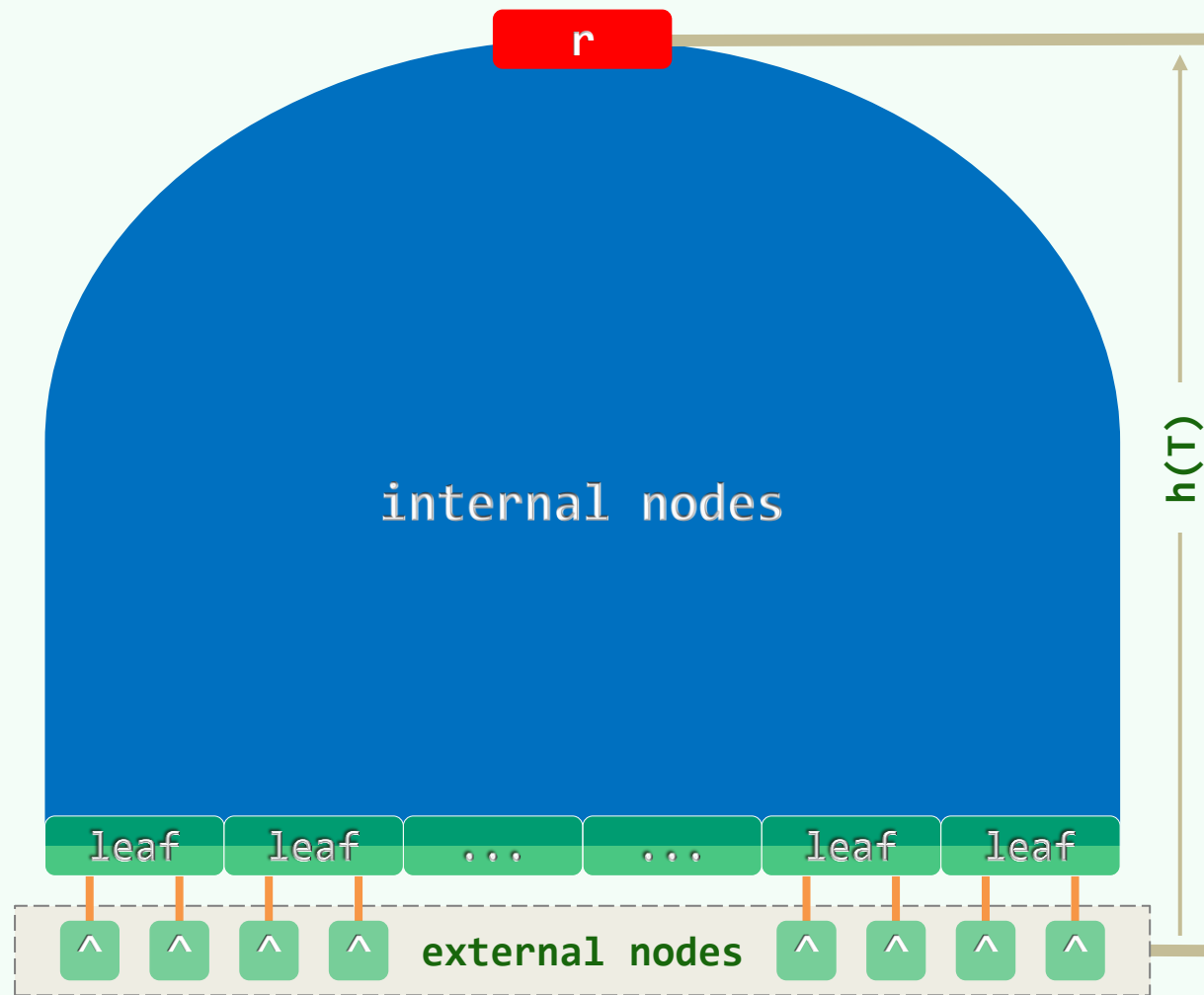
$$N + 1 = n_h \leq m^h$$

$$h \geq \lceil \log_m (N + 1) \rceil = \Omega(\log_m N)$$

❖ 相对于BBST：

$$\begin{aligned} & (\log_m N - 1) / \log_2 N \\ &= \log_m 2 - \log_N 2 \approx 1 / \log_2 m \end{aligned}$$

若取 $m = 256$ ，树高约降低至原先的 **1/8**



08-B5

高级搜索树

B-树：插入

说再见，在这梦幻国度，最后的一瞥
清醒让我，分裂再分裂

邓俊辉

deng@tsinghua.edu.cn

算法

```
template <typename T> bool BTree<T>::insert( const T& e ) {
```

```
    BTNP<T> x = search( e ); Rank k = x->key.search( e );
```

```
    if ( e == x->key[k] ) return false; //确认e尚不存在, 即x为叶节点
```

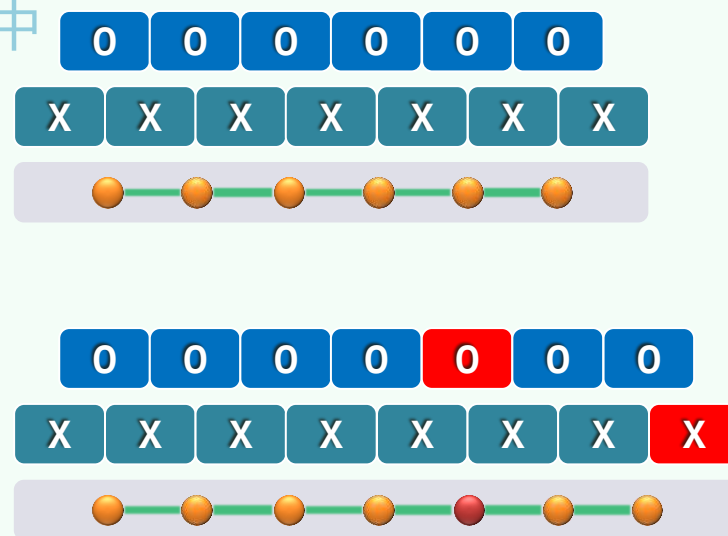
```
    x->key.insert( k+1, e ); //e作为第k+1个关键码, 插至x中
```

```
    x->child.insert( NULL ); //相应地, 其下加个外部节点
```

```
    _size++; solveOverflow( x ); //如上溢, 则修复之
```

```
    return true; //插入成功
```

```
} //insert
```



分裂

❖ 设上溢节点中的关键码依次为:

$$\{ k_0, k_1, \dots, k_{m-1} \}$$

❖ 取中位数 $s = \lfloor m/2 \rfloor$, 以关键码 k_s 为界划分为

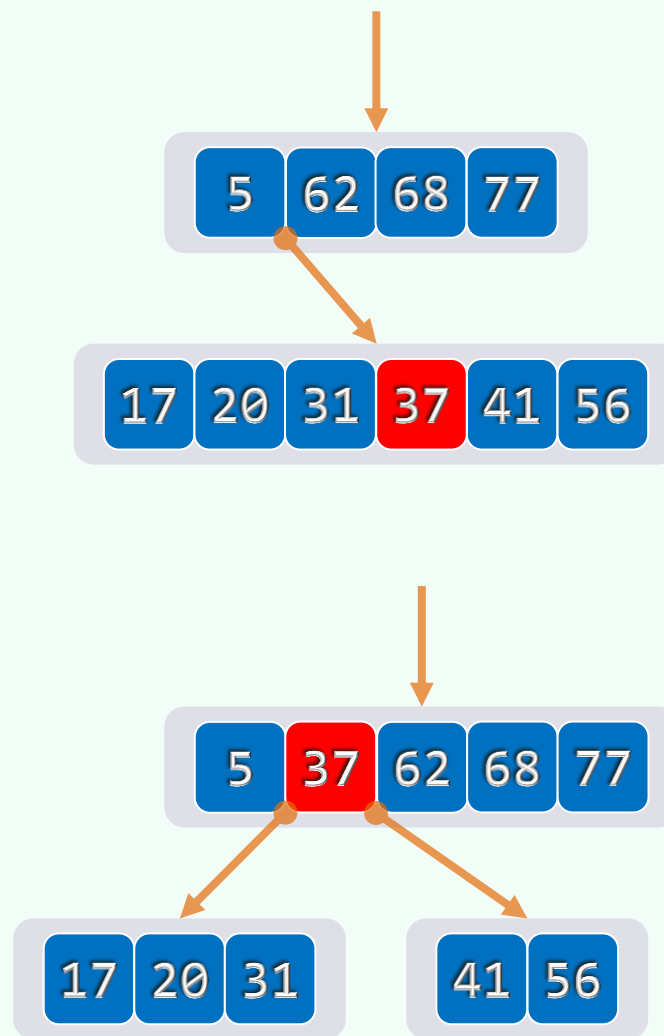
$$\{ k_0, \dots, k_{s-1} \} \quad \{ k_s \} \quad \{ k_{s+1}, \dots, k_{m-1} \}$$

❖ 关键码 k_s 上升一层, 并分裂 (split)

以所得的两个节点作为左、右孩子

❖ 不难验证, 如此分裂后

左、右孩子所含**关键码数目**, 依然符合m阶B-树的条件



再分裂

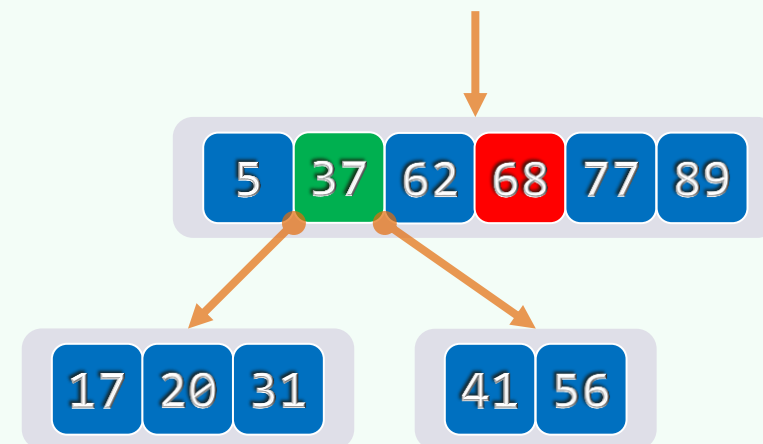
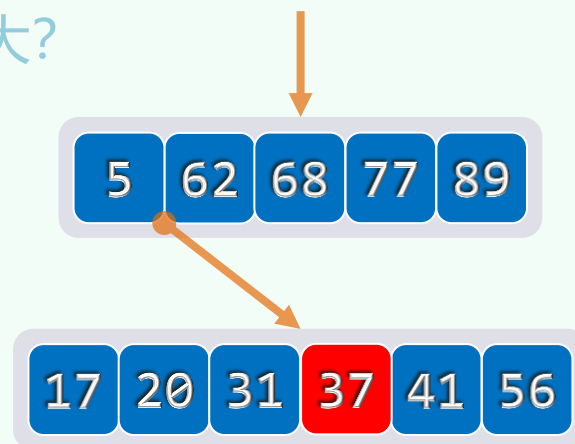
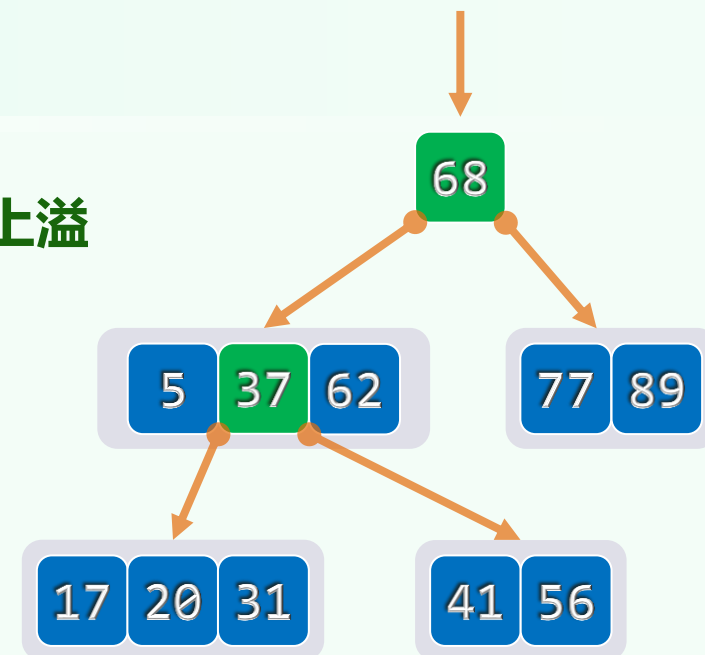
❖ 若上溢节点的父亲**本已饱和**，则在接纳被提升的关键码之后，也将上溢
此时，大可套用前法，继续分裂

❖ 上溢可能持续发生，并逐层**向上传播**
纵然最坏情况，亦不过到根 //若果真抵达树根...

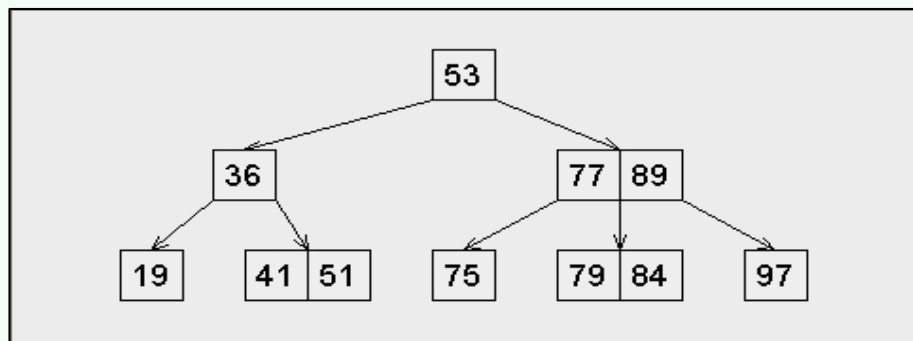
❖ 可令被提升的关键码自成节点，作为新的树根
这是B-树增高的**唯一可能** //概率多大?

❖ 注意：新生的树根仅有**两个分支**

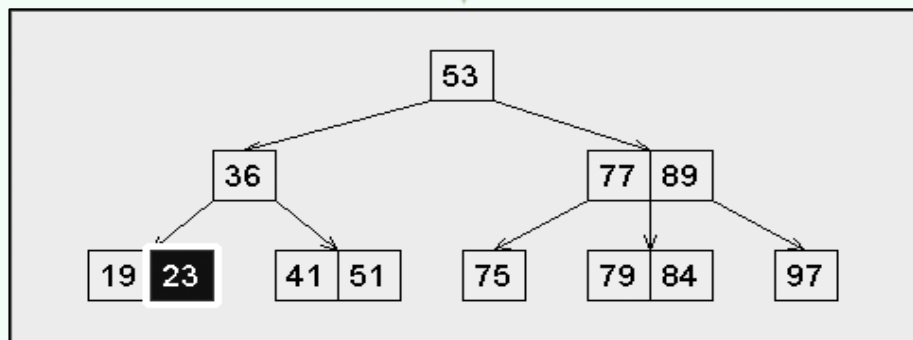
❖ 总体执行时间正比于
分裂次数， $O(h)$



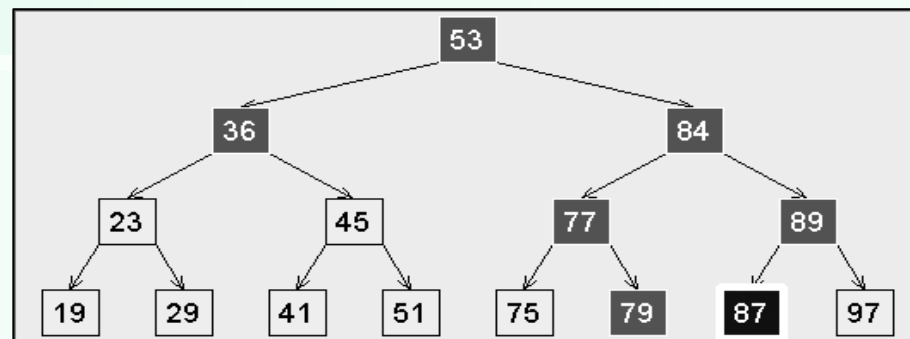
实例：(2,3)-树



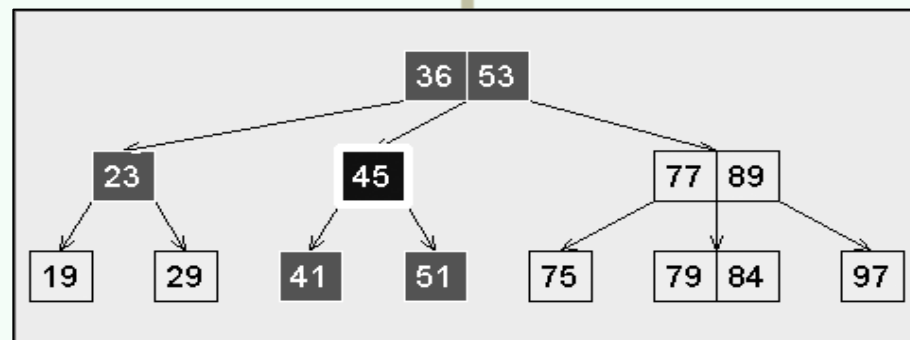
insert(23) //无需分裂



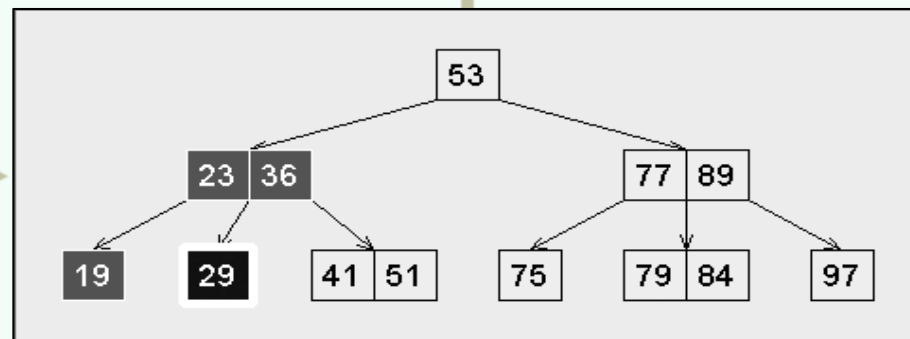
insert(29) //分裂一次



insert(87) //分裂到根



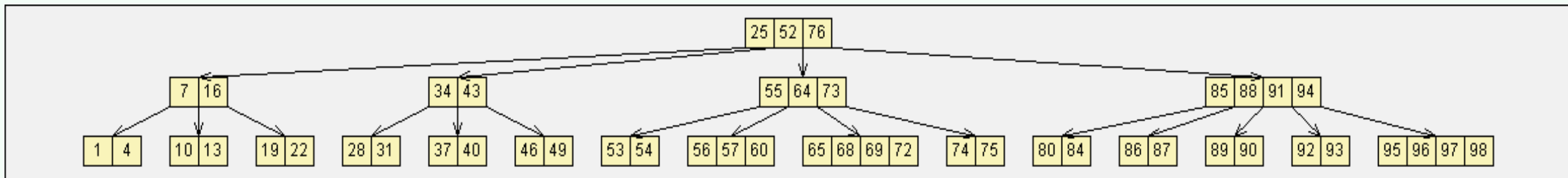
insert(45) //分裂两次



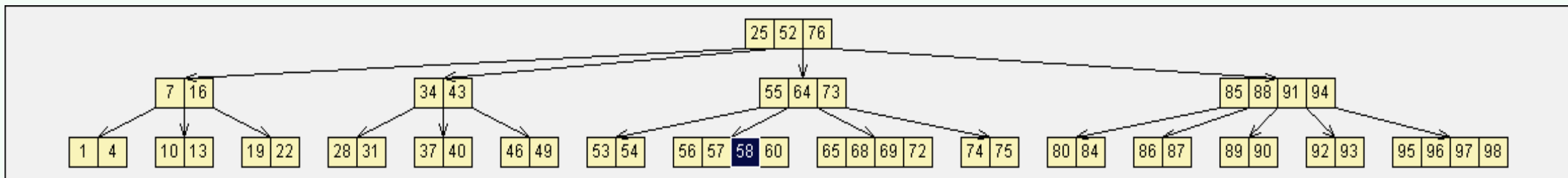
❖ 53 97 36 89 41 75 19 84 77 79 51

实例：(3,5)-树

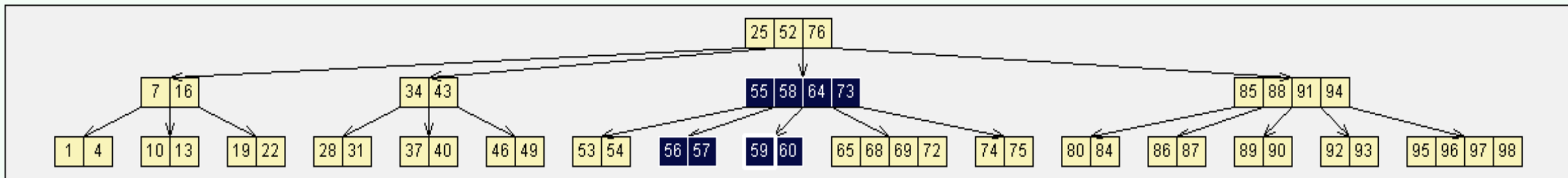
1 4 7 10 13 16 19 22 25 28 31 34 37 40 43 46 49 52 56 60 64 68 72 76 80 84 53 54 55 58 86 87 88 89 90 91 92 93 94 95 96 97 98 73 74 75 57 65 69



insert(58) //无需分裂

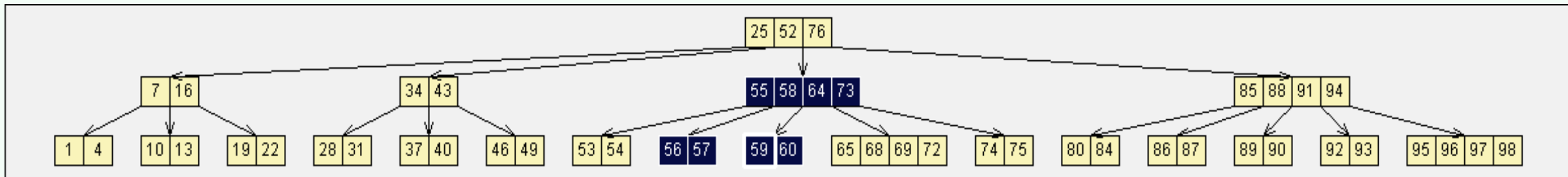


insert(59) //分裂 1 次

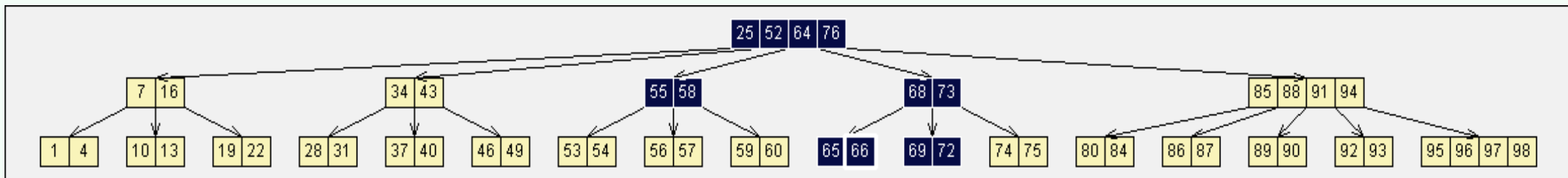


实例：(3,5)-树

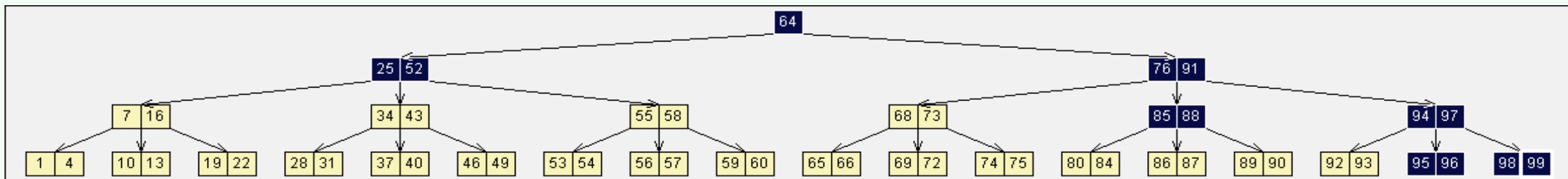
insert(59) //分裂 1 次



insert(66) //分裂 2 次



insert(99) //分裂到根



上溢修复 (1/2)

```
template <typename T> void BTree<T>::solveOverflow( BTNP<T> v ) {  
    while ( _m < v->child.size() ) { //v上溢: |key| == m == |child| - 1  
  
        Rank s = _m/2+1; //以中位数为界, 将v的后缀分裂出来, 创建新节点u  
  
        BTNP<T> u = new BTNode<T>( v->height );  
  
        u->key = Vector<T>(v->key, s); v->key.remove(s, _m);  
  
        u->child = Vector<BTNP<T>>(v->child, s); v->child.remove(s, _m+1);  
  
        u->child.traverse( SetParent<T>( u ) ); //u的孩子们统一指向u  
  
        BTNP<T> p = v->parent; //v的父亲p: 若为空, 则创建之  
  
        if ( ! p ) { _root = p = new BTNode<T>( 1 + v->height, v ); } //全树长高一层
```

上溢修复 (2/2)

```
Rank k = p->child.find( v ); //v在p中的秩
```

```
p->key.insert( k, v->key.remove( s-1 ) ); //轴点上升
```

```
p->child.insert( k+1, u ); u->parent = p; //u与p互联
```

```
v = p; // 上升一层, 如有必要则继续分裂——至多 $O(\log n)$ 层
```

```
} //while
```

```
} //solveOverflow
```

```
template <typename T> struct SetParent { //通过遍历, 统一设置child向量中的x->parent
```

```
    BTNP<T> parent; SetParent( BTNP<T> p ) : parent(p) {}
```

```
    virtual void operator()( BTNP<T> & x ) { if (x) x->parent = parent; }
```

```
}; //SetParent
```

08-B6

高级搜索树

B-树：删除

射影，变了形，反而结晶
或动了情，也要合并，或归了零
也不愿不生不死不悔的倒影

邓俊辉

deng@tsinghua.edu.cn

删除算法

```
template <typename T> bool BTree<T>::remove( const T& e ) {
```

```
    BTNP<T> x = search(e); Rank k = x->key.search(e);
```

```
    if ( e != x->key[k] ) return false; //确认e的确存在
```

```
    if ( x->child[0] ) { //即便x不是叶子，亦可经过如下的腾挪，使之“成为”叶子
```

```
        BTNP<T> s = x->child[k+1]; //在右子树中一路向左，末端叶节点s中的首个关键码
```

```
        while ( s->child[0] ) s = s->child[0]; //就是e的后继
```

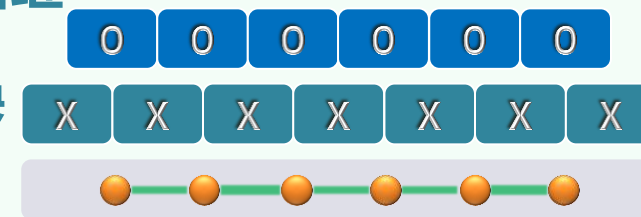
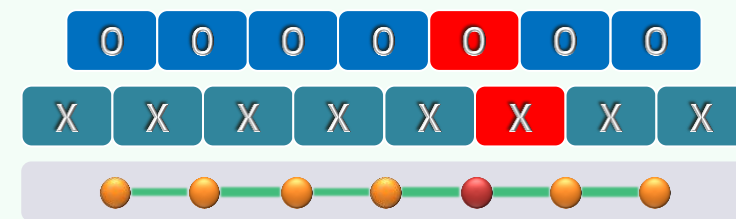
```
        x->key[k] = s->key[0]; x = s; k = 0; //令e与之交换
```

```
    } //至此，x必是叶节点，且e“是”其中的第k个关键码
```

```
    v->key.remove( k ); v->child.remove(); _size--; //删除e以及其下的一个外部节点
```

```
    solveUnderflow( x ); return true; //如下溢，则修复之
```

```
} //remove
```



旋转

❖ 非根节点[V]下溢时，必恰有 $\lceil m/2 \rceil - 2$ 个关键码和 $\lceil m/2 \rceil - 1$ 个分支

❖ 视其左、右兄弟[L]、[R]的规模，可分三种情况加以处理

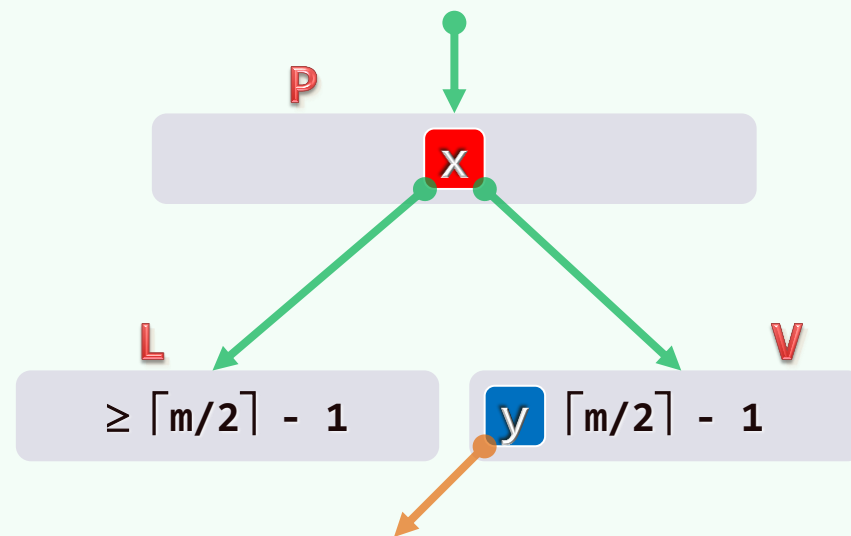
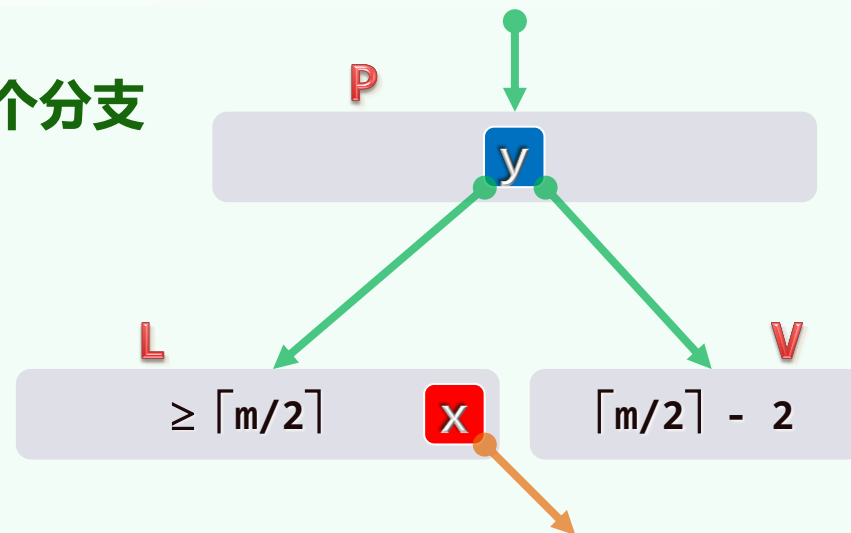
1) 若[L]存在，且至少包含 $\lceil m/2 \rceil$ 个关键码

- 将[P]中的分界关键码[y]移至[V]中（作为最小关键码）
- 将[L]中的最大关键码[x]移至[P]中（取代原关键码[y]）

❖ 如此旋转之后，局部乃至全树都重新满足B-树条件
下溢修复完毕

2) 若[R]存在，且至少包含 $\lceil m/2 \rceil$ 个关键码

- 也可旋转，完全对称



合并

3) L 和 R 或不存在，或均不足 $\lceil m/2 \rceil$ 个关键码——即便如此

- L 和 R 仍必有其一（不妨以 L 为例），且
- 恰含 $\lceil m/2 \rceil - 1$ 个关键码

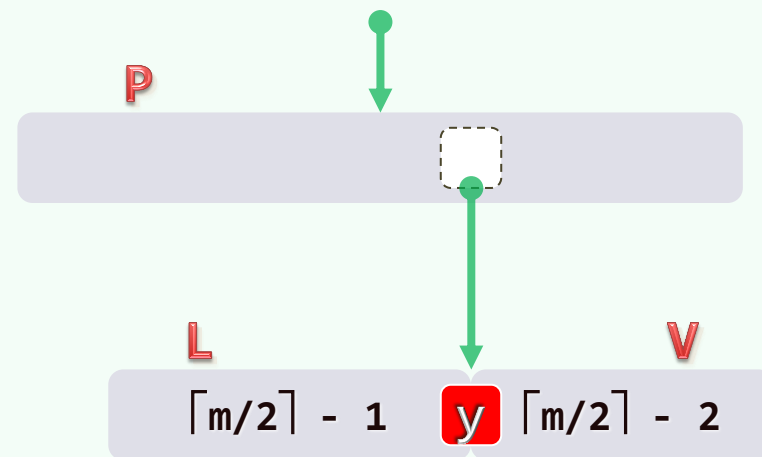
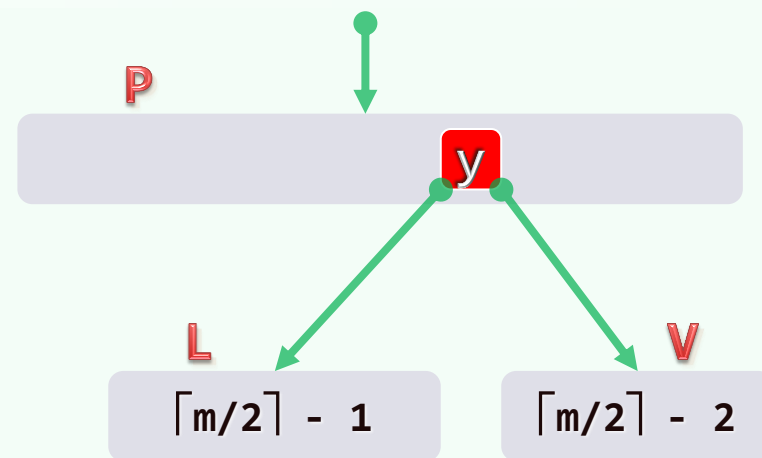
❖ 从 P 中抽出介于 L 和 V 之间的分界关键码 y

- 通过 y 做粘接，将 L 和 V 合成一个节点
- 同时合并此前 y 的孩子引用

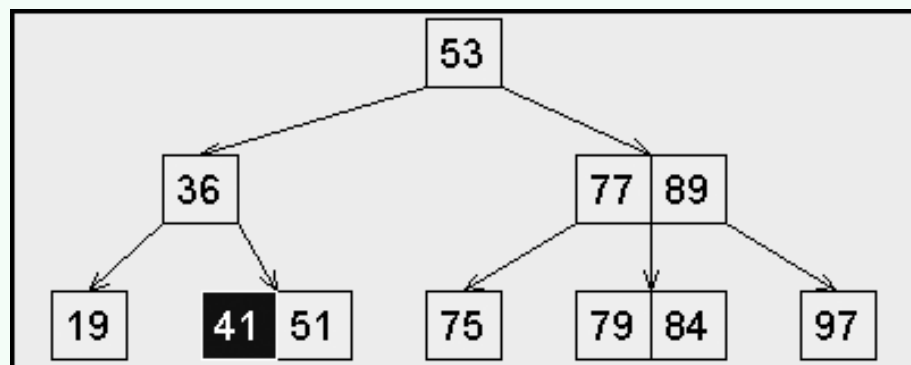
❖ 此处下溢得以修复，但可能继而导致 P 下溢

若果真如此，大可套用前法，继续旋转或合并

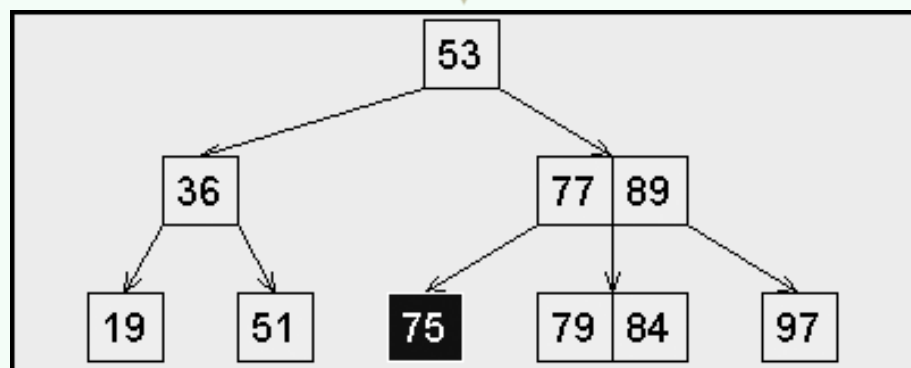
❖ 下溢可能持续发生并向上传播；但至多不过 $O(h)$ 层



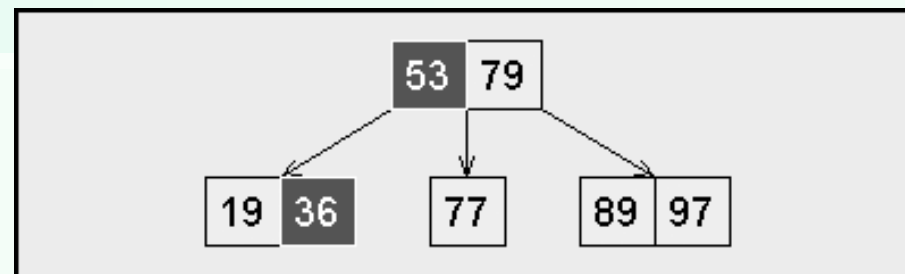
实例：(2,3)-树：底层节点



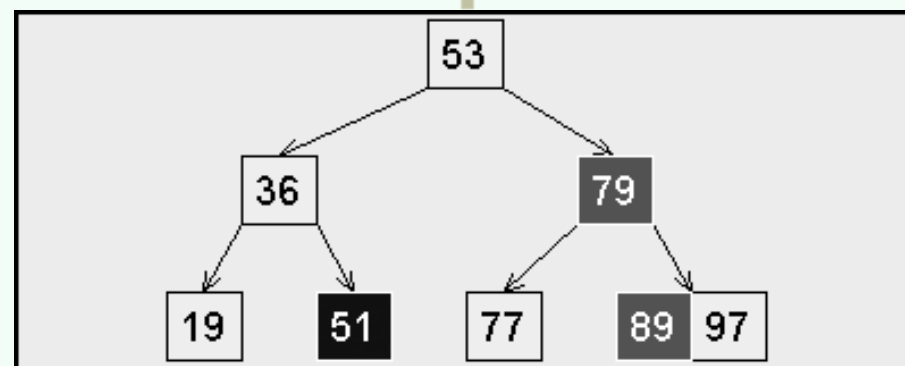
remove(41) //直接删除



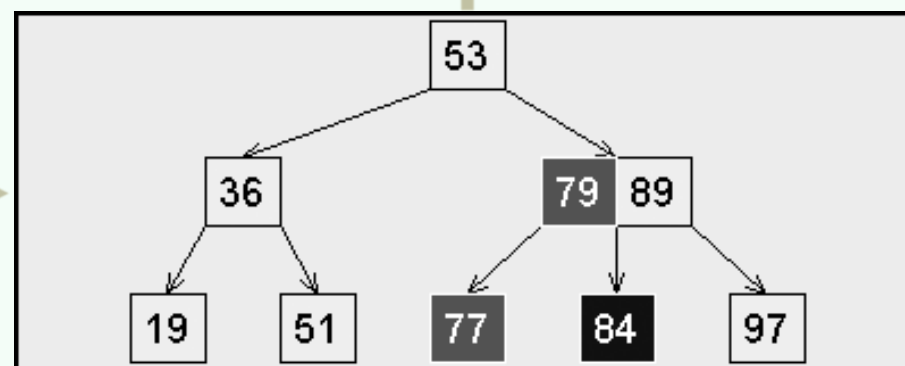
remove(75) //旋转



remove(51) //多次合并



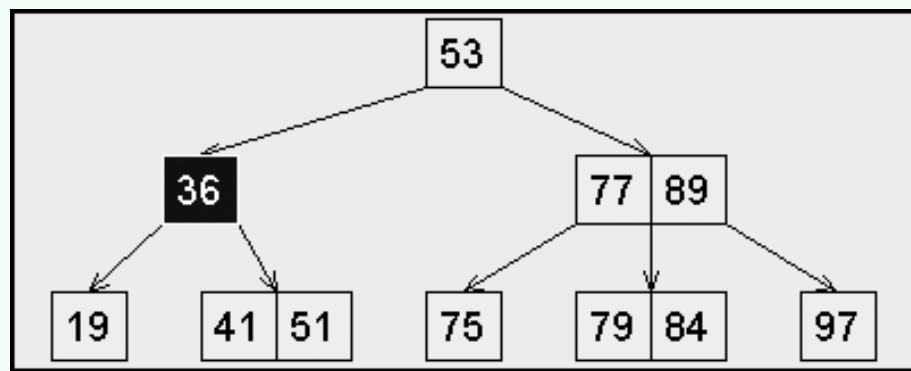
remove(84) //单次合并



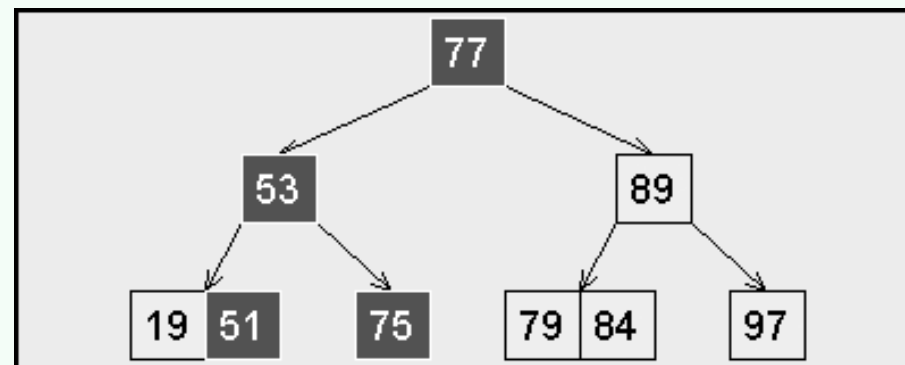
❖ 53 97 36 89 41 75 19 84 77 79 51

实例：(2,3)-树：非底层节点

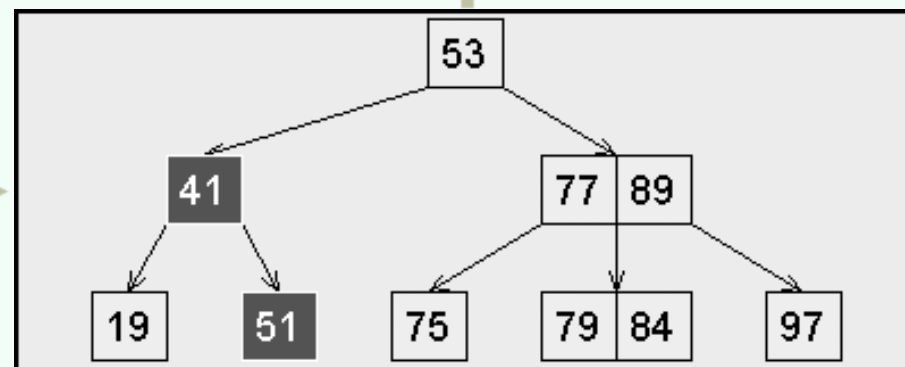
❖ 53 97 36 89 41 75 19 84 77 79 51



remove(36)



remove(41)



下溢修复

```
template <typename T> void BTree<T>::solveUnderflow( BTNP<T> v ) {  
    while ( _m > 2*v->child.size() ) { //v下溢:  $(m-1)/2 == \text{ceil}(m/2)-1 == |child|$   
        BTNP<T> p = v->parent;  
  
        /* 情况#0 : 已到树根, 全树可能降低一层 */      break;  
  
        /* 情况#1L: 通过右旋, 向左兄弟借关键码 */      break;  
  
        /* 情况#1R: 通过左旋, 向右兄弟借关键码 */      break;  
  
        /* 情况#2: 合并 */  
  
        v = p; //上升一层, 继续修复—至多可能上溯 $O(\log n)$ 层、合并 $O(\log n)$ 次  
    } //while  
} //solveUnderflow
```

下溢修复：情况#0：已到树根，可能降低一层

```
while ( _m > 2*v->child.size() ) {  
    BTNP<T> p = v->parent;  
  
    if ( !p ) { //已到树根  
        if ( v->key.size() < 1 && 0 < _size ) //若树根已空（树还没空），则  
            { _root = v->child[0]; _root->parent = NULL; delete v; } //整树降低一层  
        break; //随即收工  
    } //情况#0  
  
    /* 后续情况 */  
  
    v = p; //上升一层，继续修复—至多可能上溯 $O(\log n)$ 层、合并 $O(\log n)$ 次  
}  
//while
```

下溢修复：情况#1L：通过右旋，向左兄弟借关键码

```
Rank k = p->child.find( v ); //确认v系p之第k个孩子
BTNP<T> ls = 0 < k ? p->child[k-1] : NULL; //左兄弟
if ( ls && (_m-1)/2 < ls->key.size() ) { //v有个足够“胖”的左兄弟
    v->key.insert( 0, p->key[ k-1 ] ); //p的轴点关键码转入v（作为最小关键码）
    p->key[ k-1 ] = ls->key.remove(); //ls的最大关键码转入p
    v->child.insert( 0, ls->child.remove() ); //ls的最大孩子过继给v
    if ( v->child[0] ) v->child[0]->parent = v; //作为v的最小孩子
    break; //随即收工
} //情况#1L

/* 后续情况 */
```

下溢修复：情况#1R：通过左旋，向右兄弟借关键码

```
Rank k = p->child.find( v ); //确认v系p之第k个孩子
BTNP<T> rs = k < p->key.size() ? p->child[k+1] : NULL; //右兄弟
if ( rs && (_m-1)/2 < rs->key.size() ) { //v有个足够“胖”的右兄弟
    v->key.insert( p->key[ k ] ); //p的轴点关键码转入v（作为最大关键码）
    p->key[ k ] = rs->key.remove( 0 ); //rs的最小关键码转入p
    v->child.insert( rs->child.remove( 0 ) ); //rs的最小孩子过继给v
    if ( v->child[( _m-1)/2] ) v->child[( _m-1)/2]->parent = v; //作为v的最大孩子
    break; //随即收工
} //情况#1R

/* 后续情况 */
```

下溢修复：情况#2：左、右兄弟均或不存在或不够“胖”，与之合并

```
/* 前述情况 */
```

```
if ( ls ) rs = v;    else { ls = v; k++; } //以ls、rs命名之后，以下将rs归入ls
```

```
ls->key.insert( p->key.remove( k-1 ) ); p->child.remove( k ); //p吐还轴点
```

```
rs->child.traverse( SetParent<T>( ls ) ); //rs的孩子们统一指向ls
```

```
ls->child.join( rs->child ); //过继子树
```

```
ls->key.join( rs->key ); //过继关键码
```

```
delete rs; //右兄弟功成身退
```

```
v = p; //上升一层，继续修复—至多可能上溯 $O(\log n)$ 层、合并 $O(\log n)$ 次
```

```
} //while
```

```
} //solveUnderflow
```

08-C1

高级搜索树

红黑树：动机

所有的过去
都留下了痕迹
哪怕是
一次最微妙的心动
一声最轻渺的叹息

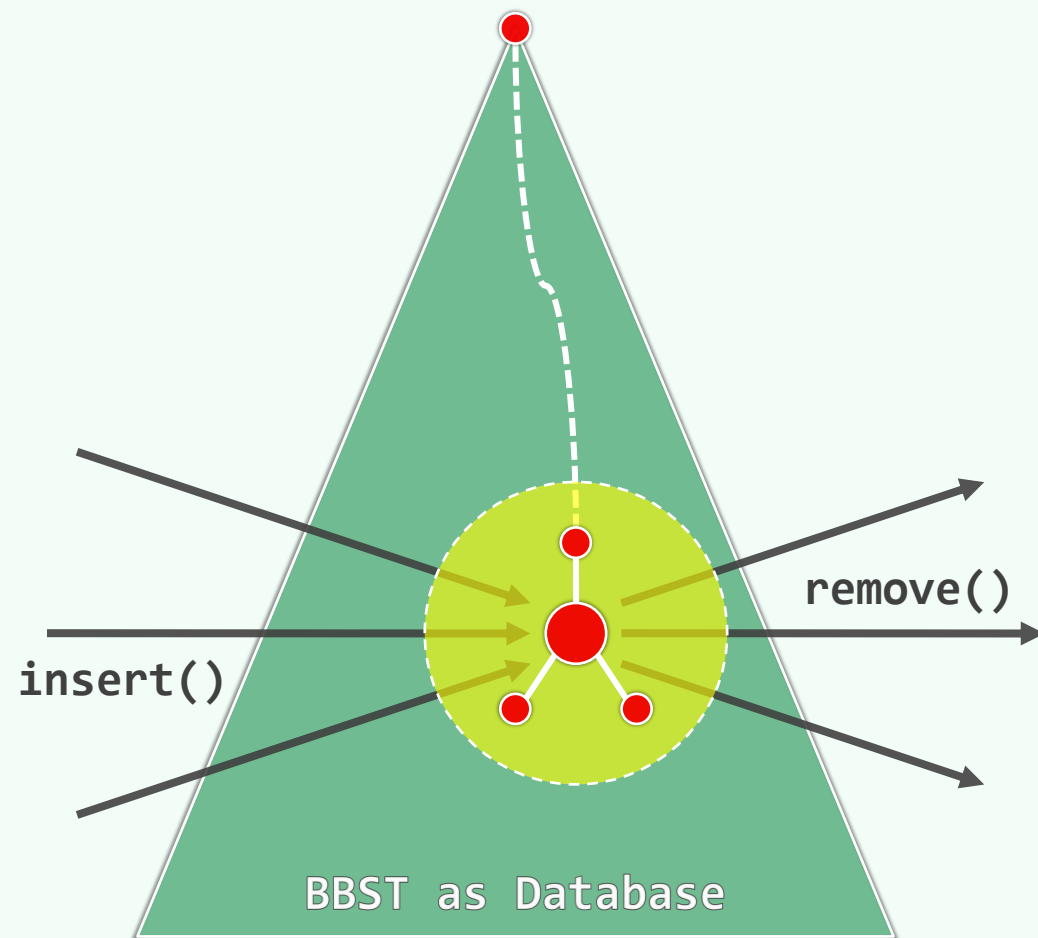
因为过去要进入未来，所以有了故事；因为在深夜里，你会想不起你是怎么从原来走到现在，所以有了故事；当记忆被抹去，当你除了故事就再无任何可以去记忆、可以被记住的东西的时候，因为要有永恒，所以有了故事

邓俊辉

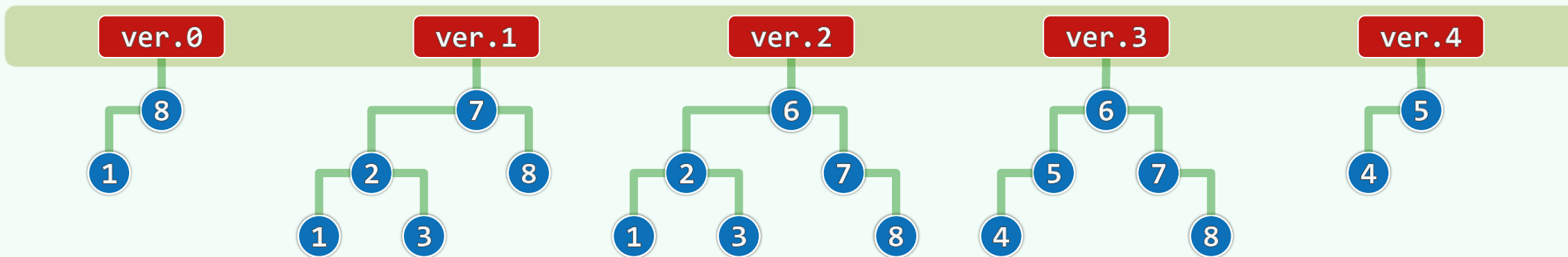
deng@tsinghua.edu.cn

并发性/Concurrency: 单系统 (数据) + 多进程

- ❖ 数据项在修改之前先**加锁**，之后**解锁**
访问延迟主要取决于 “lock ~ unlock” 周期
- ❖ 对于BST而言，每次修改过程中，唯**结构有变** (reconstruction) 处才需加锁
访问延迟主要取决于这类局部之**数量**...
- ❖ Splay: 结构变化剧烈，最差可达 $\mathcal{O}(n)$
- ❖ AVL: `remove()` 时 $\mathcal{O}(\log n)$ —— 尽管
`insert()` 时可保证 $\mathcal{O}(1)$
- ❖ Red-Black: 无论 `insert/remove`, **均不超过** $\mathcal{O}(1)$



❖ 蛮力实现: 每个版本独立保存; 各版本自成一个搜索结构



Fully Persistent: search(**ver**, key); insert(**ver**, key); remove(**ver**, key)

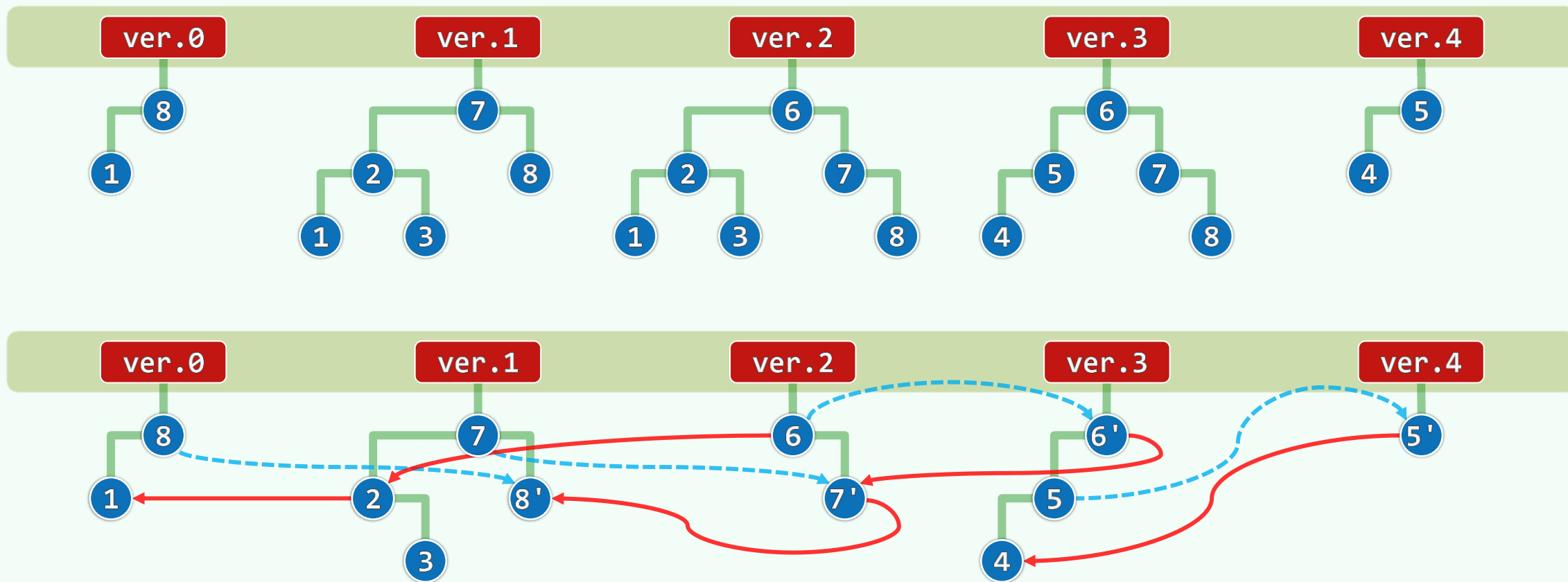
❖ 单次操作 $\mathcal{O}(\log h + \log n)$, 累计 $\mathcal{O}(n \cdot h)$ 时间/空间 // $h = |\text{history}|$

❖ 挑战: 可否将**空间**复杂度控制在 $\mathcal{O}(n + h \cdot \log n)$ 以内, 甚至更低?

❖ 可以! 为此需利用相邻版本之间的**相关性**...

压缩存储：大量**共享**，少量**更新**：每个版本的新增复杂度，仅为 $\mathcal{O}(\log n)$

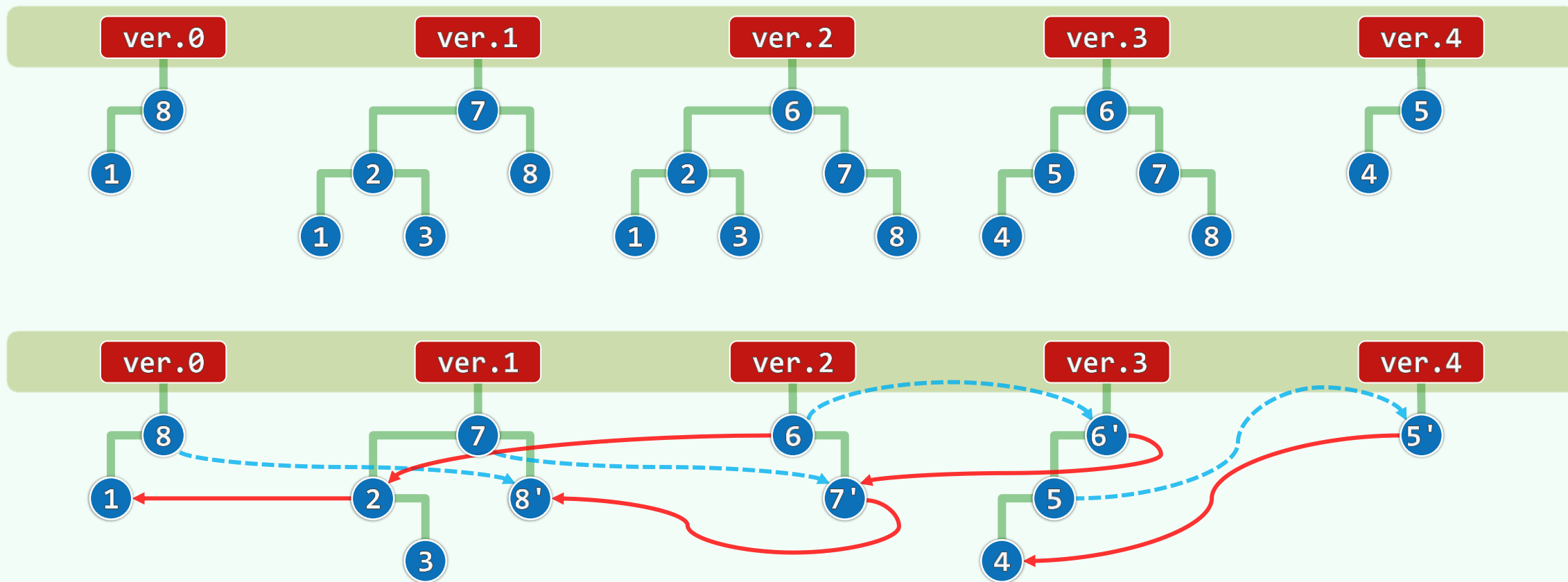
❖ **Partial Persistence**：仅支持对历史版本的**读取** // 监控录像、飞行器黑盒子、代码版本管理...



❖ 这类情况下，还可进一步提高至总体 $\mathcal{O}(n + h)$ 、单版本 $\mathcal{O}(1)$...

重构: $O(1)$ 次旋转 ~ $O(1)$ 新增空间

❖ 为此, 就树的**代数结构**的而言, 相邻版本之间的差异不能超过 $O(1)$



❖ 很遗憾, AVL、Splay等BBST**均不具备**这一性质; 须另辟蹊径...

java.util.TreeMap

```
import java.util.*;

public class TestTreeMap {

    public static void main( String[] args ) {

        TreeMap scarborough = new TreeMap();

        scarborough.put( "P", "parsley" );
        scarborough.put( "S", "sage" );
        scarborough.put( "R", "rosemary" );
        scarborough.put( "T", "thyme" );

        System.out.println( scarborough );

    }

}
```

08-C2

高级搜索树

红黑树：结构

这时，我看见两只大蚂蚁，一只红不棱登，另一只个儿特大，差不离有半英寸长，是黑不溜秋的，它们两个正在相互凶殴...

把照妖鏡來照這廝誰真誰假，教他假滅真存

邓俊辉

deng@tsinghua.edu.cn

规则：0~3

❖ [0]. 由红、黑两类节点组成

```
#define RBT_BLACK 1
```

```
#define RBT_RED 0
```

❖ [1]. 树根必为黑色

```
_root->color == RBT_BLACK
```

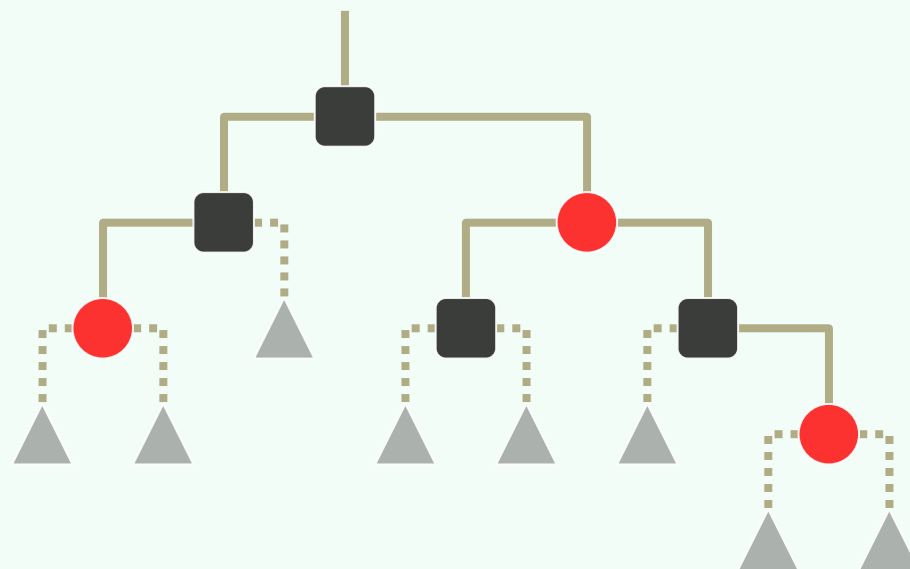
❖ 统一引入外部节点NULL，成为真二叉树

❖ [2]. 外部节点均为黑色

```
#define Color(x) ( (x) ? (x)->color : RBT_BLACK )
```

❖ [3]. 红节点为只能有黑孩子（及黑父亲）

```
false == IsRed(x) && ( IsRed(x->lc) || IsRed(x->rc) || IsRed(x->parent) )
```



规则：4

❖ [4]. 从同一节点x到其每个后代NULL，沿途经过的黑节点一样多

❖ 这个数目（不计x自身）称作x的黑高度

```
#define H_NULL 0 //空的红黑树（黑）高度为0
```

```
#define Height(x) ( (x) ? (x)->height : H_NULL )
```

❖ 顾名思义，只有黑后代才会对黑高度有贡献

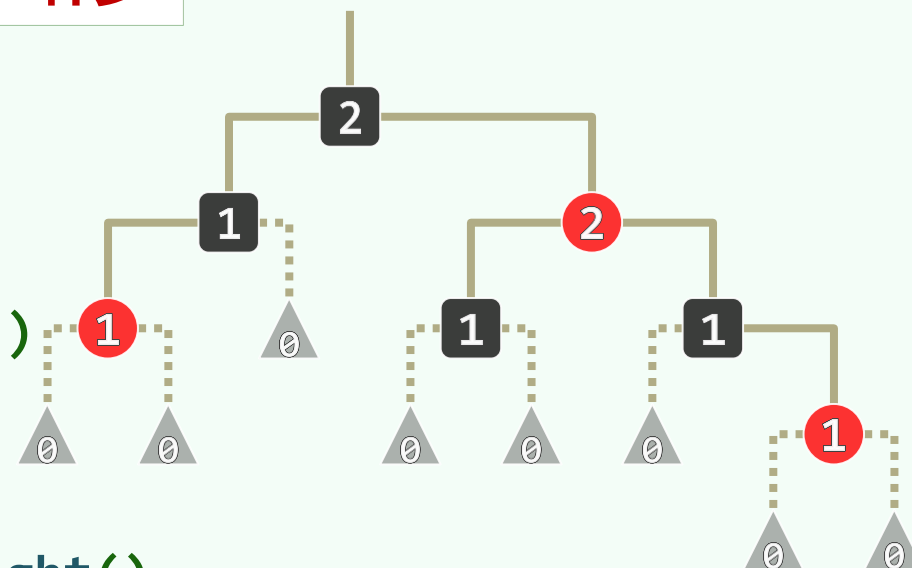
```
template <typename T> int BinNode<T>::updateHeight()
```

```
{ return height = max( HeightL(this), HeightR(this) ); }
```

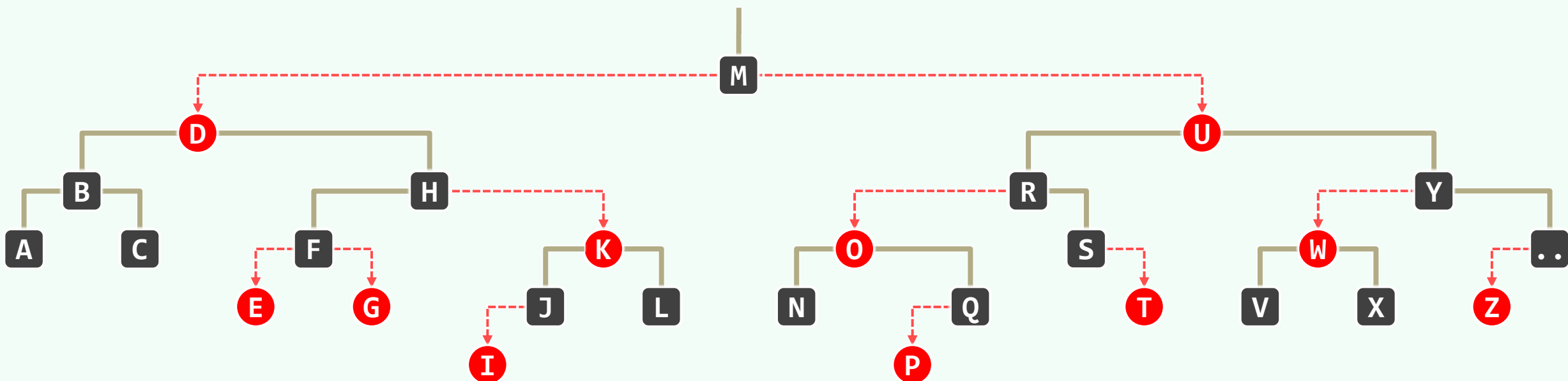
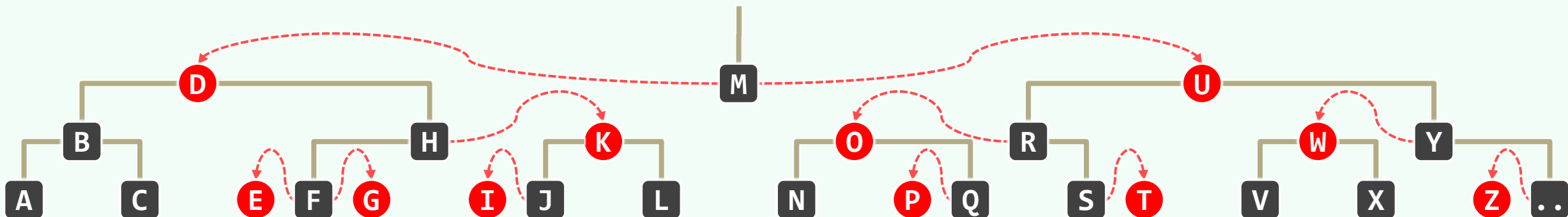
```
#define HeightL(x) ( Height((x)->l) + Color((x)->l) )
```

```
#define HeightR(x) ( Height((x)->r) + Color((x)->r) )
```

```
#define RbtBalanced(x) ( !(x) || ( HeightL(x) == HeightR(x) ) )
```

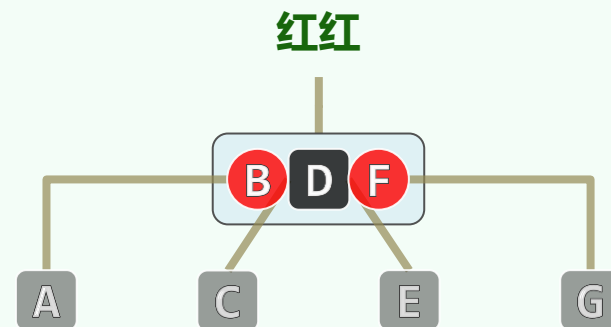
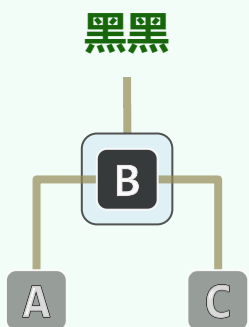
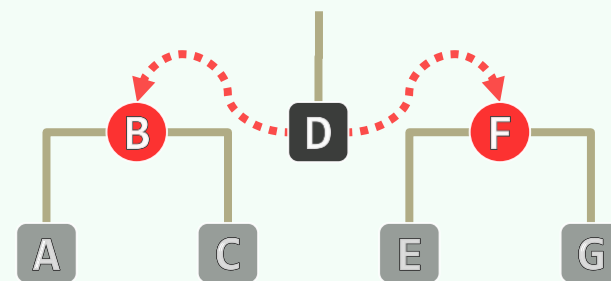
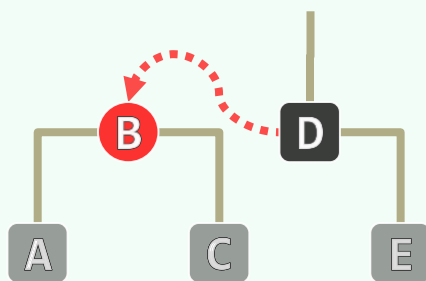
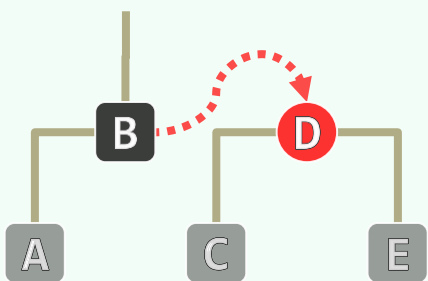
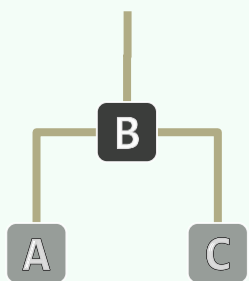


直观理解：将红节点**提升**至与其（黑）父亲等高，红边**折叠**起来...



每棵红黑树，都对应于一棵(2,4)-树

❖ 将黑节点与其红孩子视作**关键码**，再**合并**起来... //元音 + 辅音 = 音节



❖ 无非四种**组合**，各对应于4阶B-树的一类**超级节点** //反过来呢？

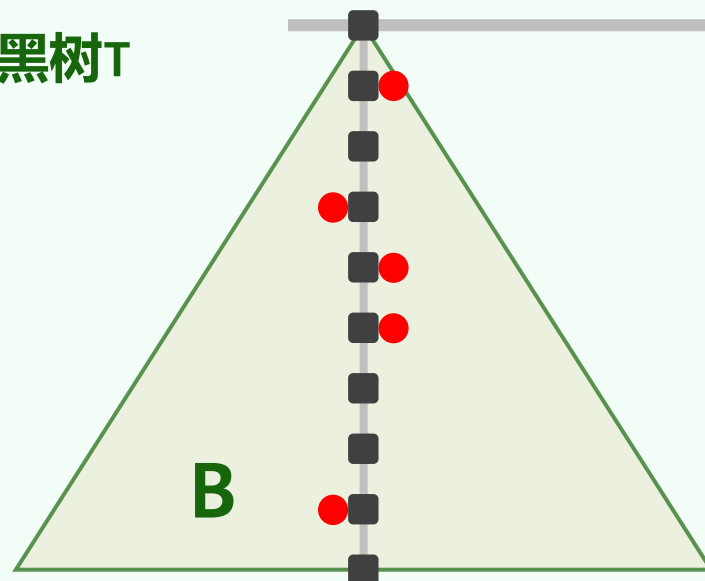
红黑树也是BBST

❖ 断言：包含 n 个内部节点的红黑树 T

$$\text{高度 } h = \mathcal{O}(\log n)$$

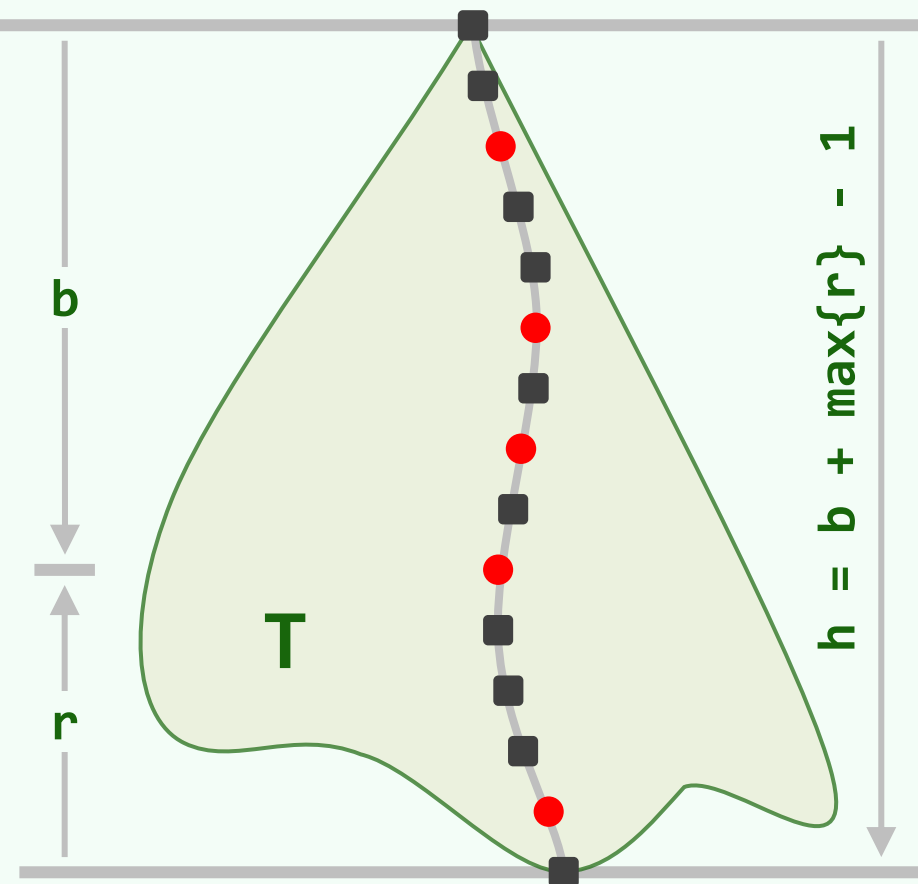
❖ T 的黑高度若记作 b ，则也是

对应的(2,4)-树 B 的高度



❖ 从 T 的根出发，在抵达任一NULL之前，会经过

恰好 b 个黑节点，以及 $r \leq b$ 个红节点



❖ 于是： $h = b + \max\{r\} - 1 < 2 \cdot b = \mathcal{O}(\log n)$ // 常系数 = ?

RedBlack模板类

```
template <typename T> class RedBlack : public BST<T> { //基于BST红黑树

    public:

        BNP<T> insert( const T& ); //插入 (重写)

        bool remove( const T& ); //删除 (重写)

        //BST::search()等其余接口可直接沿用

    protected:

        void solveDoubleRed(); //双红修正

        void solveDoubleBlack(); //双黑修正

}; //RedBlack
```

08-CS

高级搜索树

红黑树：插入

莫赤匪狐，莫黑匪烏；惠而好我，攜手同車

有理走遍天下，无理寸步难行

邓俊辉

deng@tsinghua.edu.cn

插入与双红

```
template <typename T> BNP<T> RedBlack<T>::insert( const T & e ) {
```

```
    BST::insert( e ); //先按BST规则插入新节点_x并染红，则条件1、2、4依然满足
```

```
    if ( _p != _x ) solveDoubleRed(); //但3未必，或需做双红修正
```

```
    return _x;
```

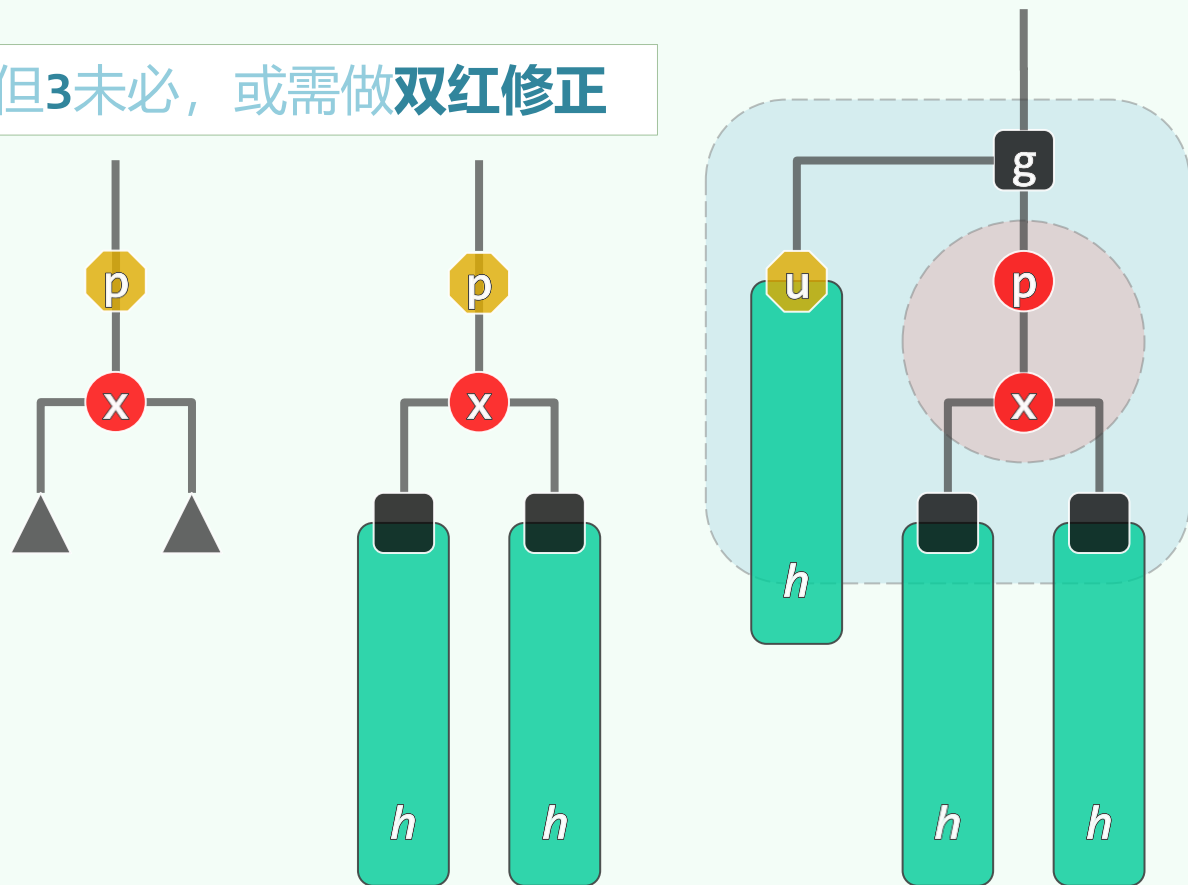
```
} //insert
```

❖ 双红: $p \rightarrow \text{color} == x \rightarrow \text{color} == \text{RBT_RED}$

❖ 祖父: $g = p \rightarrow \text{parent}$ //必存在，且必黑

叔父: $u = \text{Uncle}(x) = \text{Sibling}(p)$

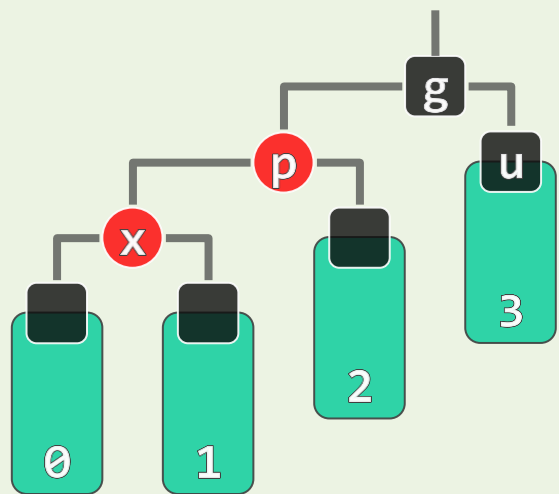
❖ 视 u 的颜色（无非两种），分别加以处理...



双红修正

```
template <typename T> void RedBlack<T>::solveDoubleRed( BNP<T> x ) {  
    for ( BNP<T> x = _x; ; ) {  
        if ( !x->parent ) { x->color = RBT_BLACK; break; } //调整至根  
        BNP<T> p = x->parent; if ( IsBlack( p ) ) break; //x之父p为黑  
        BNP<T> g = p->parent; //否则, x之祖父g必存在且黑  
        BNP<T> u = Uncle( x ); //以下, 视x之叔父u的颜色分别处理  
        if ( IsBlack( u ) ) { /* RR-1: u为黑 (或NULL) */ }  
        else { /* RR-2: u为红 */ }  
    } //for  
} //solveDoubleRed
```

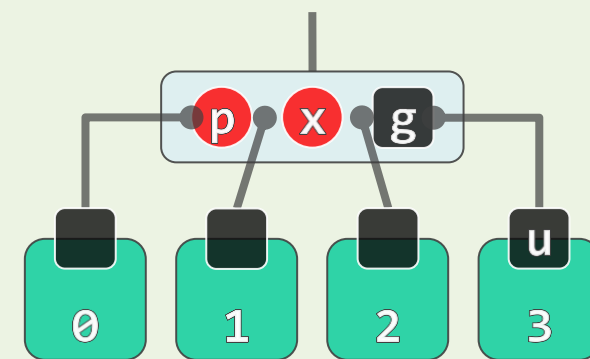
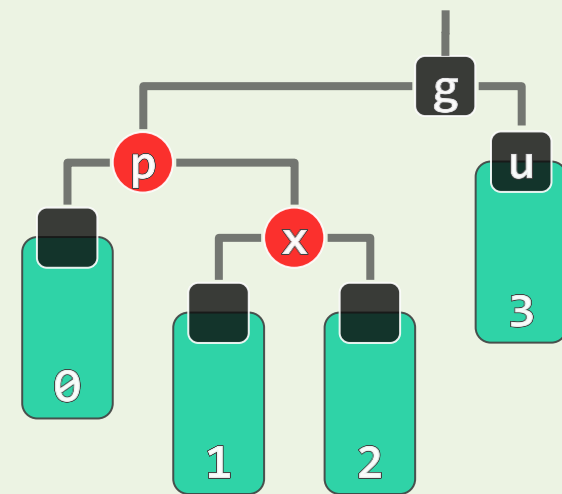
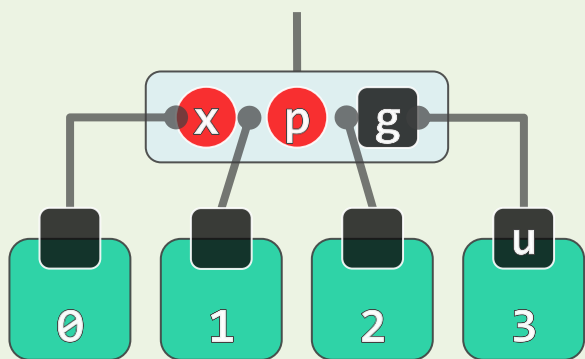
RR-1: $u \rightarrow \text{color} == B$



❖ 此时, x 、 p 、 g 的四个孩子
(可能是外部节点)

- 全为黑, 且
- 黑高度相同

❖ 另两种对称情况, 自行补充



RR-1: 复衡算法及原理

❖ rotate(g)

b转黑, a或c转红

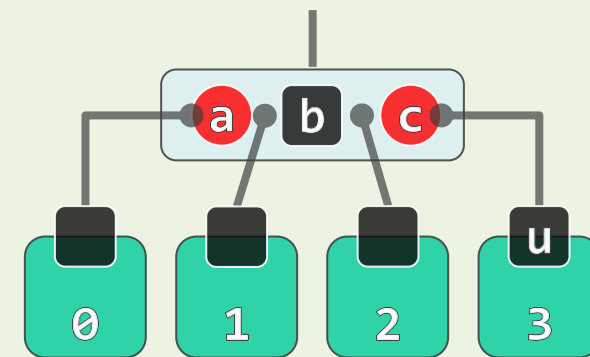
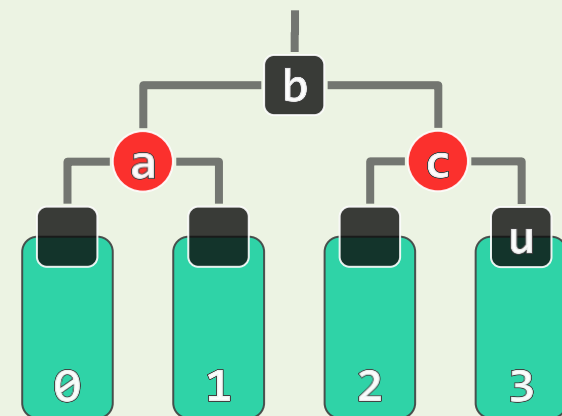
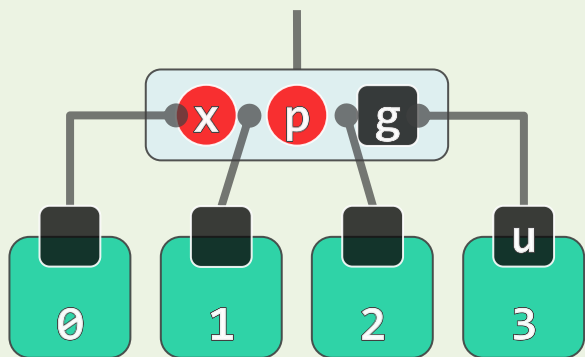
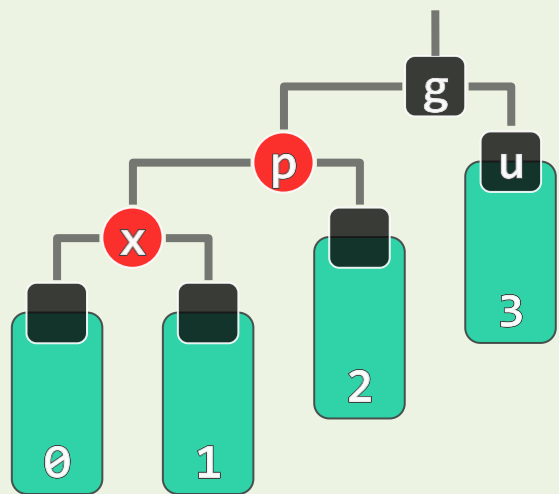
❖ 从B-树的角度, 如何理解?

所谓“非法”, 无非是...

❖ 在某**三叉**节点中插入红关键码后
原黑关键码不再居中 (RRB或BRR)

❖ 调整的效果, 无非是
将三个关键码的颜色改为**RBR**

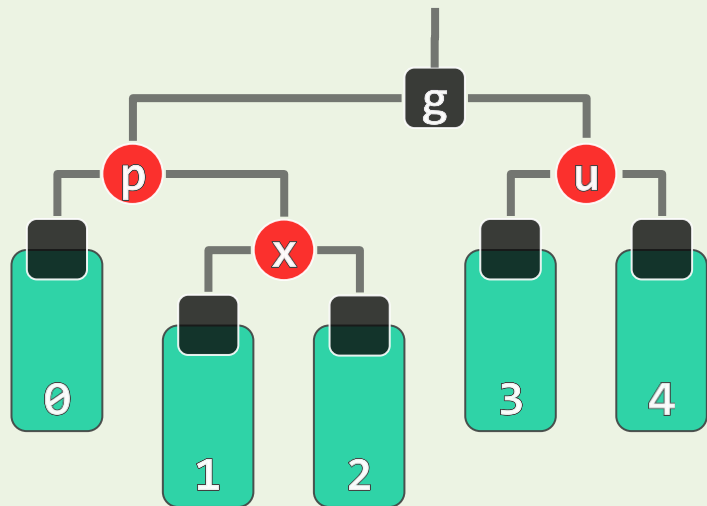
❖ 如此调整, **一蹴而就**



RR-1: 复衡算法之实现

```
for ( BNP<T> x = _x; ; ) {  
    /* ..... */  
    if ( IsBlack( u ) ) { //RR-1: u为黑或NULL  
        BNP<T> b = rotate( g ); //绕g旋转后  
        b->color = RBT_BLACK; //根节点染黑  
        b->lc->color = b->rc->color = RBT_RED; //孩子们染红  
        break; //即完成修复  
    } else { /* RR-2: u为红 */ }  
} //for
```

RR-2: $u \rightarrow \text{color} == R$

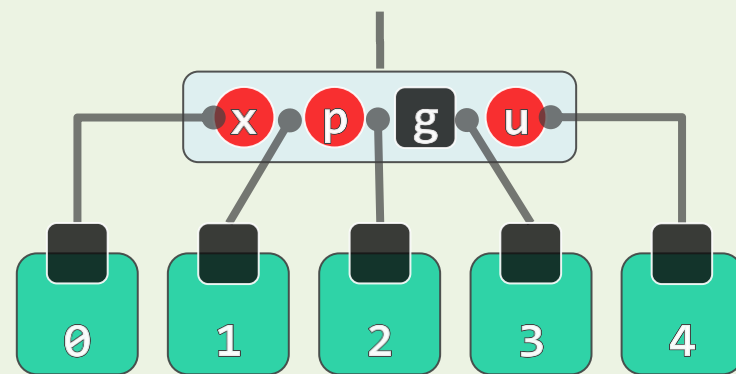
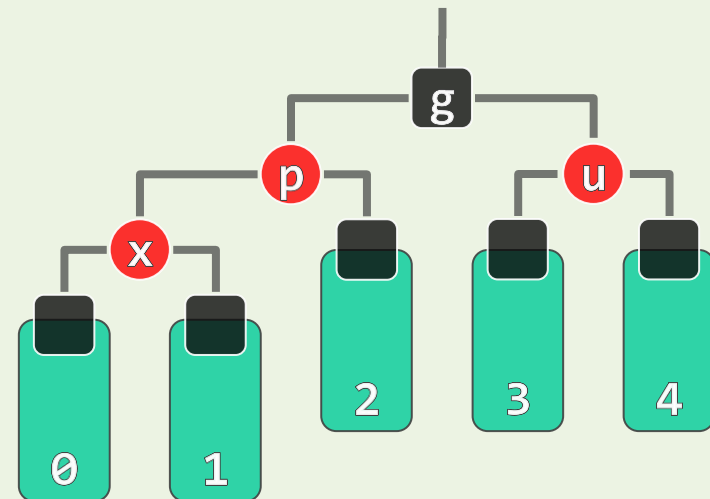
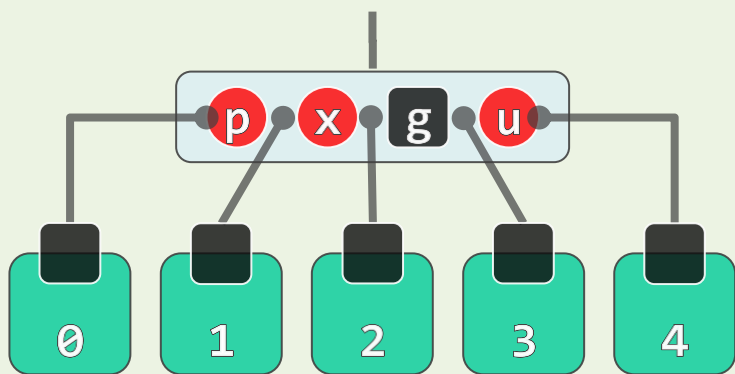


❖ 在B-树中，等效于

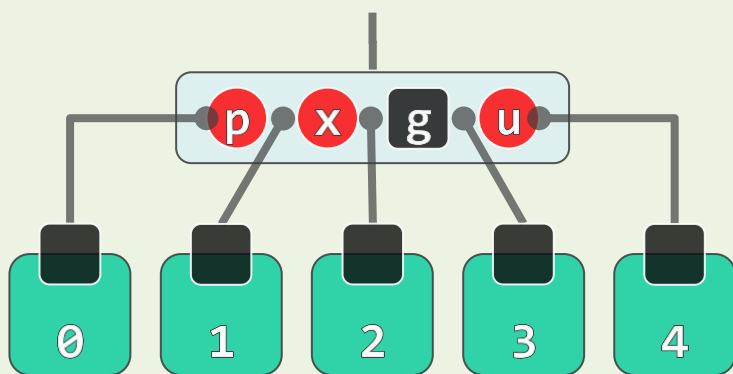
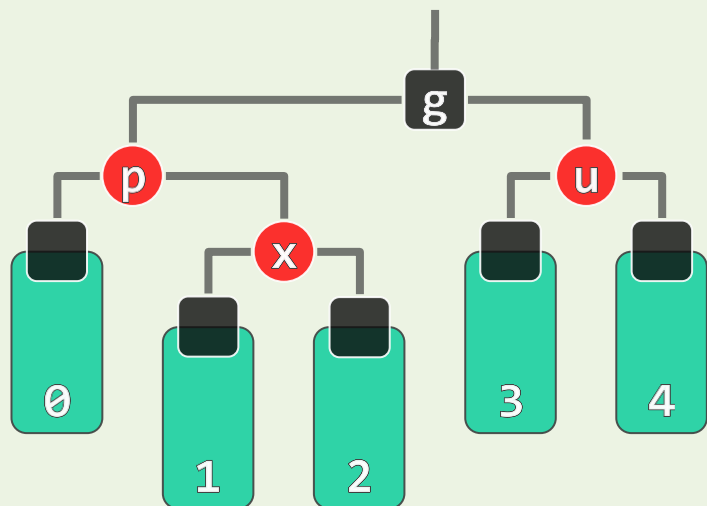
超级节点发生上溢

❖ 另两种对称情况

请自行补充



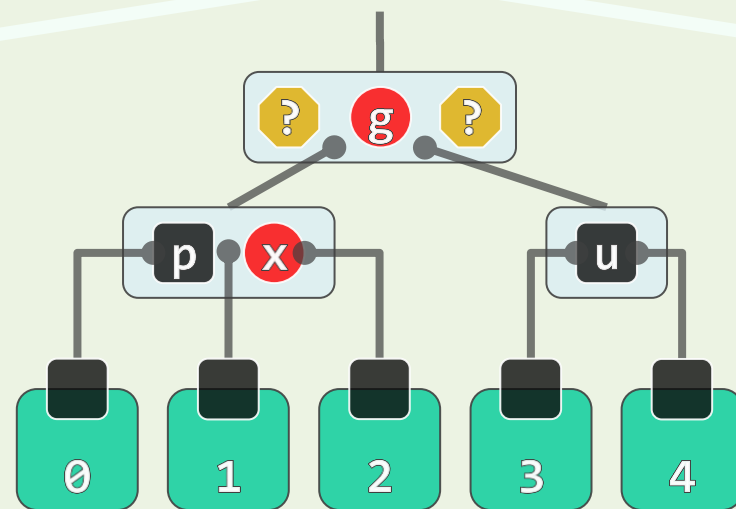
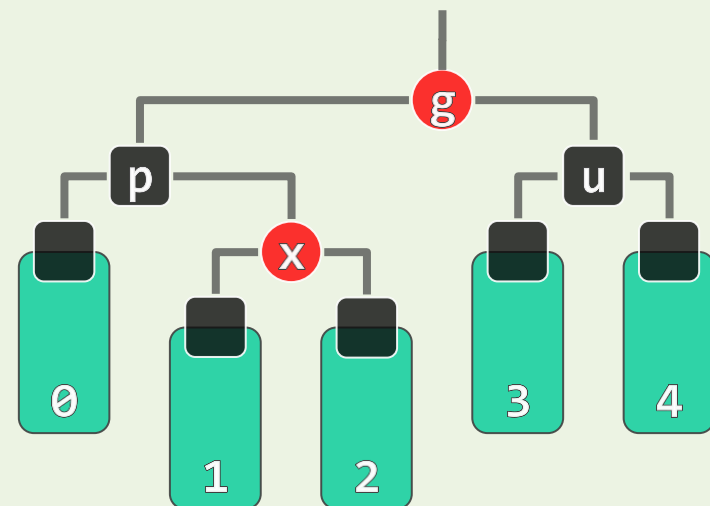
RR-2: 复衡算法及原理



❖ p与u转黑, g转红
在B-树中, 等效于...

❖ 节点分裂
关键码g上升一层

❖ 红黑树并无旋转
结构保持不变



RR-2: 可能向上蔓延, 直至树根

❖ 既是分裂, 也应**可能**会继续向上传播: g 与 $g \rightarrow \text{parent}$ **再次构成双红**

❖ 果真如此, 只需**等效**地将 g 视作新插入节点

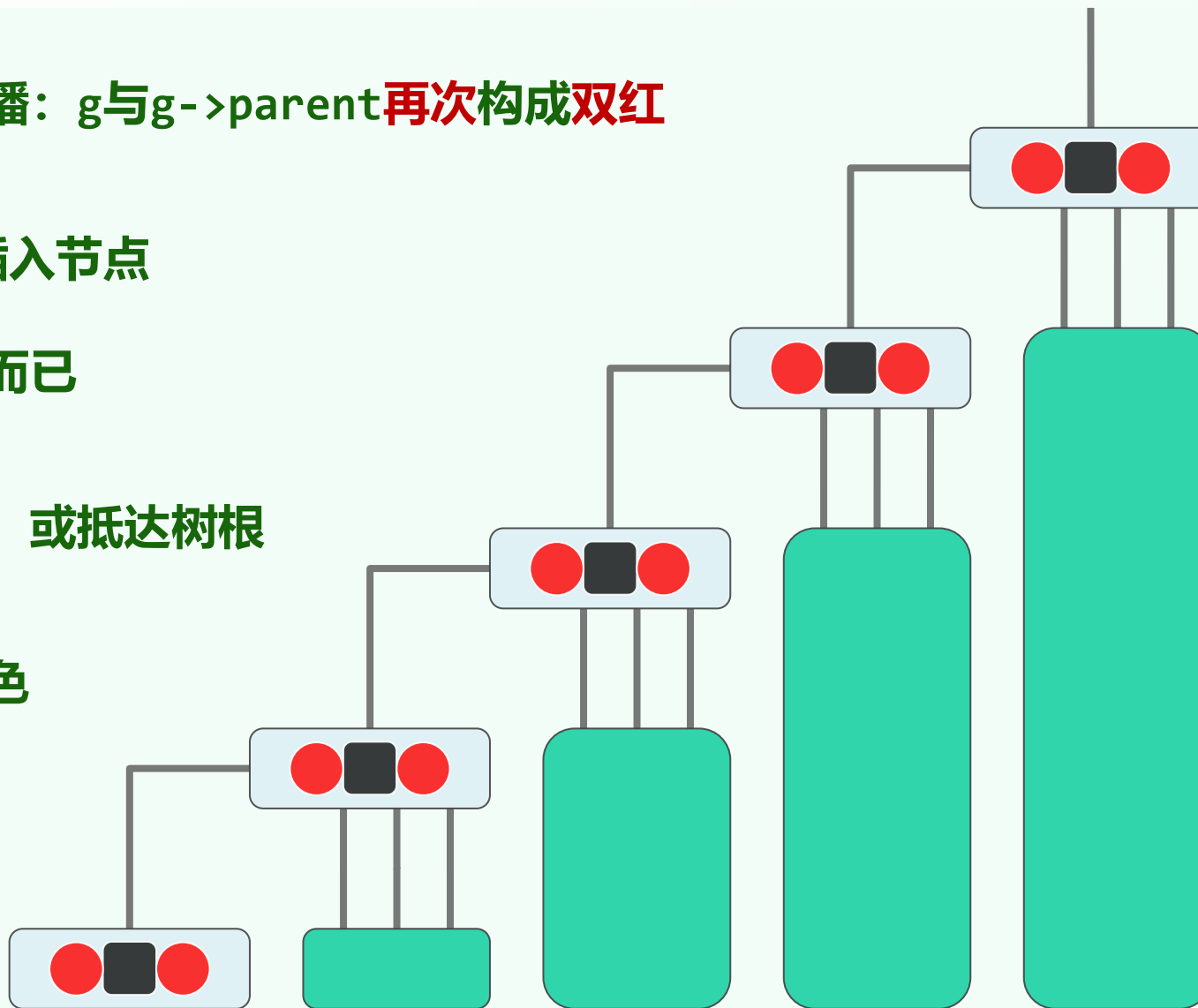
无非还是以上两种情况, 如法处置而已

❖ 直到所有条件均得满足: 不再双红, 或抵达树根

❖ 果真蔓延至根, 则强行将其转为黑色

整树黑高度随之加一

❖ 所幸, 树高不过 $O(\log n)$



RR-2: 复衡算法之实现

```
for ( BNP<T> x = _x; ; ) {
```

```
    /* ..... */
```

```
    if ( IsBlack( u ) ) { //RR-1: u为黑 (含NULL)
```

```
    } else { //RR-2: u为红色
```

```
        p->color = u->color = RBT_BLACK; //p、u由红转黑
```

```
        g->color = RBT_RED; g->height++; //在B-树中g是上交的关键码，故染红
```

```
        x = g; //继续上溯
```

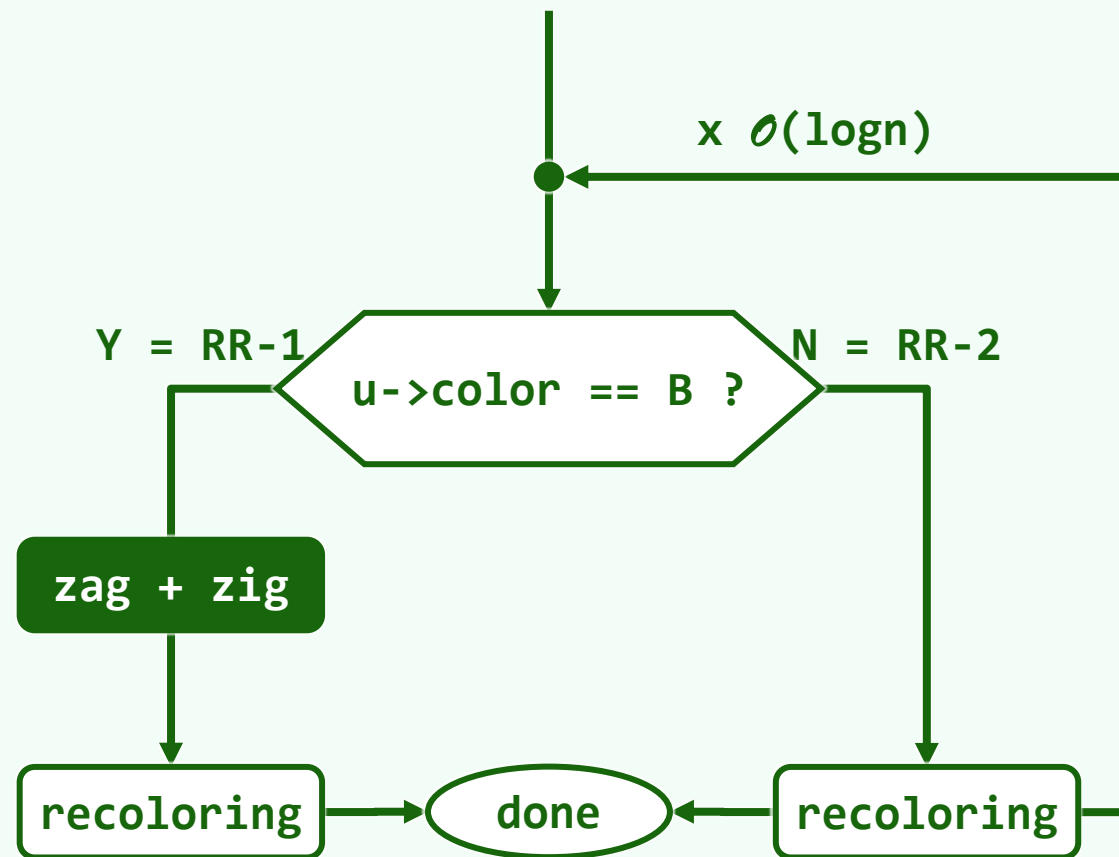
```
    } //else
```

```
} //for
```

复杂度

- ❖ 重构、染色均只需常数时间，故只需统计其总次数
- ❖ `RedBlack::insert()` 仅需 $O(\log n)$ 时间
- ❖ 其间至多做 $O(\log n)$ 次重染色、 $O(1)$ 次旋转

	旋转	染色	此后
u为黑	1~2	2	调整随即完成
u为红	0	3	可能再次双红 但必上升两层



08-C4

高级搜索树

红黑树：删除

近朱者赤，近墨者黑

他仿佛这一刻才第一次看见这些颜色，并为它们取下崭新又美妙的名字。在这里，没有人会在冬天时哀悼已逝去的夏天或春天

邓俊辉

deng@tsinghua.edu.cn

等效删除

❖ 首先按照BST的规则做删除

不妨设 $_p$ 的孩子 $_x$ 被删除，替代者为 $_r$

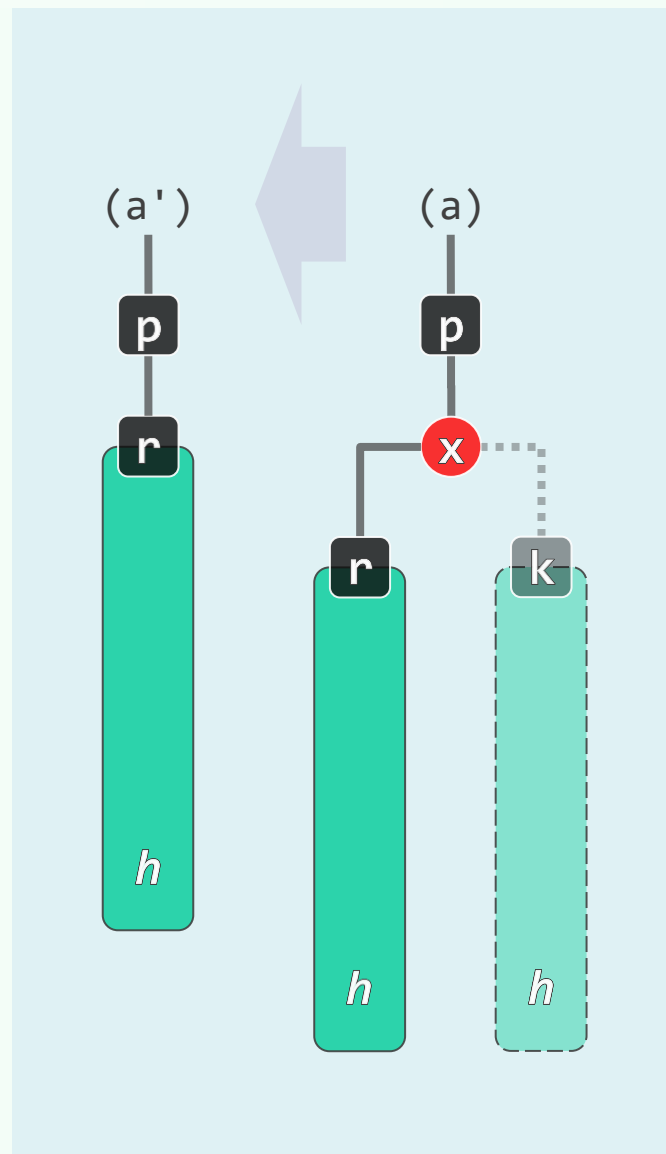
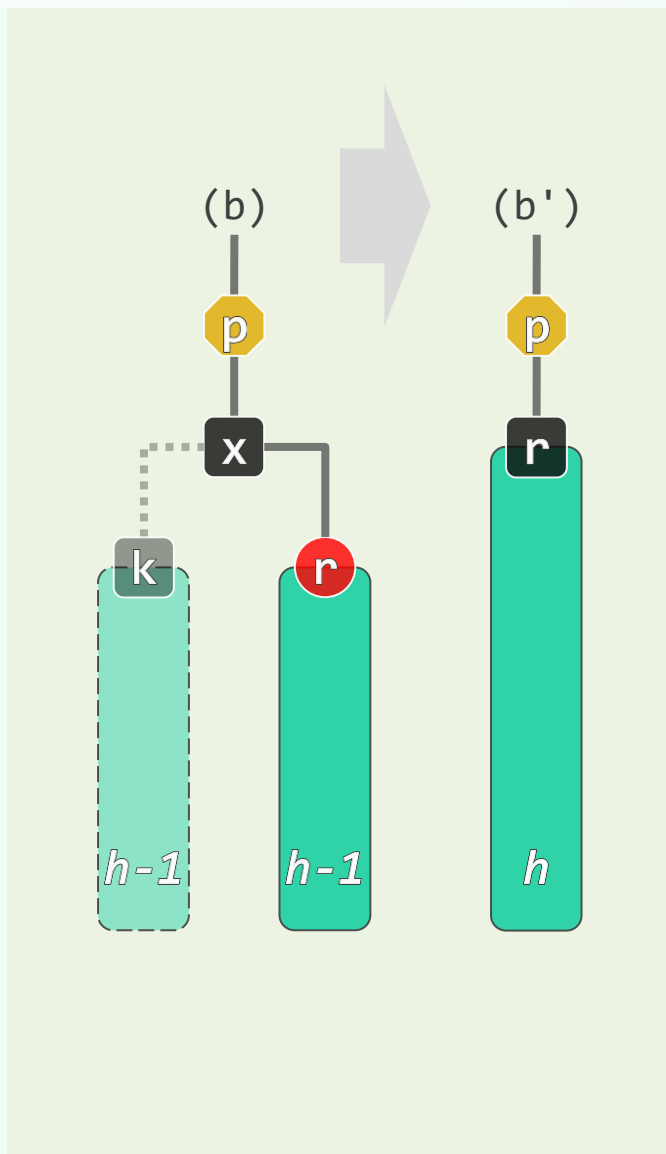
❖ 在随后的重平衡过程中

作为事件焦点的 $_x$ 会逐层上升

❖ 为统一描述和简化实现

可**假想地、等效地**理解为：

- 有一棵黑高度与 $_r$ 相等的子树 k
随同 $_x$ **一并**被摘除
- 尽管实际上， k **从未**存在过



删除算法

```
template <typename T> bool RedBlack<T>::remove( const T & e ) {  
    if ( !BST::remove( e ) ) return false; //先按BST规则删除_p的孩子_x, 接替者为_r  
    if ( !RbtBalanced( _p ) ) //若失衡 (即原_x为黑)  
        if ( IsRed( _r ) ) //若接替者_r为红, 则  
            _r->color = RBT_BLACK; //只需令其继承_x的黑色, 黑高度随即复原  
        else //否则 (即原_x及_r均为黑色)  
            solveDoubleBlack(); //从_p出发, 做双黑修正  
    return true;  
} //remove
```

其一为红

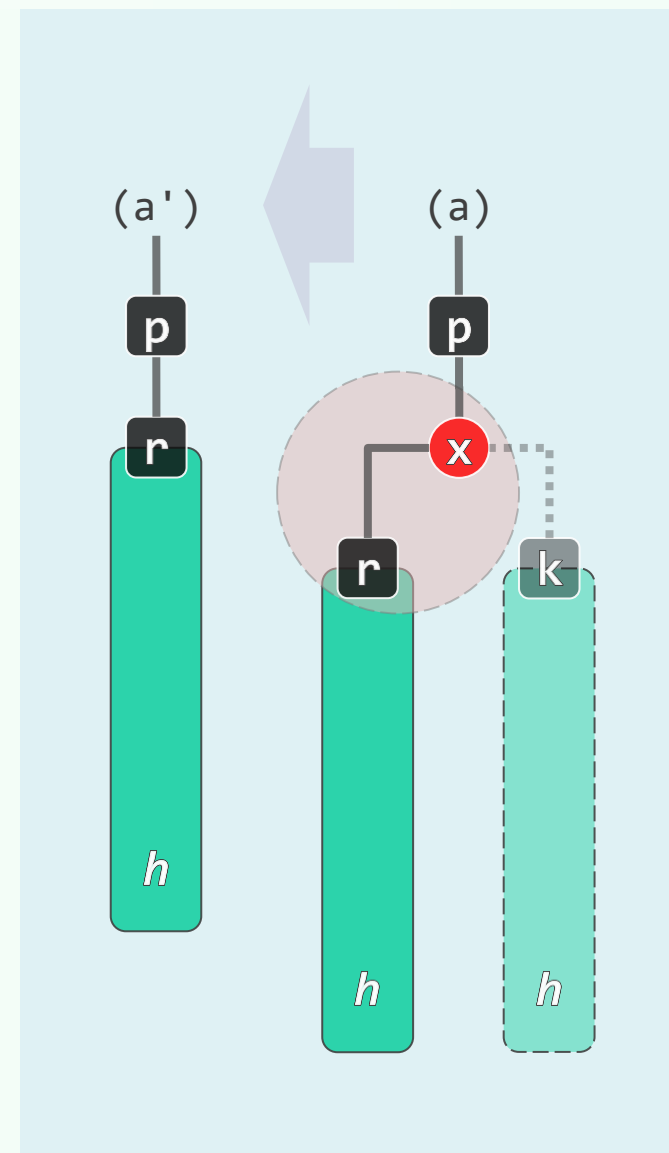
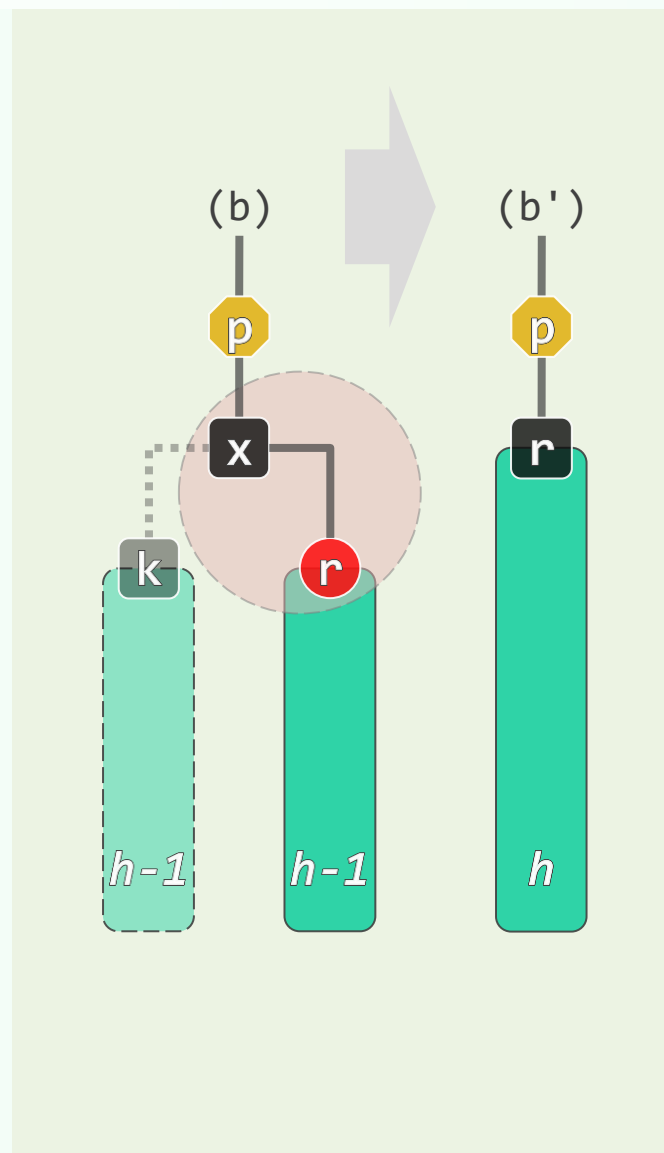
❖ 完成BST::remove()之后

- 红黑树的条件1、2依然满足
- 但条件3、4却不见得

❖ 在原树中

- 若_x为红，则条件3与4自然满足
- 若_r为红，则令其继承_x的黑色...

❖ 总之，无论_x或_r为红 //当然，不会都红 则3与4均不难满足——删除遂告完成！



双黑

❖ [Double Black]

`x->color == r->color == RBT_BLACK`

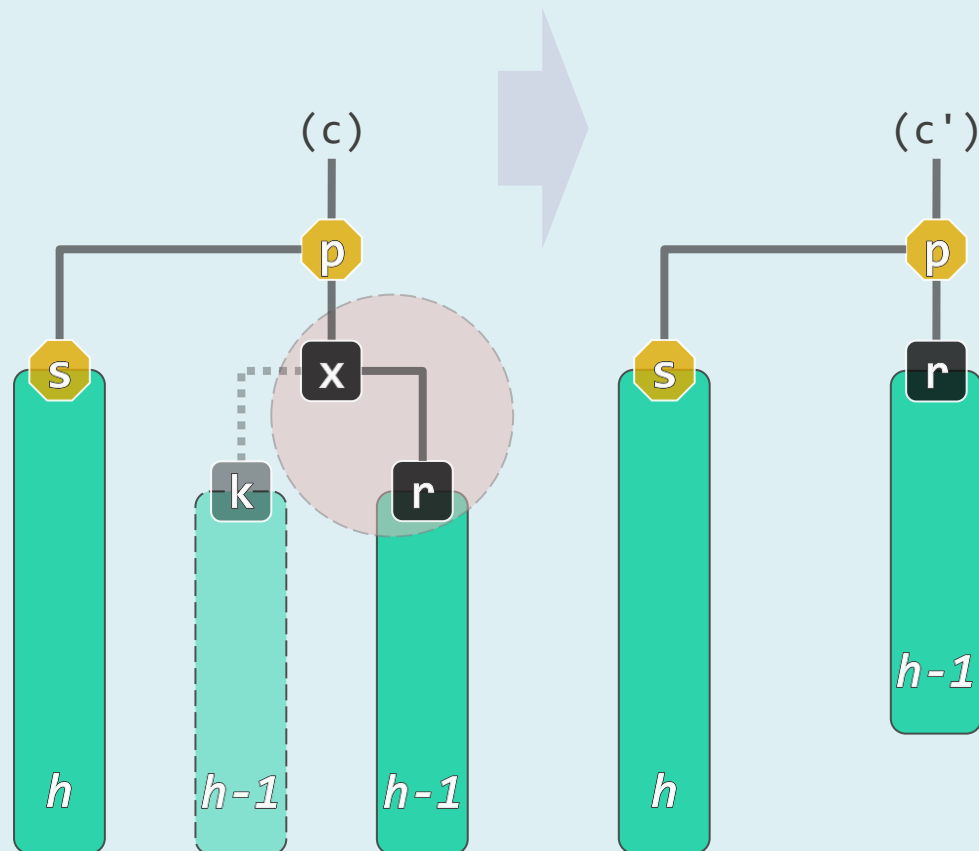
❖ 摘除_x并代之以_r后，全树黑深度不再统一

//稍后可见，等效于B-树中x所属节点的下溢

❖ 在新树中，考查_r的

- 父亲: `_p = _r->parent`
- 兄弟: `s = sibling( _r )`

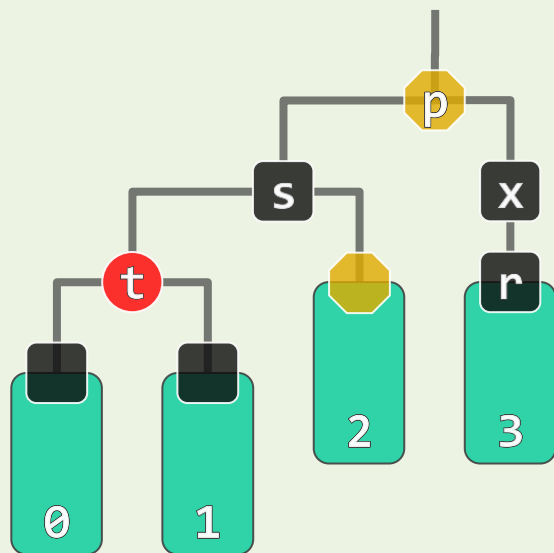
❖ 无非四种情况，分别加以处理...



双黑修正

```
template <typename T> void RedBlack<T>::solveDoubleBlack() {  
    while ( _p ) {  
        BNP<T> s = ( _r == _p->lc ) ? _p->rc : _p->lc; // _r的兄弟s  
        if ( IsBlack( s ) ) // 兄弟s为黑  
            if ( IsRed(s->lc) || IsRed(s->rc) ) { /* 黑s有红孩子: BB-1 */ }  
            else { /* 黑s无红孩子: BB-2R或BB-2B */ }  
        else { /* 兄弟s为红: BB-3 */ }  
    } // while  
} // solveDoubleBlack
```

BB-1: s 为黑, 且至少有一个红孩子 t



❖ rotate(p)

$t \sim a$

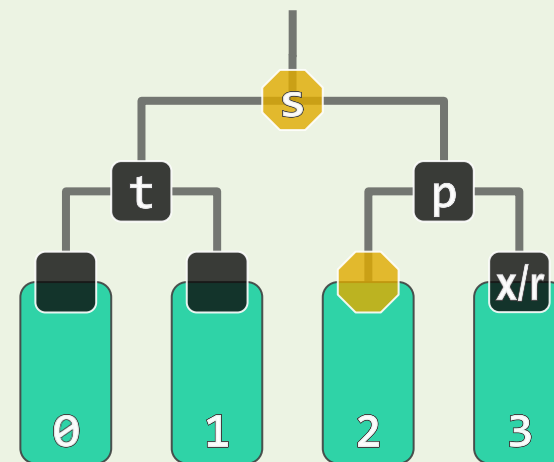
$s \sim b$

$p \sim c$

❖ r 保持黑

a 、 c 染黑

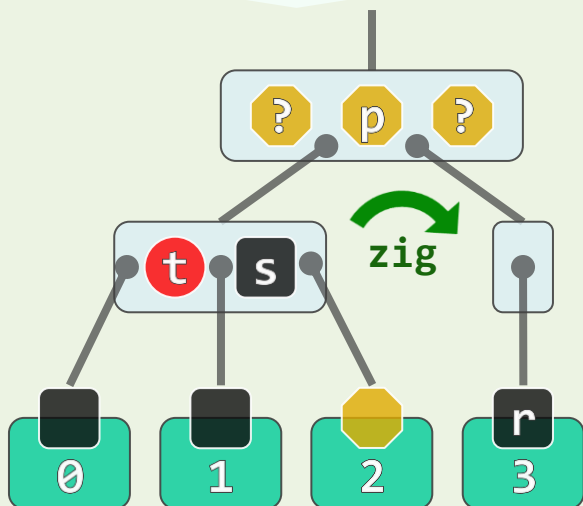
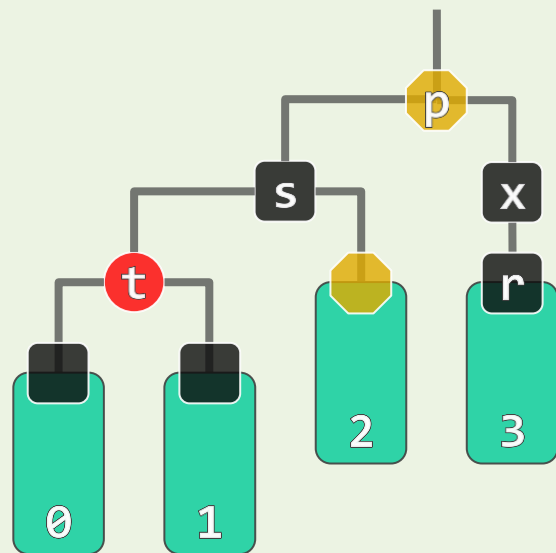
b 继承 p 的原色



❖ 如此, 红黑树性质在全局得以恢复——删除完成!

❖ 在对应的B-树中, 以上操作等效于...

BB-1: 复衡算法之原理



❖ 在对应的B-树中

- 通过关键码的**旋转**
- 消除超级节点的下溢

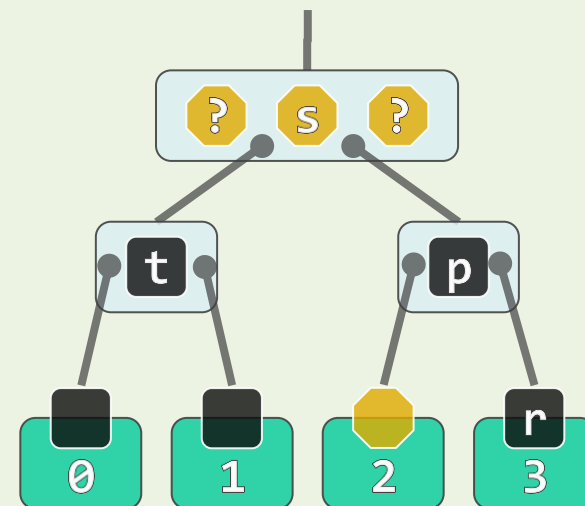
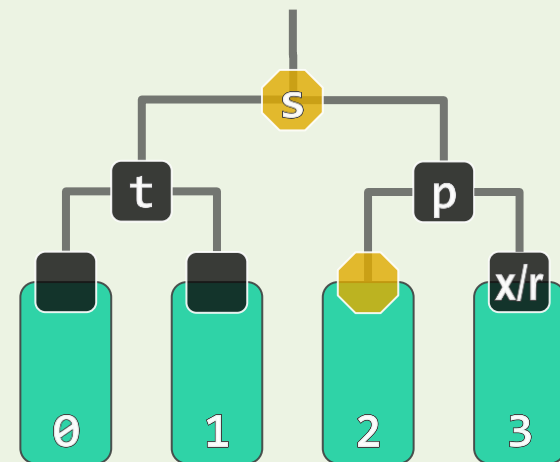
❖ p若为红

问号之一为黑关键码

❖ p若为黑

必自成一个超级节点

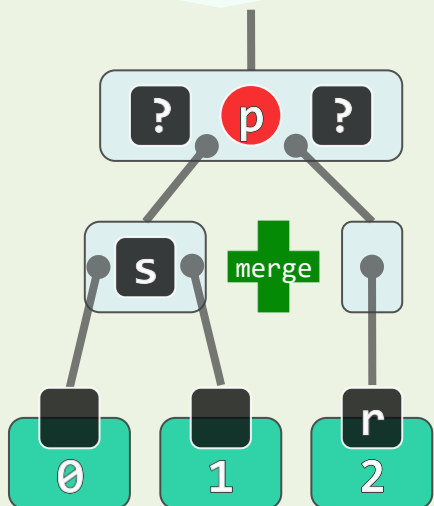
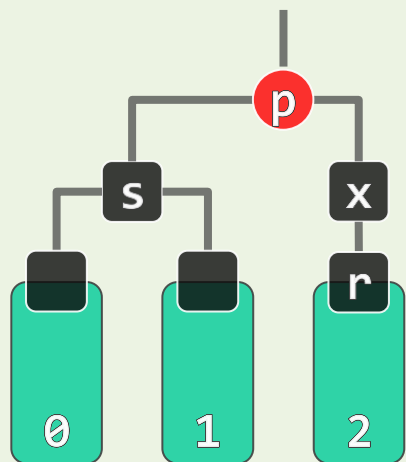
❖ 无论如何，必然**一蹴而就**



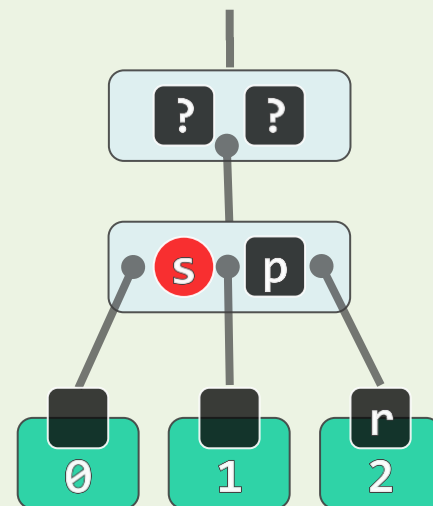
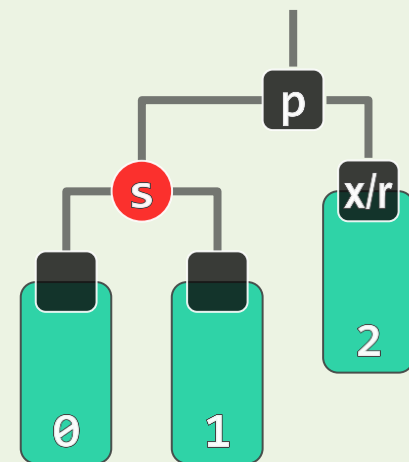
BB-1: 复衡算法之实现

```
while ( _p ) {  
    if ( IsBlack( s ) ) //兄弟s为黑  
        if ( IsRed(s->lc) || IsRed(s->rc) ) { //黑s有红孩子: BB-1  
            BNP<T> b = rotate( _p ); //沿_p、s、t旋转后, 新根继承原根的  
            b->color = _p->color; b->height = _p->height + 1; //颜色与高度  
            b->lc->color = b->rc->color = RBT_BLACK; //左子、右子 (均必非NULL) 染黑  
            break; //一蹴而就  
        } else { /* 黑s无红孩子: BB-2R或BB-2B */ }  
    else { /* 兄弟s为红: BB-3 */ }  
}  
//while
```

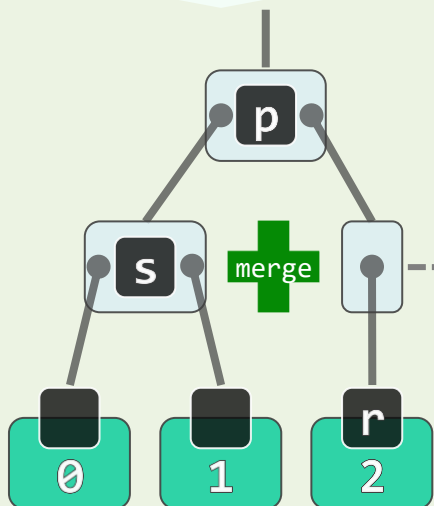
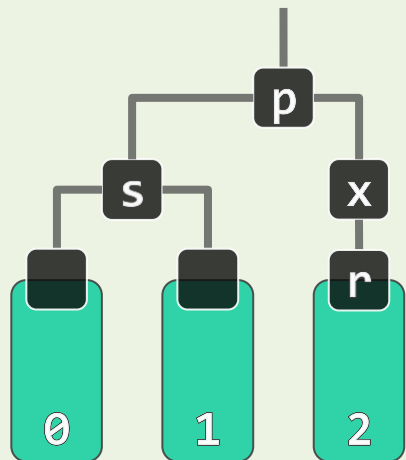
BB-2R: s为黑, 且两个孩子均为黑; p为红



- ❖ r保持黑; s转红; p转黑
- ❖ 在对应的B-树中, 等效于
下溢节点与兄弟合并
- ❖ 红黑树性质在**全局**得以恢复
- ❖ 失去关键码p后, 上层节点
不会继而下溢, **一蹴而就!**
- ❖ 合并之前, 在p之左或右侧
还应有一个黑关键码



BB-2B: s为黑, 且两个孩子均为黑; p为黑



❖ s转红; r与p保持黑

- 红黑树性质在局部得以恢复
- B-树中, 下溢节点与兄弟合并
- 红黑树并无旋转, 结构不变

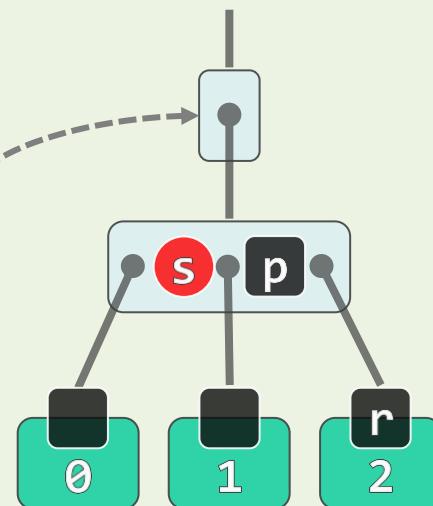
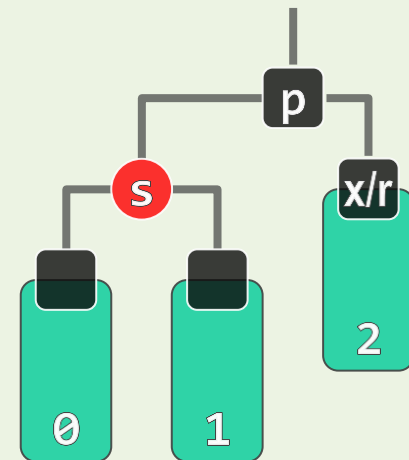
❖ 可等效理解为, 下溢节点此前拥有一个关键字, 以及另一棵子树...

❖ 合并前, p和s均属于单关键字节点

孩子的下溢修复后, 父节点必然继而下溢

❖ 好在可继续分情况处理

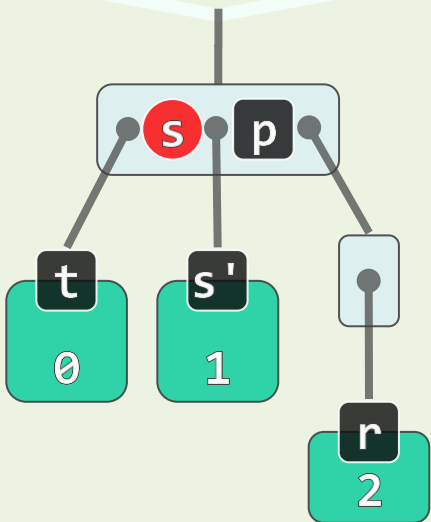
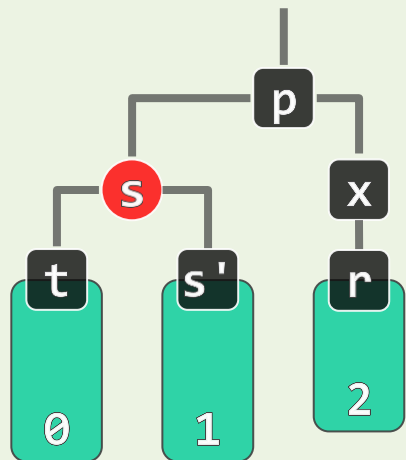
高度递增, 至多 $\mathcal{O}(\log n)$ 层



BB-2(R+B): 复衡算法之实现

```
while ( _p ) {  
    if ( IsBlack( s ) ) //兄弟s为黑  
        if ( IsRed(s->lc) || IsRed(s->rc) ) { //黑s有红孩子: BB-1  
        } else { //黑s无红孩子: BB-2R或BB-2B  
            s->color = RBT_RED; _p->height--; //s转红, _p坍塌一层  
  
            if ( IsRed( _p ) ) { _p->color = RBT_BLACK; break; } //BB-2R: _p转黑  
  
            else { _r = _p; _p = _r->parent; } //BB-2B: 黑_p的黑高度下降; 继续上溯  
        } //else  
    else { /* 兄弟s为红: BB-3 */ }  
}  
//while
```

BB-3: s为红 (其孩子均为黑)



❖ rotate(p)

s红转黑, p黑转红

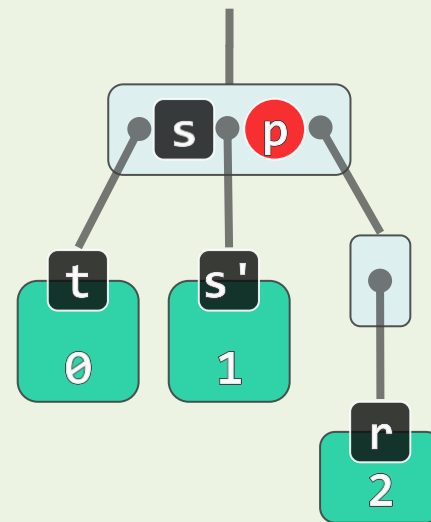
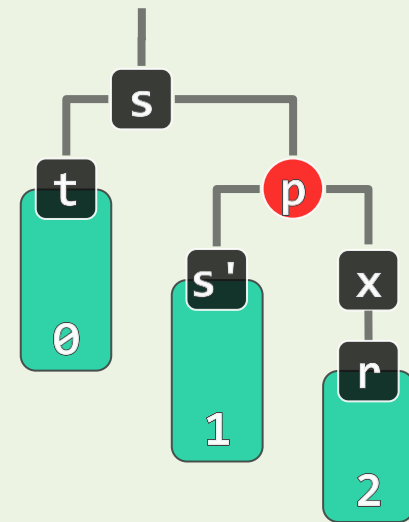
- 黑高度依然异常, 但...
- r有了一个新的黑兄弟s'
- 故转化为前述情况, 而且...

❖ 既然p已转红, 接下来

- 绝不会是BB-2B
- 而只能是...BB-2R 或 BB-1

❖ 于是, 再经一轮调整

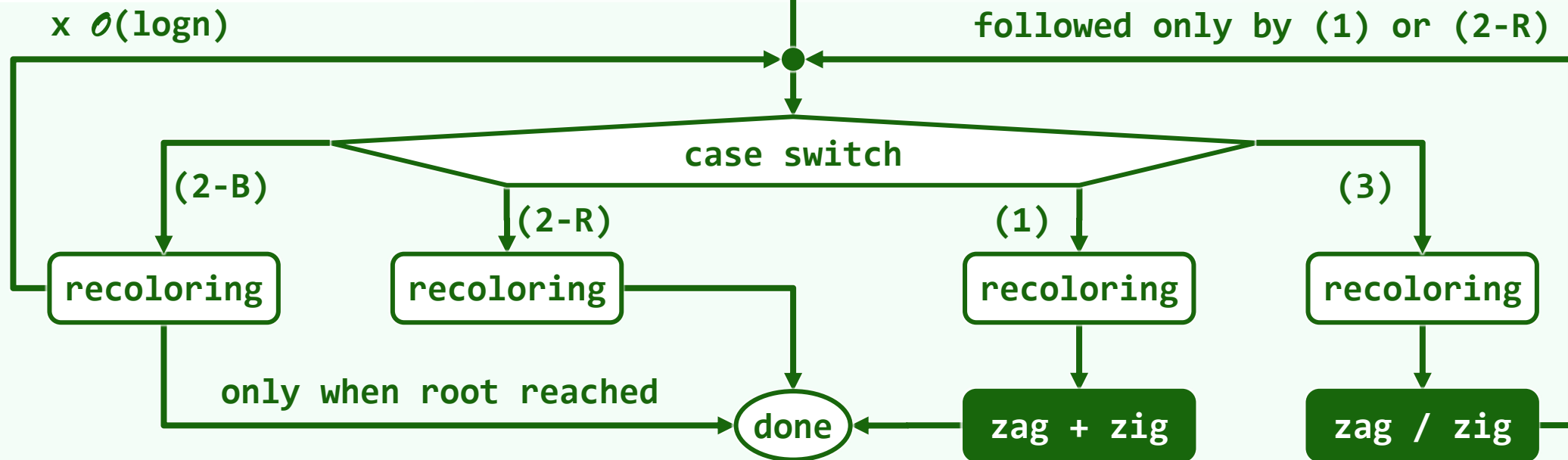
红黑树性质必然全局恢复



BB-3: 复衡算法之实现

```
while ( _p ) {  
    if ( IsBlack( s ) ) //兄弟s为黑  
        if ( IsRed(s->lc) || IsRed(s->rc) ) { //黑s有红孩子: BB-1  
        } else { //黑s无红孩子: BB-2R或BB-2B  
    else { //兄弟s为红: BB-3  
        s->color = RBT_BLACK; //s转黑  
        _p->color = RBT_RED; //_p转红  
        rotate( _p ); //沿_p、s、t旋转（接下来只能是BB-1或BB-2R）后，继续上溯  
    } //else  
}  
} //while
```

复杂度



❖ `RedBlack<T>::remove()`

仅需 $O(\log n)$ 时间

- $O(\log n)$ 次重染色
- $O(1)$ 次旋转

	旋转	染色	此后
(1) 黑s有红子t	1~2	3	调整随即完成
(2R) 黑s无红子, p红	0	2	调整随即完成
(2B) 黑s无红子, p黑	0	1	必再次双黑, 但将上升一层
(3) 红s	1	2	转为(1)或(2R)

08-D1

高级搜索树

跳转表：构思与结构

去沿江上下，或二十里，或三十里，選高阜處
置一烽火臺，每臺用五十軍守之

邓俊辉

deng@tsinghua.edu.cn

动机与思路

❖ [William Pugh, 1989]

Skip Lists: A Probabilistic **ALTERNATIVE** to Balanced Trees

❖ 基于朴素的**List**结构, 采取**随机**策略

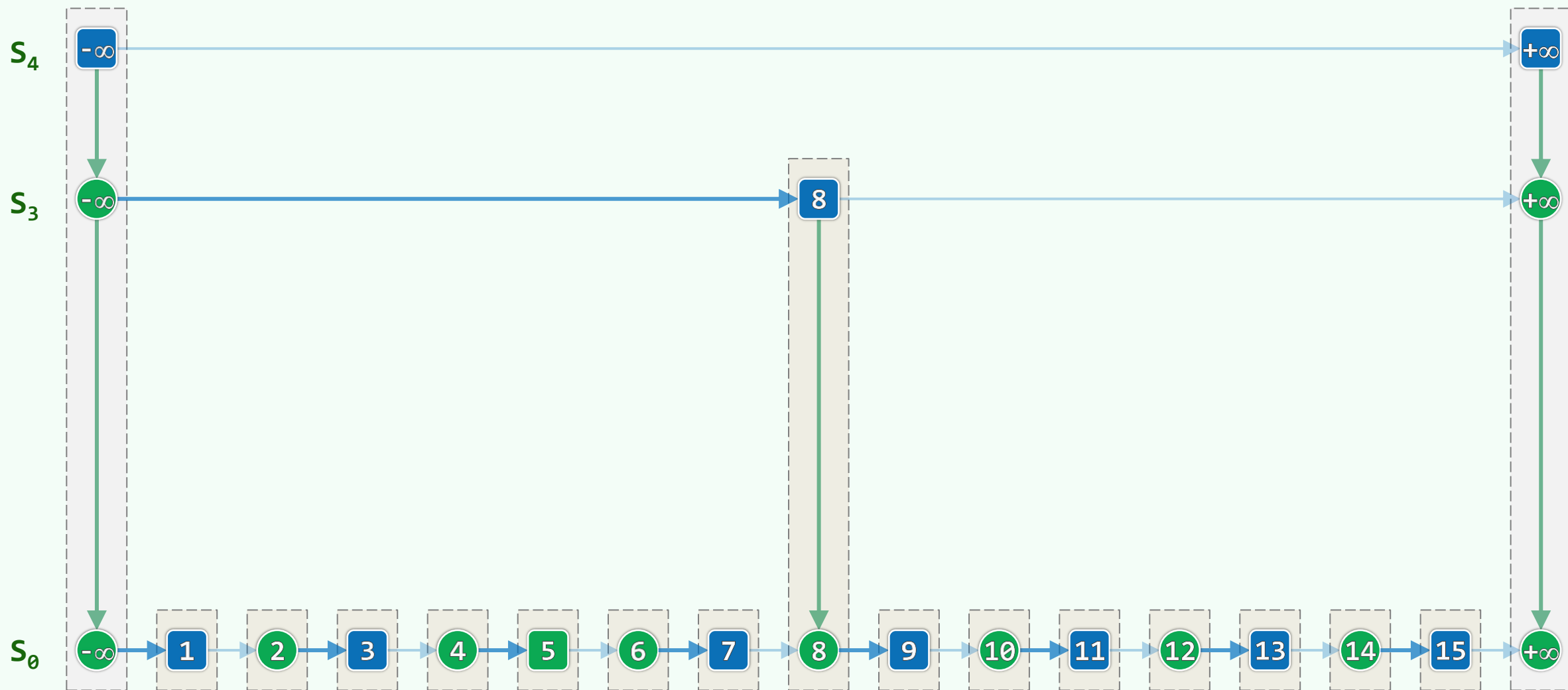
- 从**数学期望**的角度控制查找长度
- 而不再如BBST那样**严格**地控制

❖ 较之BBST, insert/remove操作

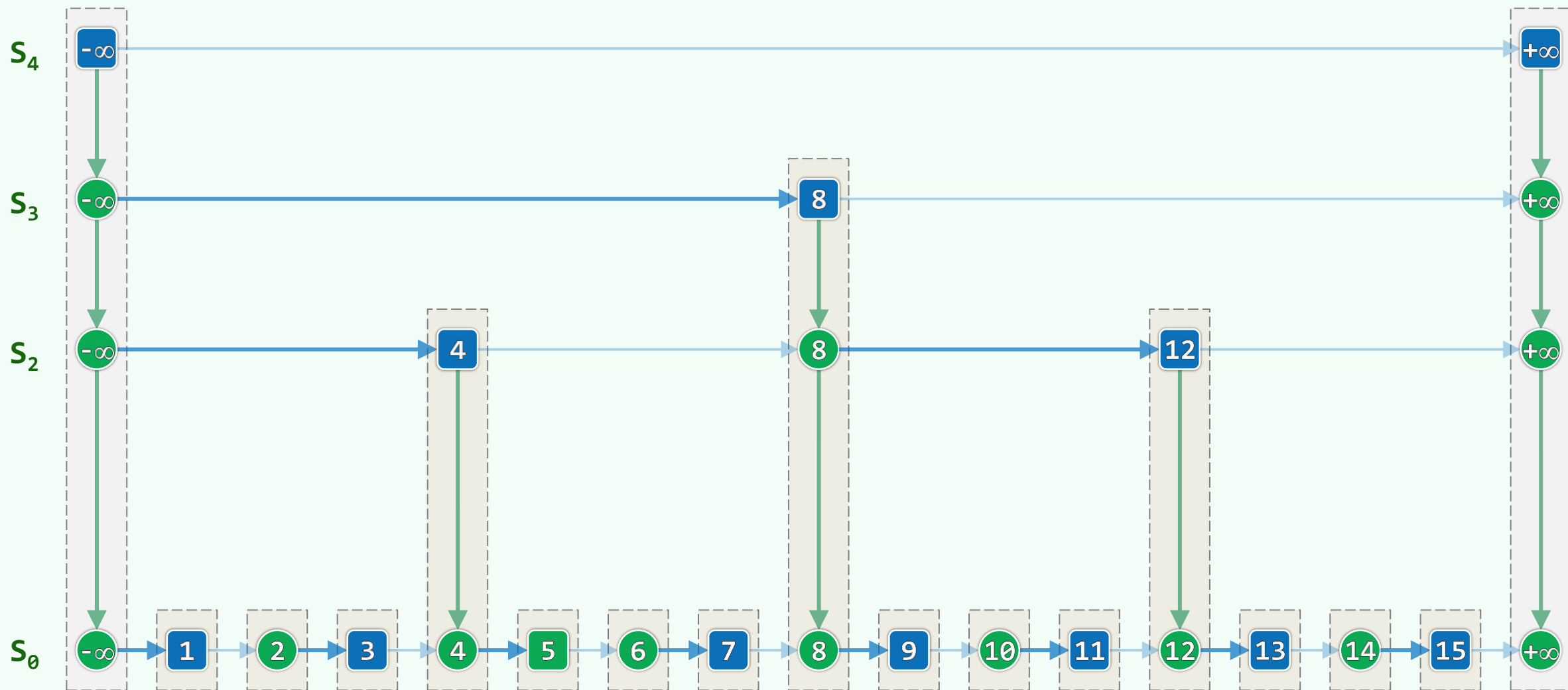
- **实现**更加简捷
- **性能**显著提高



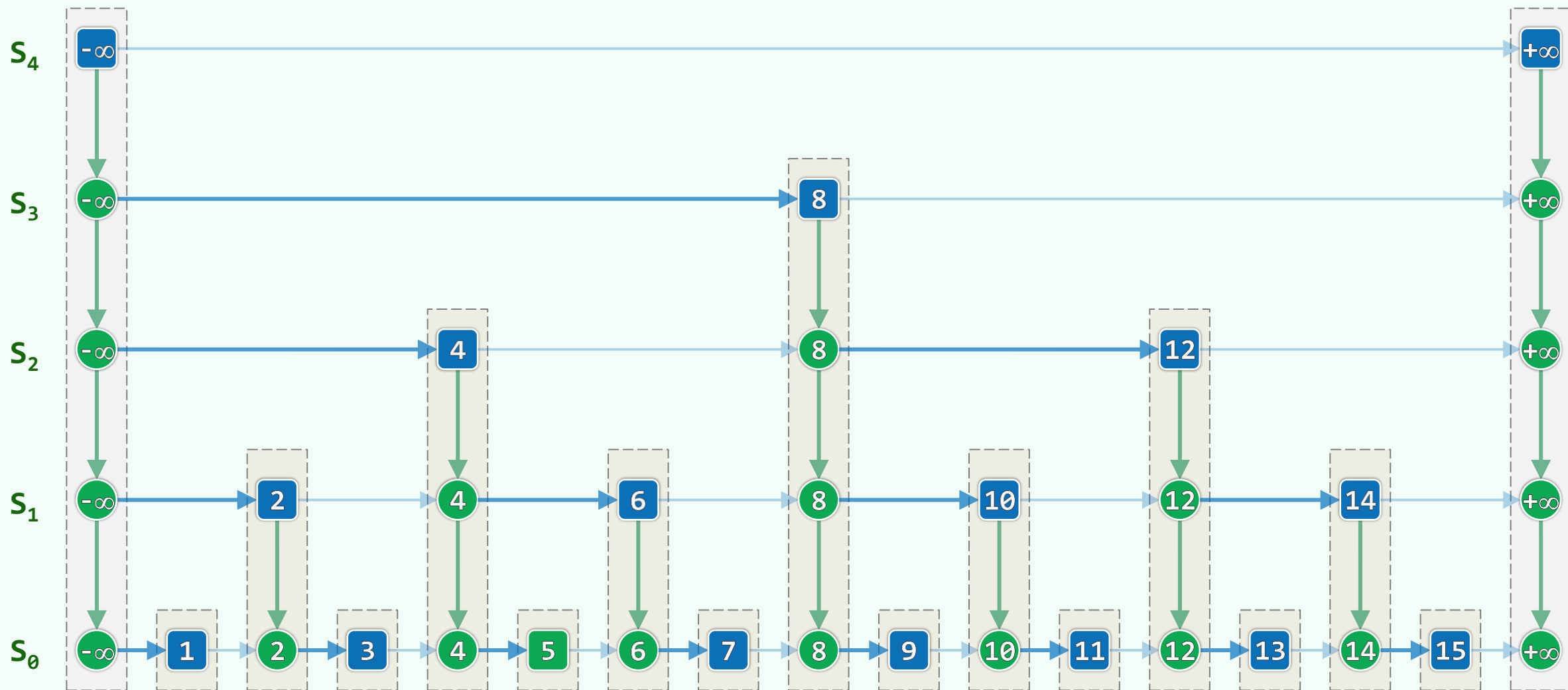
构思：加一层捷径，查找时间 $\rightarrow O(1 + n/2)$



构思：再加上一层，继续细分 --> $O(2 + n/4)$



构思：持续细分，直到 $\log n$ 层 $\rightarrow O(\log n + n/n)$



结构：理想 + 确定

❖ 逐层折半地**抽稀**的列表： $S_0, S_1, \dots, S_h$

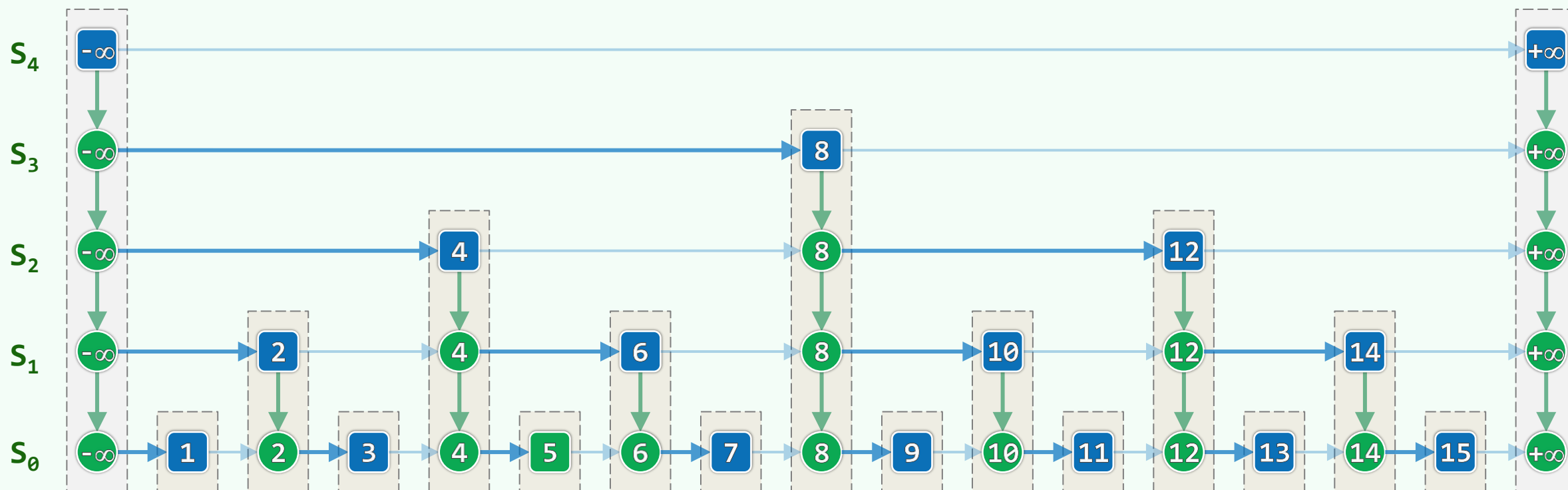
依次叠放并**耦合**： S_0/S_h 称作**底/顶层**

❖ 空间 $O(n)$ ， $search()$ 耗时 $O(\log n)$

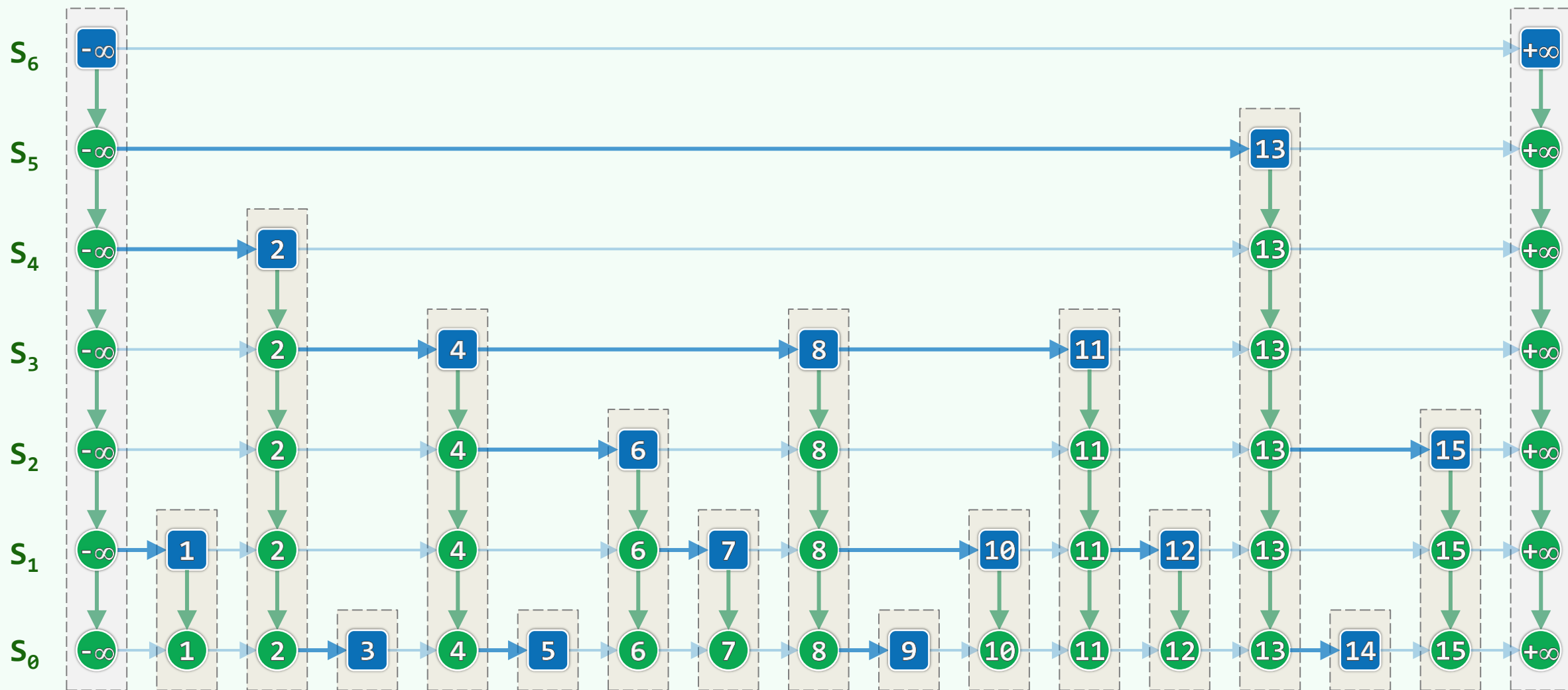
❖ 横向为**层/level**： $prev()$ 、 $next()$ 、 哨兵

❖ 纵向成**塔/tower**： $above()$ 、 $below()$

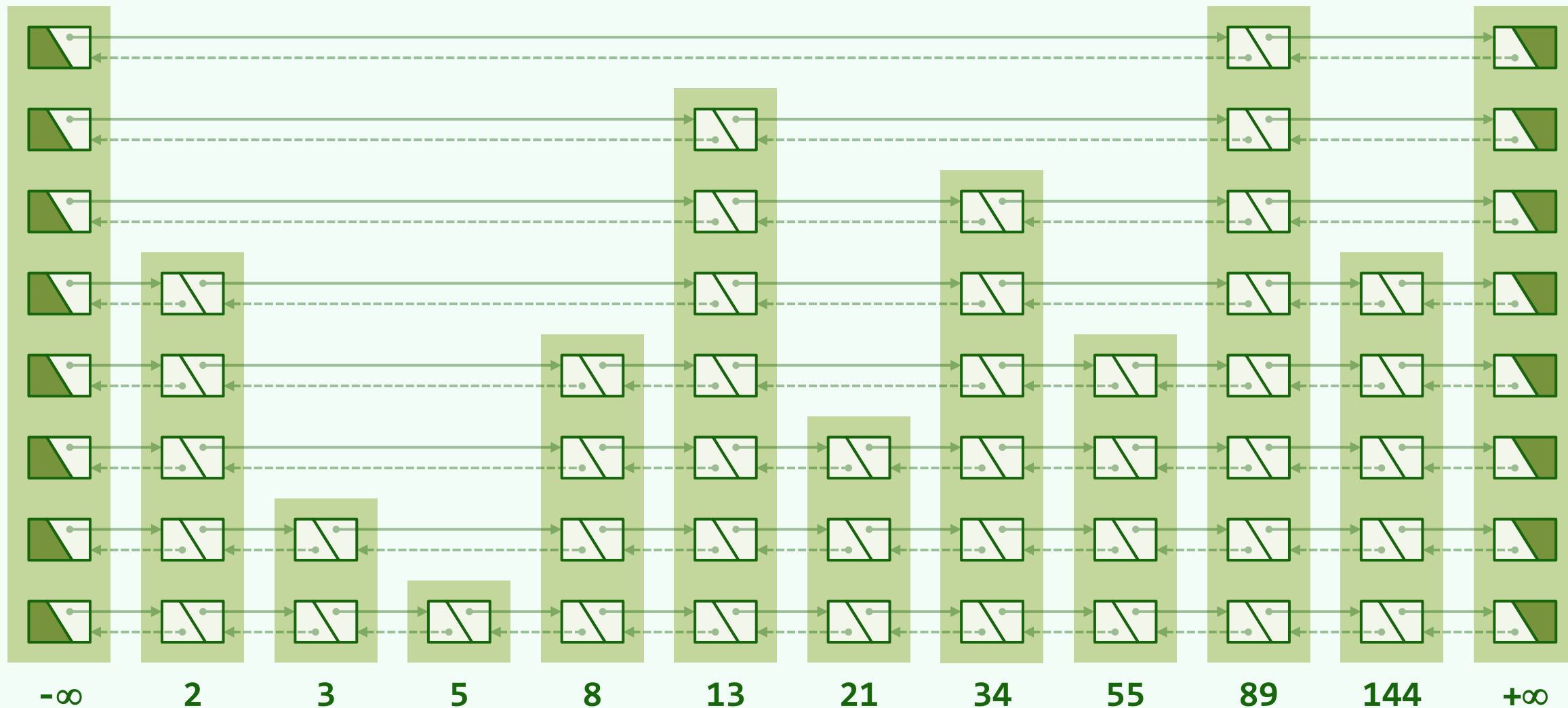
❖ 然而，如何应对...**动态**操作呢？



结构：实际 + 随机



实现：构思



实现: SkipList = List Of Towers

❖ template <typename T> using TowerPosi = LNP< Tower<T> >;

❖ template <typename T>

struct Skiplist : public List< Tower<T> > {

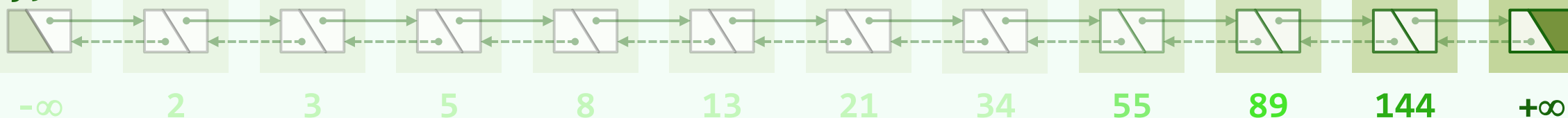
TowerPosi<T> _x; //最近一次操作所在的塔

TowerPosi<T> search(const T&); //查找

TowerPosi<T> insert(const T&); //插入

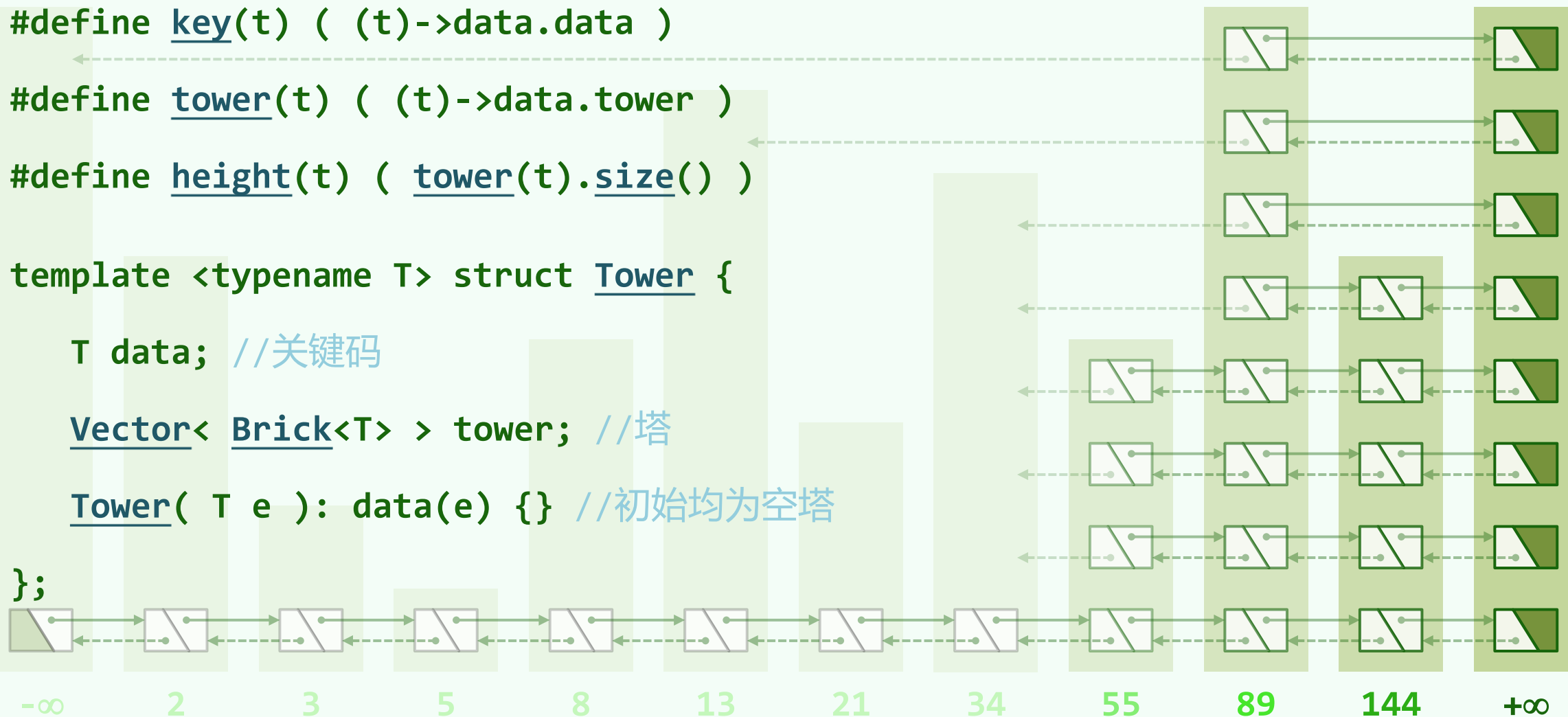
bool remove(const T &); //删除

};



实现: Tower = Vector Of Bricks

```
❖ #define key(t) ( (t)->data.data )  
    #define tower(t) ( (t)->data.tower )  
    #define height(t) ( tower(t).size() )  
  
❖ template <typename T> struct Tower {  
    T data; // 关键词  
    Vector< Brick<T> > tower; // 塔  
    Tower( T e ): data(e) {} // 初始均为空塔  
};
```



实现: Brick = Node With Hands

❖ template <typename T>

```
struct Brick {
```

```
    TowerPosi<T> pred, succ;
```

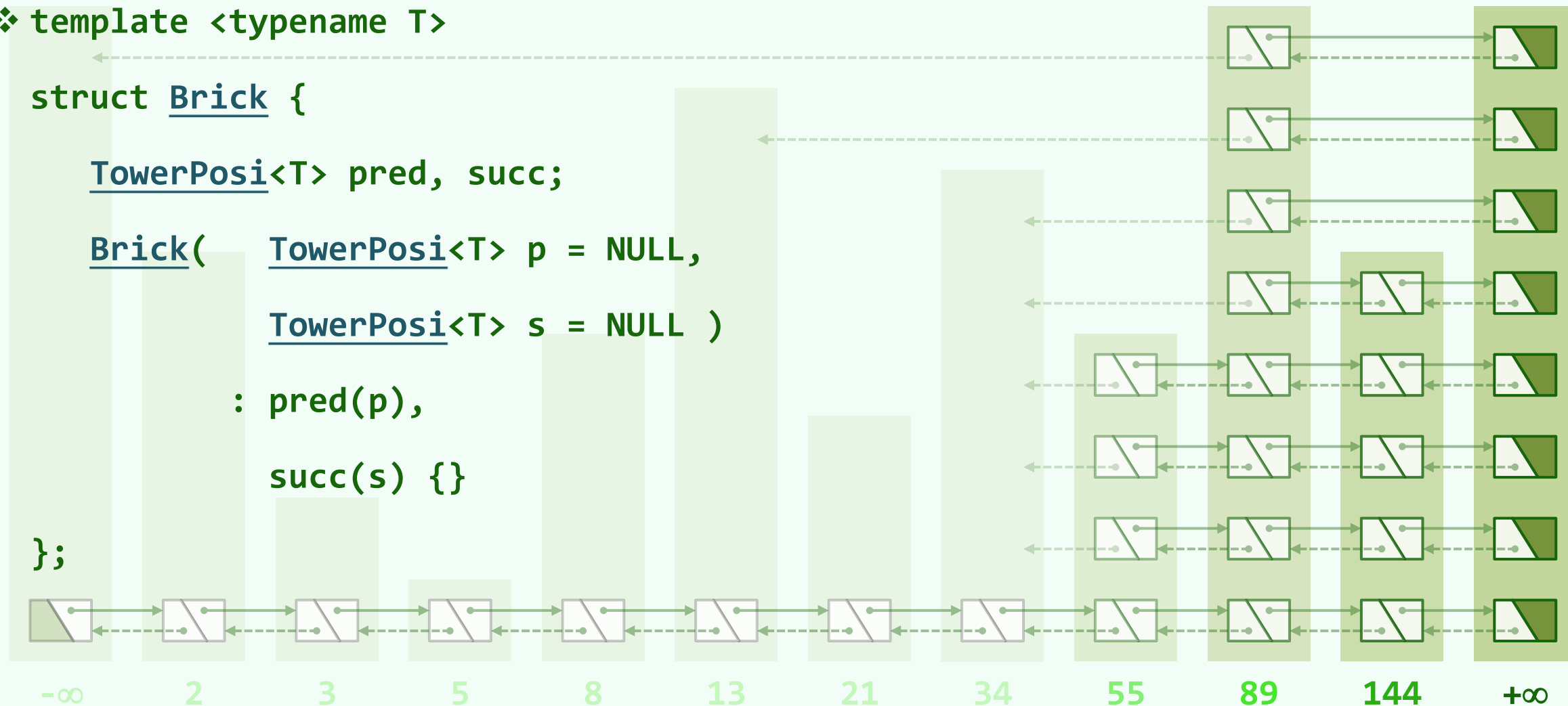
```
    Brick(    TowerPosi<T> p = NULL,
```

```
            TowerPosi<T> s = NULL )
```

```
        : pred(p),
```

```
        succ(s) {}
```

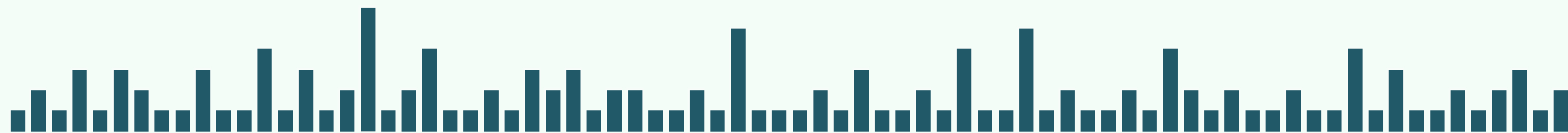
```
};
```



空间性能：统计分析

❖ 较之常规的单层列表，每次操作所需的时间会否因访问节点的增多而实质地增加？

每个节点都至多可能重复 h 份，空间复杂度会否因此有实质增加？ //先来回答后者...



❖ 逐层随机减半： S_k 中的每个关键码有 $p = 1/2$ 的概率，在 S_{k+1} 中依然出现

——稍后将会看到，这一性质可以通过投掷硬币来保证

❖ 于是：所有塔的高度符合几何分布： $Pr(h = k) = p^{k-1} \cdot (1 - p)$

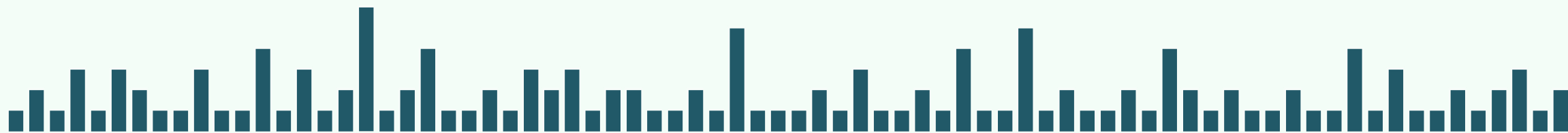
期望的塔高： $\mathbb{E}(h) = 1/(1 - p) = 2$

❖ 什么，没有学过概率？不要紧，有直观的解释...

空间性能：初等估计

❖ 既然逐层随机减半，故 S_0 中任一关键码在 S_k 中依然出现的概率为 2^{-k}

第 k 层节点数的期望值 $\mathbb{E}(|S_k|) = n \cdot 2^{-k} = n/2^k$



❖ 于是，所有节点期望的总数（即各层列表所需空间总和）为

$$\mathbb{E}(\sum_k |S_k|) = \sum_k \mathbb{E}(|S_k|) = n \times \sum_k 2^{-k} < 2n = \mathcal{O}(n)$$

❖ 结论：跳转表所需空间为 expected- $\mathcal{O}(n)$

类比：半衰期为1年的放射性物质中，各粒子的平均寿命不过2年

❖ 更为细致地，塔高的方差是否足够小？ // 比照稍后对层高的分析

08-D2

高级搜索树

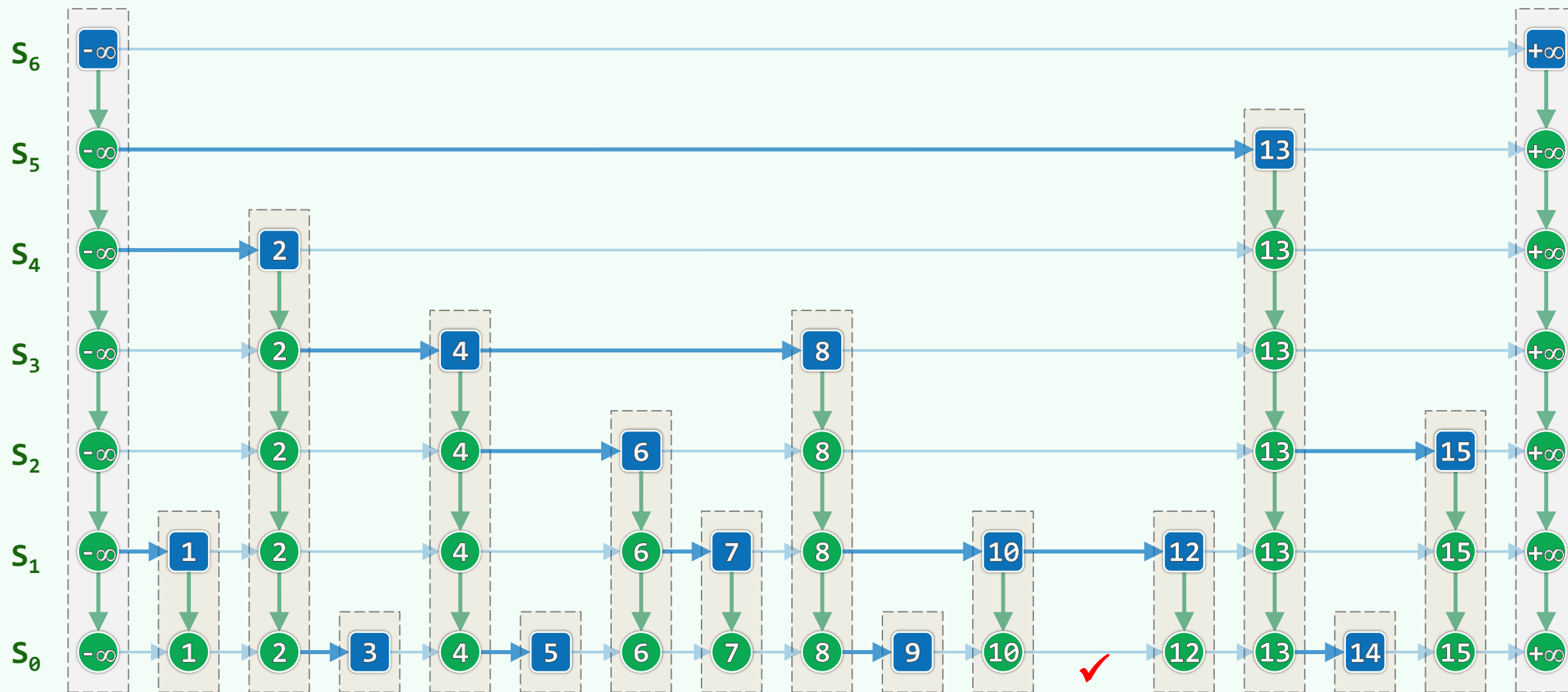
跳转表：插入与删除

如果一个人遇到不可解之事，把脑子想穿了，也找不到其中的原因，怎么办呢？
他或许会去庙里烧香，把自己的难题交给算命先生，听任他们的摆布

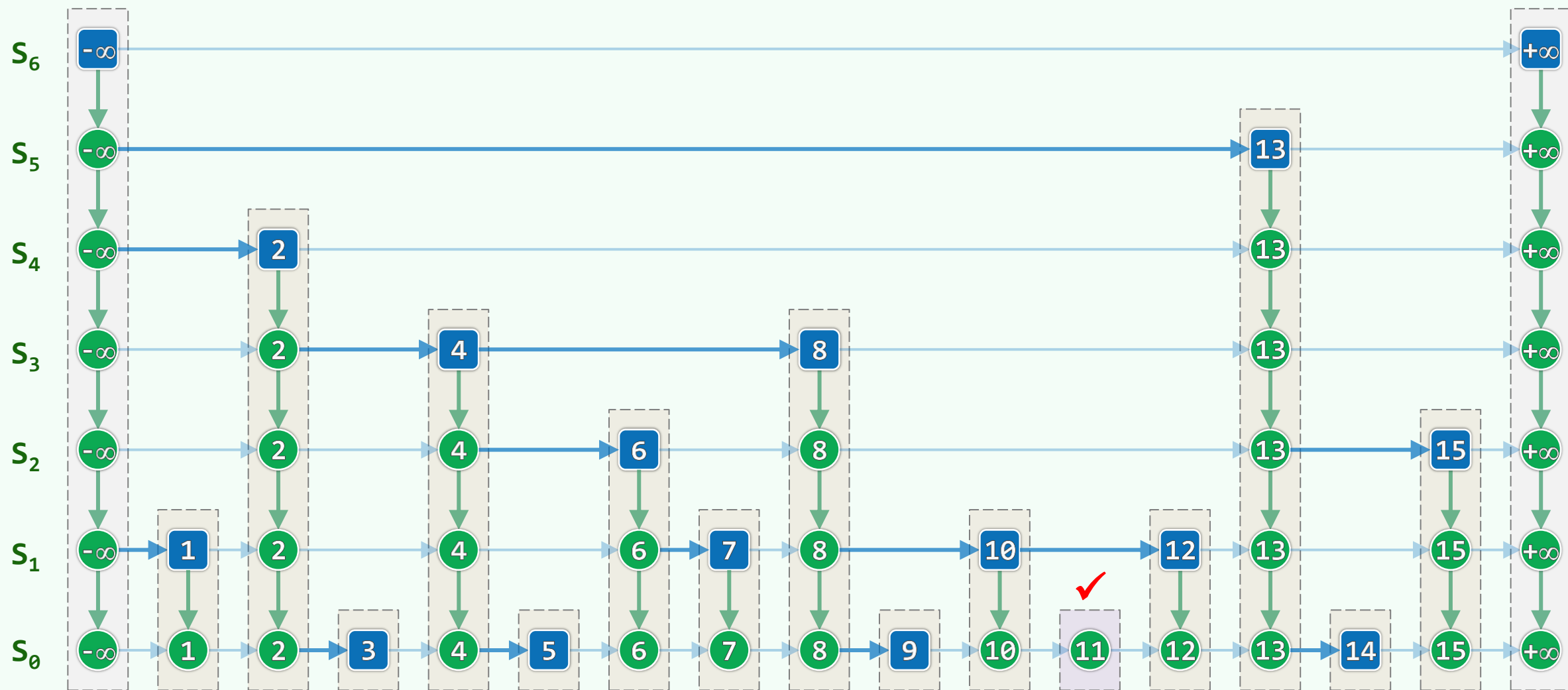
邓俊辉

deng@tsinghua.edu.cn

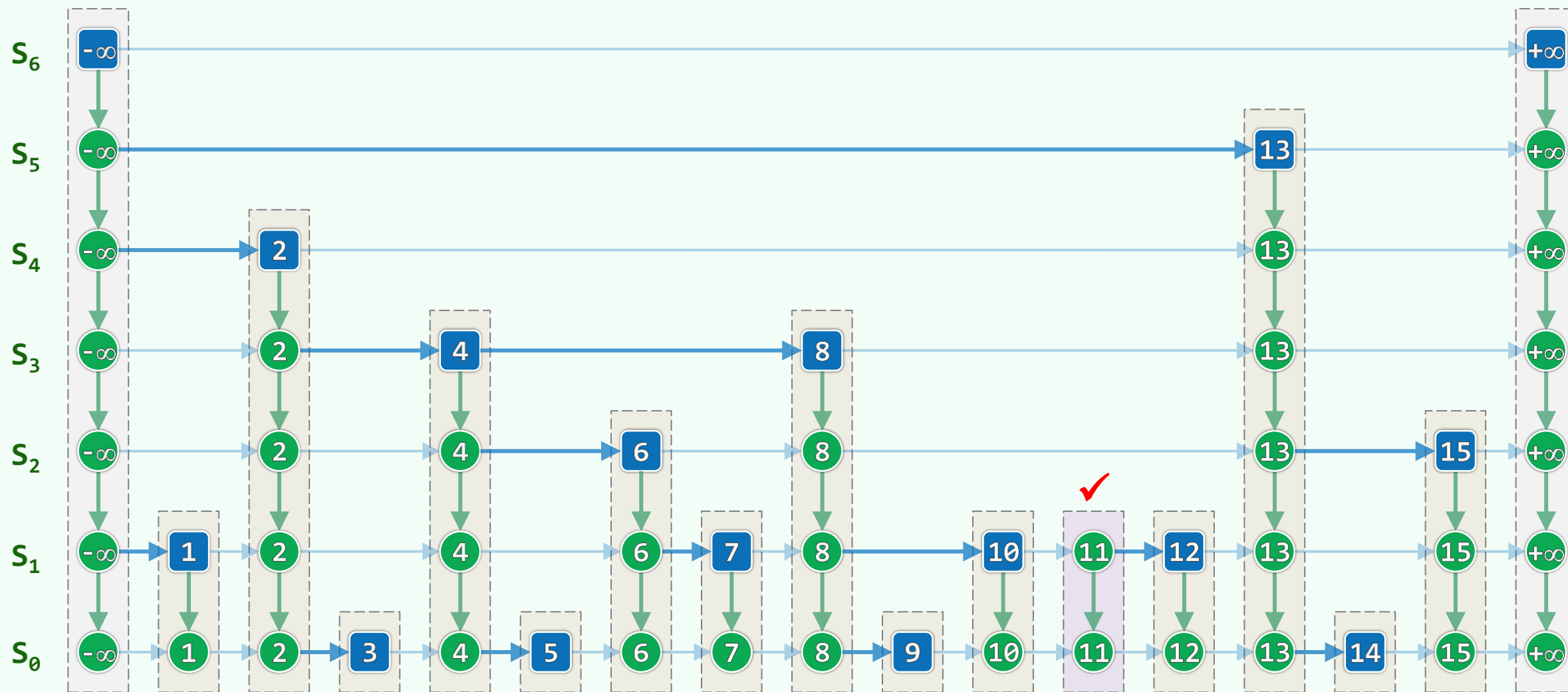
实例: $\text{insert}(11) \sim 0 = \text{remove}(11) \sim 4$



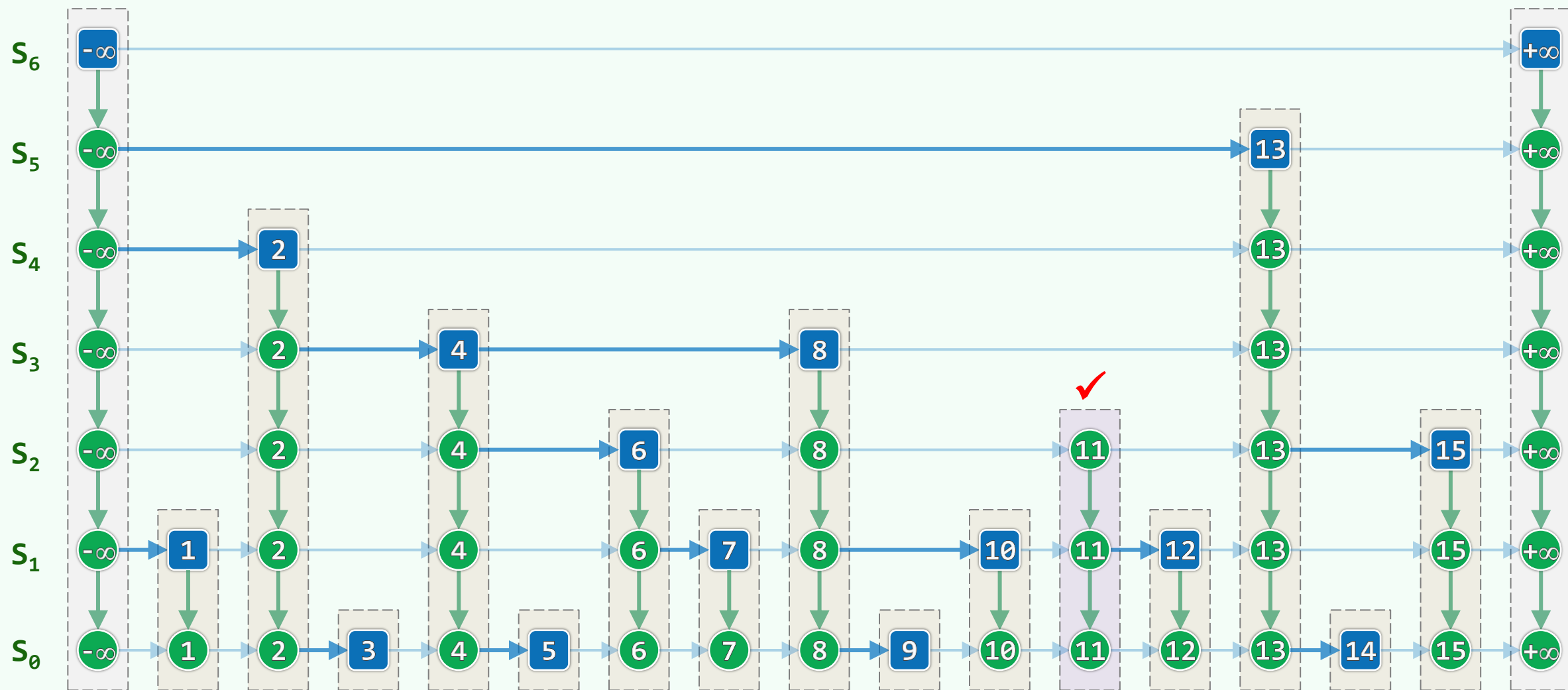
实例: $\text{insert}(11) \sim 1 = \text{remove}(11) \sim 3$



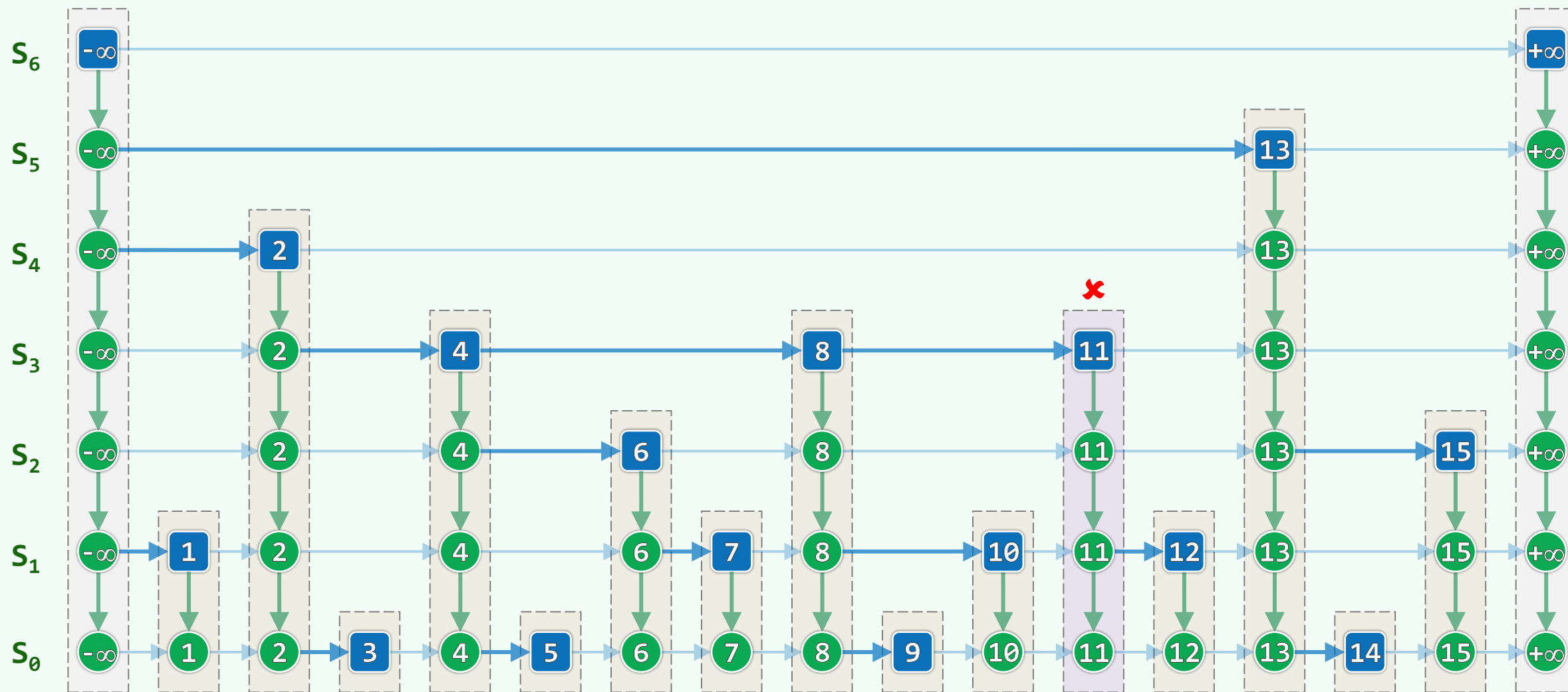
实例: $\text{insert}(11) \sim 2 = \text{remove}(11) \sim 2$



实例: $\text{insert}(11) \sim 3 = \text{remove}(11) \sim 1$



实例: $\text{insert}(11) \sim 4 = \text{remove}(11) \sim 0$



插入算法：整体

```
template <typename T> TowerPosi<T> Skiplist<T>::insert( const T & e ) {  
    search( e ); if ( ( _x != head ) && ( e == key(_x) ) ) return _x; //禁止重复  
    TowerPosi<T> pred = _x, succ = _x->succ; //同层的前驱 (或为head)、后继 (或为tail)  
    _x = List::insert( _x, Tower<T>( e ) ); //紧邻于_x之右侧, 创建一座空塔  
    for ( Rank h = 0; !h || (rand() & 1); h++ ) { //积砖成塔 (至少一层)  
        /* ... 紧邻于_x右侧, 一座新塔逐层长成 ... */  
    } //for  
    return _x; //返回新塔  
} //insert
```

插入算法：建塔

```
for ( Rank h = 0; !h || (rand() & 1); h++ ) { //积砖成塔 (至少一层)

    if ( height(head) < h+1 ) { //若总高度不足h, 则需创建一层 (仅含两个哨兵)

        tower(head).insert( Brick<T>( NULL, tail ) );
        tower(tail).insert( Brick<T>( head, NULL ) );

    }

    while ( height(pred) == h ) pred = tower(pred)[h-1].pred; //踏过若干塔顶
    while ( height(succ) == h ) succ = tower(succ)[h-1].succ; //找到左邻和右邻

    tower(_x).insert( Brick<T>( pred, succ ) ); //新塔长高一层

    tower(pred)[h].succ = tower(succ)[h].pred = _x; //左右连接

} //for
```

删除算法

```
template <typename T> bool Skiplist<T>::remove( const T & e ) {  
    search( e ); if ( ( _x == head ) || ( e != key(_x) ) ) return false; //不存在  
    for ( Rank h = height(_x) - 1; -1 != h; h-- ) { //自顶而下, 逐层拆塔  
        Brick<T> & b = tower(_x)[h];  
        tower(b.pred)[h].succ = b.succ; tower(b.succ)[h].pred = b.pred;  
        if ( head == b.pred && b.succ == tail ) //及时清除空层  
            { tower(head).remove(h); tower(tail).remove(h); }  
    }  
    List::remove( _x ); return true; //删除成功  
} //remove
```

高级搜索树

跳转表：查找及复杂度

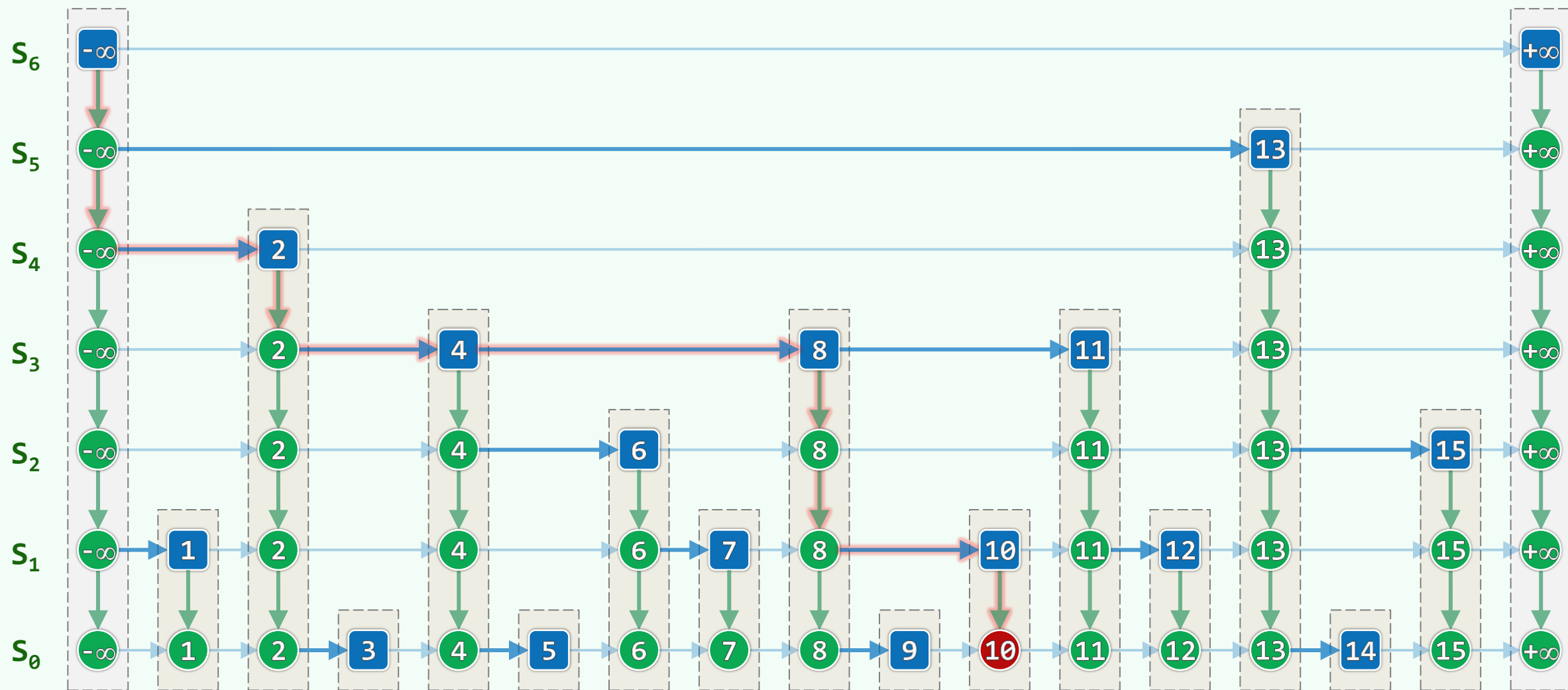
她的脚实在娇嫩，从来不下地
而在人们头顶上踱来踱去

只见参仙老怪梁子翁笑嘻嘻的站起身来，向众人拱了拱手，缓步走到庭中，忽地跃起，左足探出，已落在欧阳克插在雪地的筷子之上，拉开架子...把一路巧打连绵的“燕青拳”使了出来，脚下纵跳如飞，每一步都落在竖直的筷子之上

邓俊辉

deng@tsinghua.edu.cn

算法：由高到低，由粗到细

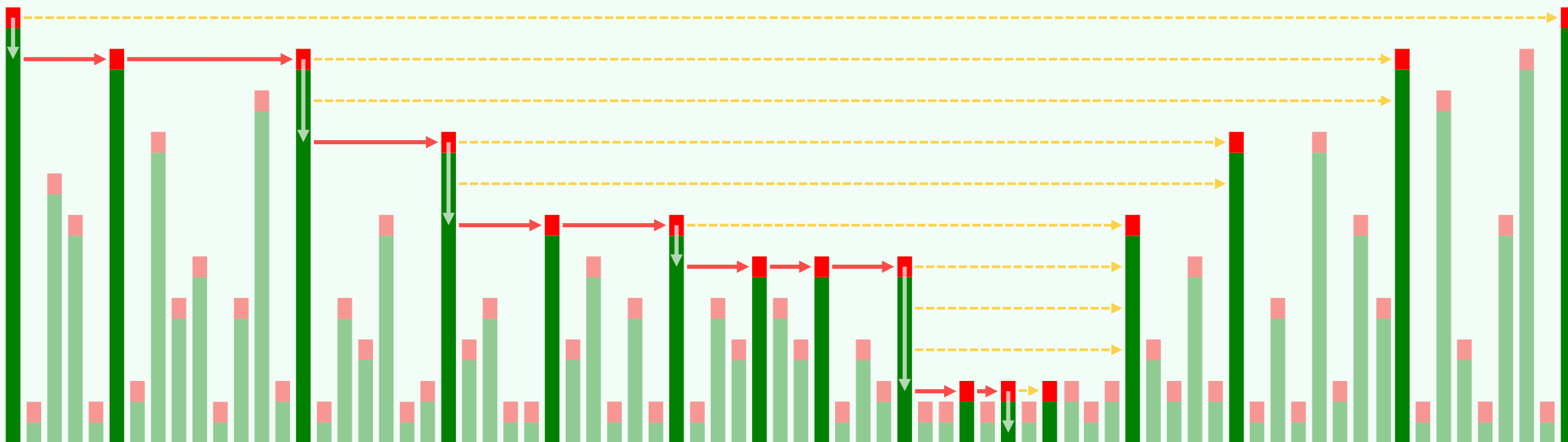


算法：实现

```
template <typename T> TowerPosi<T> Skiplist<T>::search( const T & e ) {  
    _x = head; //从head塔的顶层出发  
    for ( Rank h = height(_x) - 1; -1 != h; h-- ) //逐层下降，直至底层  
        while ( ( tower(_x)[h].succ != tail ) && ( key(tower(_x)[h].succ) <= e ) )  
            _x = tower(_x)[h].succ; //在每一层，都尽可能右移  
    return _x == head ? NULL : _x; //不大于e的最后一座塔 (head ~ NULL)  
} //search
```

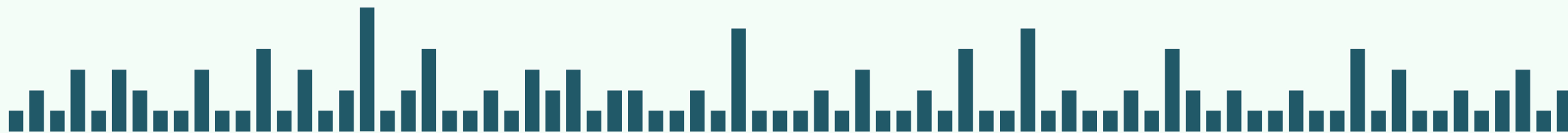
算法：总览

- ❖ 查找时间取决于**横向**、**纵向**的累计跳转次数
- ❖ 首先，会否因**层次**过多，导致**纵向**跳转过多？



纵向跳转 ~ 层高

❖ 观察：一座塔高度**不低于** k 的概率为 p^k ；**低于** k 的概率为 $1 - p^k$



❖ 引理：随着 k 的增加， S_k **为空的概率急剧上升**，**非空的概率急剧下降**

$$Pr(|S_k| = 0) = (1 - p^k)^n \geq 1 - n \cdot p^k \qquad Pr(|S_k| > 0) \leq n \cdot p^k$$

❖ 推论：跳转表高度 $h = \mathcal{O}(\log n)$ 的概率**极大**

❖ 比如：若 $p = 1/2$ ，则第 $k = 3 \cdot \log_{1/p} n$ 层**非空**（当且仅当 $h \geq k$ ）的概率为

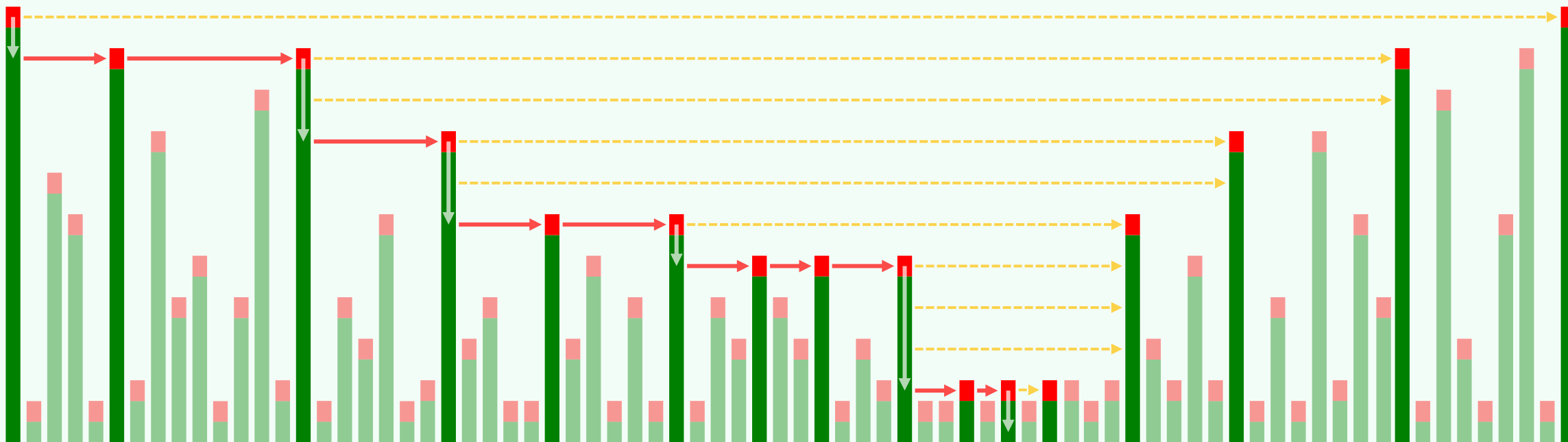
$$Pr(|S_k| > 0) \leq n \cdot p^k = n \cdot n^{-3} = 1/n^2 \mapsto 0 \quad //w.h.p.$$

❖ 结论：查找过程中，**纵向跳转的次数**，累计不过 $\text{expected-}\mathcal{O}(\log n)$

横向跳转 ~ 紧邻塔顶

❖ 那么：横向跳转是否可能很多次？比如 $\omega(\log n)$ ，甚至 $\Omega(n)$ ？

❖ 观察：在同一水平列表中，横向跳转所经节点必然依次**紧邻**，而且每次抵达都是**塔顶**

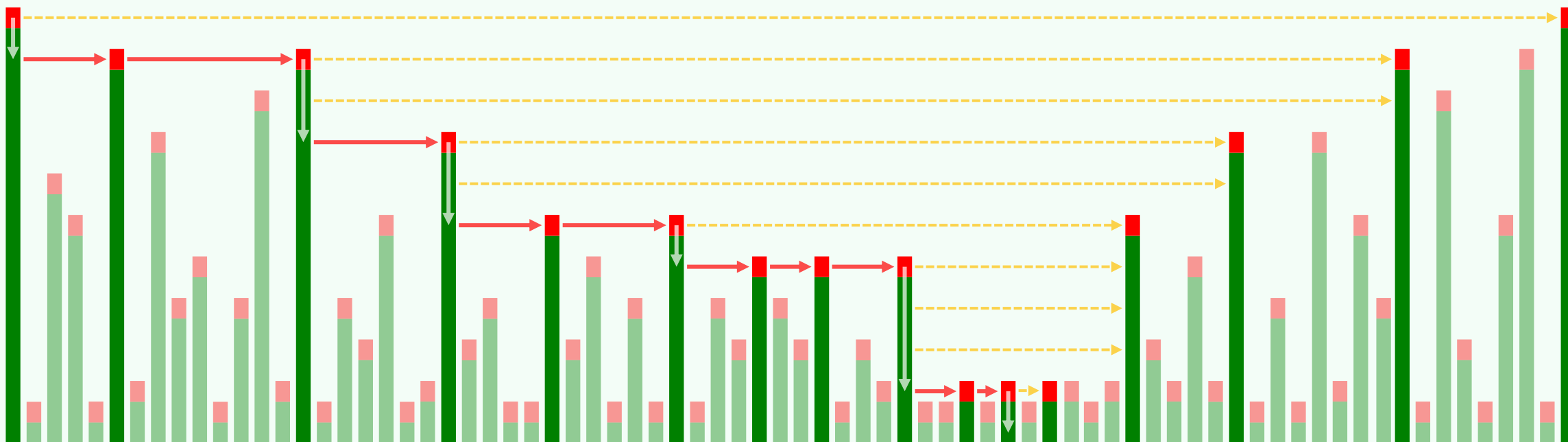


❖ 于是：若将沿同一层列表跳转的次数记作 Y ，则它符合**几何分布** $Pr(Y = k) = (1 - p)^k \cdot p$

横向跳转 ~ 期望时间成本

❖ 定理: $\mathbb{E}(Y) = (1 - p)/p = (1 - 0.5)/0.5 = 1$ 次

❖ 因此: 在同一层列表中连续跳转的期望时间成本 = 1次跳转 + 1次驻足 = 2



❖ 结论: 跳转表的每次查找, 都可在 $\leq \text{expected-}(2h) = \text{expected-}\mathcal{O}(\log n)$ 时间内完成